ZÁRÓDOLGOZAT DOKUMENTÁCIÓ

Nagypapa Házai – Közösségi Ingatlanhasználati Platform

Készítők:

Bota Csongor

Kania Áron Nontawat

Batsukh Temuujin

Viszgavezető tanárok: Szalainé Török Edit Judit, Csukárdi Rajmund, Méhes József György

2026

ASP.NET Core Blazor Server • SQL Server • Entity Framework Core • Bootstrap 5

Kapcsolat: nagypapahazai@gmail.com

TARTALOMJEGYZÉK

1. Bevezetés 4

1.1. A záródolgozat témája 4
1.2. Témaválasztás indoklása 6
1.3. A program funkcióinak ismertetése 8
1.4. Célközönség meghatározása 11
1.5. Hasonló rendszerek összehasonlítása 12

2. Fejlesztői dokumentáció 14

2.1. A program elkészítésének indoklása 14
2.2. Fejlesztőkörnyezet 18
2.2.1. Hardverkörnyezet 18
2.2.2. Szoftverkörnyezet 21
2.3. Programnyelv és fejlesztőkörnyezet választásának indoklása 24
2.4. Projektstruktúra és mappaszerkezet 30
2.5. Adatszerkezet részletes bemutatása 35
2.5.1. Az adatbázis-tervezés elvei 35
2.5.2. Adatbázis táblák részletes leírása 37
2.5.3. Táblák közötti kapcsolatok 47
2.5.4. EF Core konfiguráció – DbContext és migrációk 50
2.6. Az alkalmazás architektúrája 55
2.6.1. N-Tier architektúra 55
2.6.2. Blazor Server működési elve 58
2.6.3. API kommunikáció 60
2.6.4. Autentikáció és jogosultságkezelés 62
2.7. Algoritmusok részletes bemutatása 65
2.7.1. Felhasználói regisztráció és e-mail validáció 65
2.7.2. Bejelentkezés és AuthState kezelés 70
2.7.3. Foglalási ütközésvizsgálat 75
2.7.4. Dashboard statisztikák generálása 80
2.7.5. Admin jóváhagyási folyamat 84
2.7.6. Esemény-regisztrációs kvóta ellenőrzés 87
2.8. Tesztdokumentáció 91
2.8.1. Tesztelési módszertan 91
2.8.2. Tesztkörnyezetek 93
2.8.3. Tesztelői hozzáférés 96
2.8.4. Funkcionális tesztesetek (TC-01 – TC-14) 97
2.8.5. Nem funkcionális tesztek 107
2.9. Fejlesztési lehetőségek 111

2.9.1. E-mail értesítési rendszer ... 111
2.9.2. Mobilalkalmazás (.NET MAUI) ... 113
2.9.3. Fizetési rendszer integráció ... 115
2.9.4. Azure cloud deployment ... 117
2.9.5. Keresési és szűrési funkciók bővítése ... 119
2.9.6. Értékelési és visszajelzési rendszer ... 120

3. Felhasználói dokumentáció ... 121

3.1. Mi a Nagypapa Házai? ... 121
3.2. Regisztráció nélkül vs. regisztrációval elérhető funkciók ... 125
3.3. Szükséges eszközök és szoftverek ... 128
3.4. Telepítés és elindítás ... 132
3.4.1. Előfeltételek telepítése ... 132
3.4.2. A projekt letöltése GitHubról ... 134
3.4.3. Adatbázis konfigurálása ... 135
3.4.4. Az alkalmazás elindítása ... 137
3.4.5. Első bejelentkezés ... 138
3.5. A program részletes használata ... 140
3.5.1. A főoldal ... 140
3.5.2. Regisztráció ... 143
3.5.3. Bejelentkezés ... 147
3.5.4. Ingatlanok böngészése ... 150
3.5.5. Foglalás leadása ... 153
3.5.6. Naptár nézet ... 158
3.5.7. Események megtekintése és regisztráció ... 161
3.5.8. Saját foglalásaim ... 164
3.5.9. Admin felület részletes ismertetése ... 166
3.6. Hibajelzések és magyarázatuk ... 171
3.7. Kapcsolat és információkérés ... 175

4. Összefoglalás és köszönetnyilvánítás ... 177

4.1. A munka értékelése ... 177
4.2. Kihívások és tanulságok ... 178
4.3. A program jövője ... 179
4.4. Köszönetnyilvánítás ... 180

5. Irodalomjegyzék ... 181

1. Bevezetés

1.1. A záródolgozat témája

Jelen záródolgozat a Nagypapa Házai – Közösségi Ingatlanhasználati Platform elnevezésű webalkalmazás teljes körű fejlesztési és felhasználói dokumentációját tartalmazza. A projekt neve nem csupán egy márkanév, hanem egy metafora is: a "nagypapa házai" azok az örökségi értékkel bíró, vidéki helyszínek, amelyek generációk emlékeit hordozzák, és amelyek méltók arra, hogy a közösség számára is elérhetők legyenek.

Az alkalmazás fejlesztése 2025 szeptemberétől 2026 januárjáig tartott. A fejlesztési folyamat Agilis módszertanra épített: kéthetes sprintekben dolgoztunk, és minden sprint végén bemutattuk az elkészült funkciókat. Az első sprint az adatmodell és az API alapjait hozta létre; a második sprint a Blazor frontend alap struktúráját; a harmadik az autentikációs rendszert; a negyedik a foglalási és naptár modult; az ötödik az esemény-szervezési modult; a hatodik az admin felületet; a hetedik a tesztelést és dokumentálást.

A platform technológiai alapját a Microsoft .NET 8 ökoszisztémája képezi. Az ASP.NET Core Blazor Server keretrendszer lehetővé tette, hogy egyetlen C# kódbázisban valósítsuk meg a teljes alkalmazást – a frontend komponensektől az API végpontokon át az adatbázis-hozzáférésig. Az SQL Server Express adatbázis-kezelő és az Entity Framework Core ORM az adattárolás és lekérdezés feladatait látják el, míg a Bootstrap 5 CSS keretrendszer biztosítja a reszponzív, modern megjelenést.

A dokumentáció öt fő fejezetre tagolódik. Az első fejezet (Bevezetés) bemutatja a projekt koncepcióját, motivációját, funkcionalitását és kontextusát. A második fejezet (Fejlesztői dokumentáció) részletesen tárgyalja a technológiai döntéseket, az adatbázis-struktúrát, az architektúrát, az algoritmusokat, a tesztelést és a fejlesztési lehetőségeket. A harmadik fejezet (Felhasználói dokumentáció) a legegyszerűbb felhasználók számára is érthető útmutatást nyújt a rendszer telepítéséhez és használatához. A negyedik fejezet (Összefoglalás) a munka értékelését, a kihívásokat és a jövőbeli terveket tartalmazza. Az ötödik fejezet (Irodalomjegyzék) a felhasznált irodalom és online források listáját tartalmazza.

A záródolgozat az infojegyzet.hu weboldalon közzétett formai követelmények szerint készült, amelyek meghatározzák a betűméretet (12pt törzs, 14pt H3, 16pt H2, 18pt H1), a sortávot (1,5-es másfeles sortáv), az igazítást (sorkizárt, JUSTIFIED), a fejléc és lábléc tartalmát, a fejezetek elrendezését (minden fő fejezet új oldalon kezdődik) és a hierarchikus fejezetszámozást. A dokumentum terjedelme meghaladja a 180 oldalt.

A fejlesztési munka három fejlesztő közös alkotása: Bota Csongor elsősorban az API backend fejlesztéséért és az adatbázis-tervezésért felelt; Kania Áron Nontawat a Blazor Server frontend fejlesztéséért és a Bootstrap alapú UI kialakításáért; Batsukh Temuujin az autentikációs rendszer, a tesztelés és a dokumentáció megírásáért. A feladatkörök nem voltak merev határokkal elválasztva: a csapat tagjai szükség szerint segítségek egymásnak a különböző területeken.

1.2. Témaválasztás indoklása

A Nagypapa Házai projekt ötlete 2024 nyarán született, amikor a három fejlesztő egy nyári szabadtéri rendezvényen töltötte az idejét, és szóba kerültek a vidéki Magyarország kihasználatlan ingatlanjai. Bota Csongor mesélt nagyapja bakonyi faházáról, amelyet a nagyszülők halála óta senki sem látogat, de a fenntartása évente komoly kiadást jelent. Kania Áron elmesélte, hogy szülei balatonlellei nyaralóját évente alig két-három hétben használják, a többi időben üresen áll. Batsukh Temuujin mongoliai felmenőinek tanulságairól is szólt: a nomád kultúrában a vendégszeretet és a közösségi vagyonmegosztás természetes értékek.

Az az ötlet született, hogy miért ne lehetne egy platformot létrehozni, ahol az ilyen kihasználatlan vidéki ingatlanok tulajdonosai megoszthatják ingatlanaikat a közösséggel – nem kereskedelmi célból, nem pénzért, hanem csereajánlat alapon? Ez az ötlet megvalósíthatónak tűnt, és egyben egy záródolgozat-projekt keretében kivitelezhető feladatot is adott.

A projekt nevét Bota Csongor javasolta: "Nagypapa Házai". A névben két réteg van: egyrészt egy konkrét utalás arra, hogy a platform elsősorban olyan ingatlanokat céloz, amelyek az idősebb generációtól örökölt, vidéki, hagyományos értékeket hordoznak; másrészt egy érzelmi töltet, amely emlékeztet arra, hogy a ház több mint

egy épület – egy história, egy közösség, egy hely, ahol az emberek generációk óta összetalálkoznak.

A témaválasztást szakmai szempontok is megerősítik. Az N-Tier architektúra, a REST API tervezés, az EF Core Code-First fejlesztés, a Blazor Server SignalR kommunikáció és az autentikációs rendszer mind olyan technológiai elemek, amelyeket a záródolgozat-elvárások az alkalmazásfejlesztési szakon megkívánt kompetenciák demonstrálására alkalmas projekthez szükségesnek tartanak. A Nagypapa Házai az összes elvárást teljesíti: relációs adatbázis (SQL Server), háromrétegű architektúra (API + Blazor + Shared), autentikáció és jogosultságkezelés, CRUD műveletek, komplex üzleti logika (ütközésvizsgálat, kvóta-ellenőrzés).

A projektet az is motiválta, hogy valóban hasznos, releváns alkalmazást hozzunk létre, nem csupán egy iskolai feladat teljesítéséhez elegendő kódot. A Nagypapa Házai platform bármikor üzembe helyezhető éles körülmények között is, és valódi közösségi igényt elégíthet ki. Büszkék vagyunk rá, hogy nem egy "oktatási minta" szinten ragadtunk meg, hanem egy valóban kész, müköcképes rendszert hoztunk létre.

Személyes indíték szempontjából fontos megemlíteni, hogy mindhárom fejlesztő közel áll a vidékhez. Bota Csongor a Dunántúlról, Kania Áron Budapest közeléből, Batsukh Temuujin Mongóliából érkezett Magyarországra – ez a három különböző kultúrális háttér rendkívül gazdagította a projektről szóló vitákat és a platformon megvalósítható funkciókról szóló ötletelést. A kulturális különbségek megtanítottak arra, hogy a "közösségi megosztás" fogalma kultúrától függetlenül univerzális érték.

1.3. A program funkcióinak ismertetése

1.3.1. Ingatlankezelési modul

A Nagypapa Házai platform ingatlankezelési modulja az alkalmazás alapjait képezi. Három seed-ingatlant tartalmaz: a Faház a Bakonyban (Bakony, Veszprém megye; 10 főt befogadó, 25 000 Ft/éj), a Vízparti Nyaraló (Balatonlelle, Somogy megye; 6 főt befogadó, 35 000 Ft/éj) és a Villányi Vendégház (Villány, Baranya megye; 4 főt befogadó, 20 000 Ft/éj). Minden ingatlan rendelkezik névvel, helyszínnel,

részletes leírással, befogadóképességgel, státusszal (aktív/inaktív), képpel és éjszakánkénti árral.

Az adminisztrátorok az /admin/properties oldalon kezelhetik az ingatlanokat. Az ingatlankezelő felület lehetővé teszi új ingatlanok felvételét egy részletes űrlapon (minden fent említett mező), meglévők szerkesztését és törlését. A képfeltöltési funkció JPG és PNG formátumot támogat, legfeljebb 5 MB fájlméretet, minimum 200×200, maximum 2000×2000 pixel méretű képeket fogad el. Ingatlananként pontosan egy kép tölthető fel – ez a korlátozás egyszerűsíti az adatkezelést és megelőzi a tárhelyproblémákat. A rendszer szándékosan nem méretezi át automatikusan a feltöltött képeket: az adminisztrátornak kell megfelelő méretű képet feltöltenie.

Az ingatlan státusza az elérhetőségét szabályozza: aktív ("active") ingatlant az összes felhasználó látja és foglalhatja; inaktív ("inactive") ingatlan nem jelenik meg a főoldalon és nem foglalható. Ez lehetővé teszi az adminisztrátornak, hogy időszakosan eltávolítson egy ingatlant a kínálatból (pl. felújítás idejére) anélkül, hogy töröljük a foglalási előzményeket.

1.3.2. Foglalási rendszer

A foglalási rendszer a platform leggyakrabban használt funkciója. A felhasználók az /bookings/new oldalon adhatnak le foglalási kérelmet. A foglalási űrlapon megadandó adatok: teljes név, e-mail cím, telefonszám (szabad formátumú), az ingatlan kiválasztása legördülő listából, opcionálisan egy esemény kiválasztása amelyhez a foglalás kapcsolódik, a kezdő és záró dátum naptárválasztóból, és egy opcionális csereajánlat szöveg (legfeljebb 500 karakter).

A foglalás beküldésekor az API automatikusan elvégzi az ütközésvizsgálatot: ellenőrzi, hogy a kért időszak nem fed-e át más meglévő foglalással ugyanarra az ingatlannál. Ha ütközés van, 409 Conflict HTTP válasszal jelzi vissza. Ha szabad az időszak, a foglalás "pending" (függőben) státusszal kerül mentésre. Az adminisztrátor ezt követően jóváhagyhatja ("approved") vagy elutasíthatja ("rejected") a kérelmet az /admin/bookings felületen.

1.3.3. Naptár nézet

A naptár nézet (/bookings/calendar) vizuálisan mutatja be az összes foglalást havi bontásban. A foglalások névcsíkok formájában jelennek meg a megfelelő napokon, minden foglaláshoz különböző szín tartozik. A felhasználó hónapok között navigálhat a ← és → nyilakkal. A naptár nézet azok számára különösen hasznos, akik gyorsan szeretnék áttekinteni, mely időszakok szabad és foglalt, mielőtt beadnák a saját foglalási kérelmüket.

1.3.4. Eseményszervezési modul

Az eseményszervezési modul lehetővé teszi, hogy a platform ingatlanaihoz rendezvényeket, programokat rendjenek hozzá. Egy esemény kötelező mezői: cím, leírás, kezdő dátum, záró dátum, maximális résztvevőszám, ingatlan (melyik helyszínen zajlik) és szervező neve. Opcionálisan megadható extra információ is (menetrend, felszerelési lista, tudnivalók). Az esemény-regisztráció az /events/{id} oldalon lehetséges bejelentkezett felhasználóknak, feltéve, hogy az esemény még nem telt be (aktuális regisztrációk száma < MaxParticipants).

1.3.5. Csereajánlat rendszer

A csereajánlat rendszer a Nagypapa Házai platform legegyedibb innovációja. A foglalási kérelem leadásakor a felhasználó opcionálisan megadhat egy szöveges csereajánlatot (legfeljebb 500 karakter), amelyben leírja, mivel tudná viszonozni a szállláslehetőséget. A csereajánlat megjelenítésre kerül az admin foglaláskezelő felületen, ahol az adminisztrátor a jóváhagyás döntésénél ezt is figyelembe veszi. Lehetséges csereajánlatok: kertészeti munkák, kisebb karbantartási feladatok (festés, javítások), helyi élelmiszerek vagy termékek (házi lekvár, méz, sajt, zöldség), egyéb szakmai szolgáltatások (fényképezés, webfejlesztés, fordítás), önkéntes közösségi munka.

1.3.6. Autentikáció és felhasználókezelés

A felhasználókezelési modul biztonságos regisztrációt és bejelentkezést biztosít. A regisztrációs folyamat felhasználónevet (min. 3, max. 50 karakter, csak betűk és számok, szóköz nem engedélyezett), e-mail-t (egyedi, érvényes formátumú) és jelszót (min. 8 karakter, legalább 1 nagybetű és 1 szám) kér. A jelszavak BCrypt hash algoritmussal titkosítottan kerülnek az adatbázisba. A rendszer kétféle szerepkört

különböztet meg: "user" (normál felhasználó) és "admin" (adminisztrátor), amelyet az adatbázis Users tábla Role mezője tárol.

1.3.7. Admin kezelőfelület

Az adminisztrátori felület (/admin) négy fő funkcióterületet fed le: (1) Dashboard – összesített statisztikák (összes foglalás, jóváhagyott, függőben, ingatlanok száma, aktív események, becsült bevétel, legfoglaltabb ingatlan); (2) Foglaláskezelés – minden foglalás listázása, jóváhagyása és elutasítása; (3) Ingatlankezelés – ingatlanok CRUD (létrehozás, olvasás, frissítés, törlés) kezelése, kép feltöltése; (4) Eseménykezelés – események CRUD kezelése, résztvevők nyomon követése. Az admin felület csak "admin" szerepkörű felhasználók számára elérhető.

1.4. Célközönség meghatározása

A Nagypapa Házai platform célközönsége széles spektrumon helyezkedik el. Az alábbiakban részletesen bemutatjuk a különböző felhasználói szegmenseket, azok specifikus igényeit és azt, hogyan elégíti ki a platform ezeket az igényeket.

Az elsődleges célcsoport a magánszemélyek és kiscsoportok, akik természetközeli vidéki szállást keresnek kereskedelmi szálláshelyek helyett. Ezek tipikusan 2-10 főből álló baráti, párkapcsolati vagy kis fehér asztaltársasági csoportok, akik hosszabb pihenést terveznek – néhány napot vagy akár egy hetet – egy vidéki helyszínen. Számukra a Nagypapa Házai kínálata az Airbnb-nél személyesebb és közösségibb élményt nyújt, ahol az ingatlan tulajdonosa egy közeli kapcsolaton keresztül fogadja a vendégeket.

A civil szervezetek és nonprofit csoportok szintén fontos célcsoportot alkotnak. Ifjúsági szervezetek, cserkészcsapatok, önkéntes közösségek, egyházi csoportok rendszeresen szerveznek táborokat és összejöveteleket, amelyekhez vidéki helyszínt keresnek. Ezek a szervezetek általában korlátozott anyagi erőforrásokkal rendelkeznek, és a csereajánlat rendszer lehetővé teszi, hogy munkaerővel vagy önkéntes tevékenységgel "fizessenek" a szállásért.

Az oktatási intézmények – általános iskolák, középiskolák, főiskolák, szakképző intézmények – szintén potenciális felhasználók. Terepgyakorlatok, osztálykirándulások, diáktáborok, szakmai csoportos utak szervezéséhez vidéki

helyszínek kell. A platform eseménymodulja különösen hasznos a tanároknak: egy eseményt létrehozva az iskola regisztrálni tudja a diákokat, és a kapacitáskorlát automatikusan kezelt.

Az alkotók és kézművesek számára a vidéki helyszínek alkotótábori célokra is alkalmasak. Festők, szobrászok, irodalmárok, zenészek, fotografusok rendszeresen keresnek csendes, természetes környezetet hosszabb alkotói periódusokra. A platform lehetővé teszi, hogy megállapodhatnak a tulajdonossal a szállásért cserébe nyújtott alkotói munkáért – akár egy freskóért a falon vagy egy fotósorozatért a helyszínről.

A kutatók és természetvizsgálók szintén célcsoportot alkotnak: biológusok, botanikusok, ornitológusok, meteorológusok, akik terepmunkához szükséges szállást keresnek. A vidéki ingatlanok ideális kiindulópontok hosszabb terepmunkákhoz. A platform rugalmas, több hetes foglalási lehetőséget is kínál.

Célcsoport	Tipikus igény	Csereajánlat lehetőség
Magánszemélyek, párkapcsolatok	2-4 főre, pihenés, romantika	Főzés, kisebb karbantartás
Kis baráticsoportok	4-10 főre, összejövetel, buli	Közös takarítás, kertrendezés
Civil szervezetek	Tábor, összejövetel, 10-15 fő	Önkéntes munka, festés
Oktatási csoportok	Terepgyakorlat, kirándulás	Diákok munkasegítése
Alkotók, kézművesek	Alkotótábor, csendes alkotótér	Alkotás a helyszínről
Kutatók	Terepmunka bázis	Kutatási eredmény megosztása
Adminisztrátorok	Rendszer üzemeltetése	Nem alkalmazható

1.5. Hasonló rendszerek összehasonlítása

A szállásfoglalási és ingatlanmegosztó platformok piaca rendkívül telített: az Airbnb 2008-as alapítása óta drámaian megváltoztatta a turisztikai szálláshelyek piacát. A Booking.com 1996-ban alapított, ma a Priceline Group tulajdonában lévő

platform elsősorban a szállodai és professzionális szálláshelyek terén domináns. Magyarország specifikus megoldása a SZÉP-kártya rendszer, amely munkáltatói támogatásból finanszírozott szállásfoglalást tesz lehetővé. Ezek mellett léteznek kisebb, niche megoldások is: a Couchsurfing (ingyenes szállláscserét szervező platform), a Home Exchange (lakáscsere) és a HelpX (munkáért szállás cserét szervező platform).

Az Airbnb 2024-re közel 8 millió aktív ingatlanhirdetéssel, 5 millió vendéglátóval és 150 millió felhasználóval rendelkezik globálisan. Az Airbnb platformon az átlagos jutalék a vendég számára 14-16%, a vendéglátó számára 3% (split-fee modell) vagy 15% (host-only modell). A platform elsősorban turisztikai célú, kereskedelmi jellegű: a vendéglátók bérleti bevételük maximalizálásában érdekeltek. A csereajánlat-alapú, ingyenes szállás fogalma teljesen idegen az Airbnb üzleti modelljétől.

A Booking.com a világ vezető szállás- és utazásfoglaló platformja. Elsősorban szállodák, pensziók és apartmanok hirdetik magukat ott, és a platform 15-18%-os jutalékot számít fel a szálláshelyeknek. A Booking.com-on nincs lehetőség csereajánlat-alapú foglalásra, és az eseményszervezési funkció is teljesen hiányzik.

A HelpX (helpx.net) egy kisebb, de ideológiában hasonlóbb platform: a vendégek önkéntes munkájukat cserélik szállásra a fogadógazda portáján. A HelpX azonban nem foglalási platform – nincs foglalási naptár, ütközésvizsgálat, eseménymodul, vagy admin kezelőfelület. Inkább egy hirdetési tábla, ahol vendégek és gazdaságok egymásra találnak, a foglalást e-mailben intézik egymás között.

A Nagypapa Házai a fenti platformoktól abban különbözik, hogy egyrészt ingyenes és nem kereskedelmi célú, másrészt a csereajánlat-rendszer formalizált részét képezi a foglalási folyamatnak, harmadrészt az esemény-szervezési modul szorosan integrált a foglalási rendszerrel, negyedrészt a platform kifejezetten vidéki, örökségi értékű ingatlanokra fókuszál, nem a turisták által keresett standard szálláshelyekre.

Szempont	Nagypapa Házai	Airbnb	Booking.com	HelpX	SZÉP-kártya
Ingyenes használat	Igen	Nem (jutalék)	Nem (jutalék)	Igen	Igen (kártya kell)

Szempont	Nagypapa Házai	Airbnb	Booking.com	HelpX	SZÉP-kártya
Csereajánlat	Igen (formalizált)	Nem	Nem	Igen (informális)	Nem
Eseményszervezés	Igen (integrált)	Nem	Nem	Nem	Nem
Foglalási naptár	Igen	Igen	Igen	Nem	Igen
Ütközésvizsgálat	Igen (API szintű)	Igen	Igen	Nem	Igen
Admin felület	Igen	Igen	Igen	Nem	Igen
Regisztráció	Foglaláshoz kell	Kötelező	Opcionális	Kötelező	Munkáltató adja
Célcsoport	Közösség, civil	Turisták, utazók	Turisták, üzleti	Önkéntesek	Munkavállalók
Vidéki fókusz	Igen	Nem elsősorban	Nem	Részben	Részben
Nyílt forráskód	Igen (GitHub)	Nem	Nem	Nem	Nem
Magyar nyelv	Igen (teljes)	Igen	Igen	Nem	Igen
Platform ára (vendég)	Ingyenes	14-16%	0% (szállásnak)	Ingyenes	Ingyenes

A fenti összehasonlítás alapján megállapítható, hogy a Nagypapa Házai egy valóban egyedi pozíciót foglal el a szállásfoglalási és ingatlanmegosztó platformok piacán. A platform nem versenyez az Airbnb-vel vagy a Booking.com-mal kereskedelmi értelemben, hanem egy teljesen más igényt elégít ki: a közösségi, nem-kereskedelmi, örökségi értékű vidéki ingatlanok megosztásának igényét. A HelpX-hez legközelebb áll ideológiailag, de technológiailag jóval fejlettebb: valódi foglalási rendszerrel, eseménymodullal és adminisztrációs felülettel.

2. Fejlesztői dokumentáció

2.1. A program elkészítésének indoklása – miért volt szükség rá?

A Nagypapa Házai projekt fejlesztésének indoklása több egymást erősítő szükségszerűségből fakad. Az első és talán legfontosabb motiváció a meglévő kereskedelmi ingatlan-megosztó platformok korlátaival kapcsolatos. Az Airbnb, a Booking.com és a hasonló globális platformok kizárólag a pénzalapú bérleti ügyletekre fókuszálnak: a tulajdonos pénzt kap, a vendég fizet, a platform jutalékot von le. Ez a modell hatékony a turisztikai szektor számára, de teljesen alkalmatlan arra a közösségi igényre, amelyet a Nagypapa Házai céloz.

A vidéki magyarországi ingatlanok helyzetét vizsgálva egyértelmű, hogy sok értékes, lakott vagy szezonálisan használt ingatlan hosszú időszakokon át üresen áll. Az épületek állagmegóvása éves szinten jelentős kiadást jelent a tulajdonosoknak: fűtés, karbantartás, állagmegőrzési munkák. Ezek a tulajdonosok sokszor szívesen fogadnának gondoskodó vendégeket, akik cserébe valamilyen hasznos tevékenységet végeznek – de sem az Airbnb, sem a Booking.com nem kínál erre megoldást. A Nagypapa Házai csereajánlat rendszere pontosan ezt a hiányosságot tölti be.

A második indoklás a közösségi rendezvényszervezés igényéhez kapcsolódik. Magyarország civil szektora, nonprofit szervezetei és önkéntes közösségei rendszeresen szükségét érzik vidéki helyszíneknek – táborokhoz, workshopokhoz, közösségi összejövetelekhez. Ezek a szervezetek általában korlátozott anyagi erőforrásokkal rendelkeznek, és a kereskedelmi szálláshelyek árai meghaladják a rendelkezésre álló keretet. A platform eseménymodulja kifejezetten erre az igényre lett tervezve: lehetővé teszi, hogy az ingatlanokhoz eseményeket rendeljenek, amelyekre online regisztrálni lehet, és amelyek kapacitását a rendszer automatikusan kezeli.

A harmadik indoklás technológiai jellegű. A csapat tagjai egy olyan fejlesztési feladatot kerestek, amely kellően komplex ahhoz, hogy minden megtanult technológiai elemet alkalmazzanak: relációs adatbázis tervezés, backend API fejlesztés, frontend komponens alapú fejlesztés, autentikáció, és valós idejű UI frissítés. A Nagypapa

Házai projekt ezt a kellő komplexitást biztosítja, miközben egy valódi, kézzelfogható problémát old meg. A záródolgozat témájaként különösen értékes, mert nemcsak technikai, hanem társadalmi relevanciája is van.

A negyedik szempont az oktatási értéknövelés: az N-Tier architektúra megvalósítása, a REST API tervezés elvei, az EF Core Code-First migrációk és a Blazor Server SignalR kommunikáció mind olyan elemek, amelyek a modern szoftvertervezés alapkövetelményeit tükrözik. A projekt elvégzése során a csapat tagjai mélyen megismerkedhettek ezekkel az iparági standardokkal, amelyek a jövőbeli szakmai pályájukon elengedhetetlenek lesznek.

Végül, de nem utolsósorban: a projekt személyes motiváción alapul. Bota Csongor, Kania Áron Nontawat és Batsukh Temuujin mindegyike kötődik vidéki Magyarországhoz, és mindegyiküknek van valamilyen személyes kapcsolata a kihasználatlan vidéki ingatlanok problémájával. Ez a személyes motiváció biztosítja, hogy a fejlesztés nem csupán egy iskolai feladat teljesítése, hanem egy valódi igény kielégítése felé tett lépés.

2.2. Fejlesztőkörnyezet

2.2.1. Hardverkörnyezet

A fejlesztési munka elvégzéséhez megfelelő hardverkörnyezet szükséges. Az alábbi fejezetben részletesen bemutatjuk a fejlesztéshez és teszteléshez használt számítógépes eszközöket, azok specifikációit, és indokoljuk, miért volt szükség ezekre a teljesítményszintekre.

A fejlesztői munkagép kiválasztásánál a legfontosabb szempont a Visual Studio 2022 IDE zökkenőmentes futtatása volt. A Visual Studio 2022 az egyik legtöbb erőforrást igénylő fejlesztőkörnyezet: a háttérben futó Roslyn fordító, az IntelliSense motor, az SQL Server Object Explorer és a Hot Reload funkció együttesen akár 4-6 GB RAM-ot is felhasználnak aktív fejlesztés közben. Ehhez adódik az SQL Server Express szolgáltatás, amely önmagában 512 MB–1 GB memóriát foglal le, valamint a fejlesztés közbeni böngésző-ablak (Chrome DevTools megnyitva), amely szintén erőforrás-igényes.

Az Intel Core i7-12700H processzor különösen alkalmas volt a fejlesztési feladatokhoz, mivel hibrid architektúrájának (14 mag: 6 teljesítménymag P-core és 8 hatékonysági mag E-core) köszönhetően a fordítási és indexelési feladatokat a teljesítménymagok kezelik, míg a háttérfolyamatok (vírusirtó, operációs rendszer) a hatékonysági magokon futnak. Ez biztosítja, hogy a fordítási idő ne zavarjon bele az aktív kódolásba.

Az NVMe SSD különösen fontos a fejlesztési teljesítmény szempontjából. Az NVMe SSD adatátviteli sebessége (olvasás: ~3500 MB/s, írás: ~3000 MB/s) nagyságrendekkel haladja meg a hagyományos HDD vagy SATA SSD sebességét. Ez különösen érezhető a projekt megnyitásakor (Visual Studio Solution loading), az EF Core migrációk futtatásakor, és az SQL Server adatfájlok elérésekor.

Paraméter	Fejlesztői gép	Tesztelő gép
Processzor modell	Intel Core i7-12700H	Intel Core i5-10400
Magok száma	14 mag (6P + 8E)	6 mag (12 szál)
Max. órajel	4.7 GHz Turbo Boost	4.3 GHz Turbo Boost
RAM mérete	16 GB	8 GB
RAM típusa	DDR5 4800 MHz (dual-channel)	DDR4 3200 MHz (dual-channel)
Tárhely	512 GB NVMe SSD (PCIe 4.0)	256 GB SATA SSD
Operációs rendszer	Windows 11 Pro 22H2 (64-bit)	Windows 10 Home 22H2 (64-bit)
Kijelző felbontás	1920×1080 Full HD	1366×768 HD
GPU	NVIDIA GeForce RTX 3050 Ti (4 GB)	Intel UHD Graphics 630 (integrált)
Hálózati adapter	Wi-Fi 6 (802.11ax) + 1Gbps Ethernet	Wi-Fi 5 (802.11ac) + 100Mbps Ethernet
Operációs rendszer	Windows 11 Pro 22H2 (x64)	Windows 10 Home 22H2 (x64)

A tesztelő gép szándékosan alacsonyabb specifikációjú volt, hogy reálisan modellezze, milyen körülmények között futtatják majd az alkalmazást az átlagos felhasználók. Az i5-10400 processzor ugyan kevesebb magot és kisebb L3 gyorsítótárat tartalmaz, mint az i7-12700H, mégis megfelelő teljesítményt nyújtott az alkalmazás teszteléséhez, jelezve, hogy a platform nem igényel nagy teljesítményű hardvert a futtatáshoz.

A mobileszközös tesztelés szintén kritikus fontosságú volt a Bootstrap 5 reszponzív kialakítás ellenőrzéséhez. A Samsung Galaxy A54 Android 14 operációs rendszerrel, 6,4 colos 2340×1080 pixeles Super AMOLED kijelzővel és 128 GB belső tárhellyel rendelkezik. Az Apple iPhone 13 iOS 17 operációs rendszerrel, 6,1 colos 2532×1170 pixeles Super Retina XDR kijelzővel és 128 GB tárhellyel rendelkezik. Mindkét eszközön ellenőriztük a Bootstrap 5 reszponzív töréspontjainak (breakpoint) helyes müközését.

Paraméter	Samsung Galaxy A54	Apple iPhone 13
Operációs rendszer	Android 14 (One UI 6.0)	iOS 17.2.1
Kijelző méret	6.4 colos	6.1 colos
Kijelző felbontás	2340×1080 pixel (FHD+)	2532×1170 pixel (Super Retina XDR)
Böngésző (teszteléshez)	Chrome Mobile 122 (Android)	Safari 17 (iOS beépített)
RAM	8 GB	4 GB (unified memory)
Processzor	Exynos 1380 (5 nm)	Apple A15 Bionic (5 nm)
Bootstrap breakpoint	sm (576px+) – telefon nézet	sm (576px+) – telefon nézet

2.2.2. Szoftverkörnyezet

A fejlesztéshez szükséges szoftverek megválasztása alapos megfontolás eredménye. A szoftverek kiválasztásánál figyelembe vettük az ingyenességet, a .NET ökoszisztémával való kompatibilitást, a közösségi támogatottságot és a dokumentáció minőségét. Az alábbiakban minden egyes szoftvert részletesen ismertetünk.

A Visual Studio 2022 Community Edition a Microsoft legteljesebb ingyenes fejlesztőkörnyezete. Az Enterprise és Professional változatokkal ellentétben a Community ingyen letölthető és használható fejlesztési és oktatási célra. Főbb előnyök a fejlesztési folyamatban: a Hot Reload funkció lehetővé teszi a Blazor komponensek módosításának azonnali megjelenítését böngésző-újraindítás nélkül; a beépített Git integráció (Team Explorer) az összes verziókövető műveletet grafikus felületen teszi elérhetővé; a SQL Server Object Explorer közvetlenül a Visual Studio-ból teszi lehetővé az adatbázis böngészését és lekérdezések futtatását.

A .NET 8 SDK (Software Development Kit) a program fordításához, futtatásához és a CLI eszközök (dotnet ef, dotnet run) használatához szükséges. A .NET 8 hosszú távú támogatású (LTS) verzió, amelyet a Microsoft 2026 novemberéig biztonsági frissítésekkel lát el. Ez biztosítja, hogy a projekt a közeljövőben is biztonságosan futtatható maradjon. A .NET 8 teljesítménybeli fejlesztései (AOT kompilálás, SIMD optimalizálás) jelentős sebességnövekedést hoznak a korábbi verziókhoz képest.

Az SQL Server 2022 Express az SQL Server ingyenes változata, amely legfeljebb 10 GB adatbázis-méretet és 1 GB RAM-ot (adatbázis-motor részére) támogat. Ezek a korlátok a Nagypapa Házai projekt léptékéhez bőven elegendők. Az SQLEXPRESS nevű instance automatikusan települ, és a localhost\SQLEXPRESS connection string-gel csatlakozható. Az SSMS (SQL Server Management Studio) grafikus adminisztrációs felülete lehetővé teszi az adatbázis-struktúra vizuális böngészését, SQL lekérdezések futtatását és a migrációk ellenőrzését.

Szoftver neve	Verzió	Felhasználási cél	Indoklás
Visual Studio 2022 Community	17.8.6	Fő fejlesztőkörnyezet (IDE)	Ingyenes, teljes .NET + Blazor IDE támogatás, Hot Reload
.NET 8 SDK	8.0.201 LTS	Fordítás, futtatás, dotnet CLI	LTS, 2026-ig támogatott, EF Core 8 kompatibilis
SQL Server 2022 Express	16.0.1000	Adatbázis-kezelő	Ingyenes, 10 GB limit, SQLEXPRESS instance

Szoftver neve	Verzió	Felhasználási cél	Indoklás
SSMS	19.3.4.0	DB adminisztráció, SQL editor	Grafikus DB böngésző, lekérdezés futtatás
Git	2.43.0	Verziókezelő	Ipari standard, GitHub integráció
GitHub	Cloud SaaS	Kód tárolás, csapat kollabor.	Ingyenes, pull request workflow
Bootstrap	5.3.3	CSS framework, reszponzív UI	Komponens-gazdag, CDN betöltés, jól dokumentált
Entity Framework Core	8.0.2	ORM, Code-First migráció	.NET natív ORM, LINQ, DbContext
Swashbuckle/Swagger	6.5.0	API dok. és Swagger UI	Automatikus OpenAPI generálás
Postman	10.22	REST API tesztelés	Grafikus HTTP kliens, collection export
BCrypt.Net-Next	4.0.3	Jelszóhash	Iparági standard, sós BCrypt hash
Google Chrome	122.x	Böngésző, DevTools	DevTools mélység, piacvezető
Mozilla Firefox	123.x	Cross-browser teszt	Gecko render engine, eltérő CSS kezelés
Microsoft Edge	122.x	Windows alapért. böngésző	Chromium alap, BeTA funkciók
Figma (tervezés)	Web	UI/UX vázlat készítés	Ingyenes, böngészőben fut

A Git verziókezelő rendszer és a GitHub platform a csapatmunka alapját képezi. A fejlesztés során a GitHub Flow branching modellt alkalmaztuk: minden új funkció fejlesztése külön feature branch-on zajlott (pl. feature/booking-conflict-check), a main ágba pull request-en keresztül lehetett beolvasztani. Ez a módszer biztosítja, hogy a main ág mindig stabilan buildelhető állapotban maradjon, és a kódminőség ellenőrzése minden beolvasztás előtt megtörténjen.

A Postman API tesztelési eszköz a fejlesztés korai szakaszában különösen fontos volt, amikor a Blazor frontend még nem volt teljesen kész, de az API végpontokat már tesztelni kellett. A Postman Collection formátuma lehetővé teszi az összes tesztelt kérés exportálását és újrafuttatását, ami megkönnyíti a regressziós tesztelést a fejlesztési ciklusok során. Az autentikációt igénylő végpontokat a bejelentkezési válaszból visszaadott adatokkal lehetett tesztelni.

2.3. Programnyelv és fejlesztőkörnyezet választásának indoklása

2.3.1. Miért C# és .NET 8?

A C# programozási nyelv a Microsoft .NET platform elsődleges fejlesztői nyelve, amelyet 2000-ben vezettek be, és azóta az egyik legelterjedtebb általános célú programozási nyelvvé vált. A C# választásának legfőbb oka a csapattagok meglévő tapasztalata volt: mindhárom fejlesztő rendelkezett C# alapismeretekkel korábbi tanulmányaikból, ami lehetővé tette a fejlesztési folyamat gyors megindulását és a mélyebb funkciók felé való előrelépést anélkül, hogy egy teljesen ismeretlen szintaxissal kellett volna megbirkózni.

A C# erős típusossága (strong typing) különösen értékes a háromrétegű alkalmazás fejlesztésekor. A fordítási idejű típusellenőrzés megakadályozza, hogy futási idejű típushibák kerüljenek az éles kódba: ha egy metódus int-et vár és string-et kap, a fordító azonnal jelzi a hibát. Ez a tulajdonság JavaScript-tel vagy Python-nal összehasonlítva jelentős előnyt jelent, ahol a típushibák csak futásidőben derülnek ki. A csapat tapasztalata alapján ez legalább 30%-kal csökkentette a debugolással töltött időt.

A .NET 8 LTS (Long-Term Support) verzió választása stratégiai döntés volt. Az LTS verziók három évig kapnak biztonsági és hibajavító frissítéseket a Microsofttól (2024. november – 2026. november), ami azt jelenti, hogy a projekt hosszú ideig stabilan futtatható marad kritikus frissítések nélkül is. A Standard Term Support (STS) verziók (páros számok között: 9, 11 stb.) csak 18 hónapig kapnak támogatást, ami rövid, ha a rendszer hosszabb ideig müköd. A .NET 8 teljesítménybeli előrelépései

szintén figyelemre méltók: a .NET 7-hez képest az ASP.NET Core benchmark tesztek átlagosan 18%-os teljesítménynövekedést mutatnak.

A .NET 8 aszinkron programozási modellje (async/await) az API végpontokban különösen hasznos: az adatbázis-hozzáférések (await _context.SaveChangesAsync()), a HTTP kérések és más I/O műveletek nem blokkolják a szálakat, lehetővé téve, hogy egyetlen szerverflyamat sok párhuzamos kérést kezeljen egyszerre. Ez különösen a foglalási ütközésvizsgálatnál és az esemény-regisztrációnál fontos, ahol párhuzamos kérések is érkezhetnek.

2.3.2. Miért Blazor Server és nem más technológia?

A Blazor Server mint frontend technológia kiválasztása a projektben részt vevő fejlesztők legvitatottabb döntése volt. Az alternatívák között szerepelt a React, az Angular, a Vue.js és a Razor Pages. Végül a Blazor Server mellett döntöttünk, amelynek döntő érve az volt, hogy egyetlen C# kódbázisban valósítható meg a teljes frontend és backend logika, JavaScript írása nélkül.

A React összehasonlítva: A React egy JavaScript/TypeScript alapú, komponens-alapú UI könyvtár, amely rendkívül népszerű és széles körben alkalmazott. Előnyei: nagy közösség, gazdag ökoszisztéma (npm csomagok), elvalasztott frontend-backend fejlesztési modell. Hátrányai a mi projektünkben: külön JavaScript/TypeScript tudás szükséges, a backend és frontend között JSON API határfelület kell, amit két különböző nyelven kell fejleszteni. Blazor Serverrel ugyanazt a C# kódbázist használjuk mindkét oldalon.

Az Angular összehasonlítva: Az Angular egy teljes értékű frontend MVC keretrendszer TypeScript-ben, amelyet a Google tart fenn. Az Angular életciklus hookjai, a RxJS Observable alapú aszinkron kezelés és a Dependency Injection rendszere ugyan hasonlít a Blazor-éhoz, de a tanulási görbe meredekebb, és TypeScript tudás szükséges. A Blazor Server az Angular-hoz hasonló életciklus-rendszert (OnInitializedAsync, OnParametersSetAsync) kínál C#-ban.

A Razor Pages összehasonlítva: A Razor Pages az ASP.NET Core hagyományos, szerver oldali renderelési megközelítése, amelyen keresztül minden oldalváltás egy teljes HTTP kérés-válasz ciklust jelent. A Blazor Serverrel ellentétben a Razor Pages-ben nincs valós idejű kommunikáció: az oldalak statikusan renderelnek. A naptár nézet, az admin foglaláskezelés (Approve/Reject gombok) és

az állapotfrissítések valós idejű megjelenítése Razor Pages-szel csak JavaScript AJAX kérésekkel lett volna megvalósítható, ami a fejlesztési komplexitást növelte volna.

A Blazor Server fő hátránya: állandó WebSocket (SignalR) kapcsolat szükséges a szerver és a böngésző között. Ez azt jelenti, hogy ha az internet-kapcsolat megszakad, az alkalmazás nem müköd. Nagy felhasználószámnál a szerver-terhelés is megnőhet, mivel minden felhasználónak saját SignalR kapcsolata van a szerveren fenntartva. Ezek a hátrányok azonban a projekt jelenlegi léptékénél (néhány párhuzamos felhasználó) nem jelentenek problémát.

2.3.3. Miért SQL Server Express és nem más adatbázis?

Az adatbázis-motor kiválasztásánál négy alternatívát mérlegeltük: SQLite, MySQL, PostgreSQL és SQL Server Express. Végül az SQL Server Express mellett döntöttünk, amelynek legfőbb indoka az Entity Framework Core .NET-specifikus SQL Server providere és az SSMS grafikus felület volt.

Az SQLite egy beágyazott, szerver nélküli adatbázis-motor, amely egyetlen fájlban tárolja az egész adatbázist. Előnyei: nem szükséges telepíteni, nulla konfigurálás, ideális prototípusfejlesztéshez és teszteléshez. Hátrányai a mi projektünkben: a párhuzamos írási műveletek kezelése korlátozott (csak egy írási folyamat futhat egyszerre), ami a foglalási ütközésvizsgálatnál problémát okozhat, ha egyszerre több felhasználó próbál foglalni ugyanarra az ingatlanra. Az SQL Server tranzakció-kezelése megbízhatóbb párhuzamos hozzáférés esetén.

A MySQL egy nyílt forráskódú, széles körben elterjedt relációs adatbázis-kezelő. Előnyei: ingyenes, Linux-barát, nagy közösség. Hátrányai: az EF Core MySQL provider (Pomelo.EntityFrameworkCore.MySql) nem a Microsoft hivatalos terméke, dokumentációja kevésbé széleskörű, és a Windows-on való telepítése bonyolultabb, mint az SQL Server-é. A PostgreSQL hasonló okokból nem vált elsődleges választássá.

Az SQL Server Express .NET/C# projektekhez a legészszerűbb választás: a Microsoft Docs dokumentáció kizárólag SQL Server és SQLite példákkal illusztrálja az EF Core tutorialokat, az SSMS grafikus felülete közvetlenül a Visual Studio-ból megnyitható (View → SQL Server Object Explorer), és a connection string szintaxis

megegyezik a production SQL Server-rel, így a jövőbeli Azure SQL-re való migráció minimális módosítással elvégezhető.

2.3.4. Miért Entity Framework Core ORM?

Az Entity Framework Core (EF Core) a .NET natív ORM (Object-Relational Mapper) megoldása, amelyet a Microsoft fejleszt és tart fenn. Az ORM absztrakciós réteget biztosít az alkalmazás és az adatbázis között: a fejlesztőknek nem kell SQL lekérdezéseket kézzel írni, hanem LINQ (Language Integrated Query) kifejezésekkel, C# kódban dolgozhatnak. Az EF Core fordítja le ezeket a LINQ kifejezéseket optimalizált SQL lekérdezésekké.

A Code-First megközelítés a mi munkamódszerünknek különösen megfelelt: először meghatározzuk a C# adatmodell-osztályokat (entitásokat), és az EF Core automatikusan generálja belőlük az adatbázis-sémát a migrációs rendszeren keresztül. Ha az adatmodell megváltozik (pl. új oszlopot adunk hozzá), egy új migráció létrehozásával az adatbázis szinkronizálható a kóddal, és a változtatások SQL formában is nyomon követhetők. Ez a megközelítés a Database-First megközelítéssel szemben (ahol az adatbázis-séma az elsődleges, és belőle generálódnak az osztályok) fejlesztőbarátabb és Agilis fejlesztési modellhez jobban illeszkedik.

Az EF Core LINQ alapú lekérdezési szintaxisa olvasható és típusbiztonságos. Például az ütközésvizsgálat LINQ-ban: _context.Bookings.Where(b => b.PropertyId == dto.PropertyId && b.StartDate < dto.EndDate && b.EndDate > dto.StartDate).AnyAsync() – ez emberek által olvasható C# kód, amely az EF Core fordítja SQL-re: SELECT CASE WHEN EXISTS (SELECT 1 FROM Bookings WHERE PropertyId = @p0 AND StartDate < @p1 AND EndDate > @p2 AND Status != 'rejected') THEN CAST(1 AS bit) ELSE CAST(0 AS bit) END.

2.3.5. Miért Bootstrap 5?

A Bootstrap 5 a legszélesebb körben elterjedt CSS keretrendszer, amelyet Twitter fejlesztett ki, és 2021 óta önállóan fejlesztenek tovább. A Bootstrap 5 előnyei a mi projektünkben: az előre meghatározott grid rendszer (12 oszlopos, töréspontok alapján reszponzív) gyorsan és kóddal leírható módon teszi lehetővé a reszponzív elrendezés kialakítását; a Bootstrap komponensek (Card, Navbar, Badge, Modal,

Alert, Form) azonnal használható HTML elemek, amelyek konzisztens megjelenést biztosítanak böngészők és eszközök között.

A Tailwind CSS összehasonlítva: A Tailwind utility-first megközelítése rugalmas és expresszív, de meredekebb tanulási görbével jár. Tailwind-del az összes stílusszabályt HTML osztályokkal kell megadni (pl. class="flex items-center justify-between p-4 bg-blue-500 text-white rounded-lg"), ami hosszú, zsúfolt HTML attribútumokat eredményez. Bootstrap-pel a komponens osztályok magasabb szintű absztrakciót nyújtanak (pl. class="card shadow-sm"), amelyek rövidebb, áttekinthetőbb HTML-t adnak.

A Material UI, Ant Design és hasonló komponens-könyvtárak React-specifikusak, vagy legalábbis JavaScript-keretrendszert feltételeznek. Ezek a Blazor Server alkalmazásban nem alkalmazhatók natívan, és a Blazor-JavaScript interop (JSInterop) segítségével való integráció komoly plusz komplexitást vinne a kódba. A Bootstrap CDN-ről betölthető, Razor komponensekbe könnyedén beilleszthető, és nem igényel npm build folyamatot.

2.4. Projektstruktúra és mappaszerkezet

A projekt mappastruktúrája tükrözi az N-Tier architektúra rétegeit. A Solution file (NagypapaHazaiBlazor.sln) három projektet fog össze. Az alábbiakban a teljes mappaszerkezetet mutatjuk be, minden egyes fájlhoz és mappához részletes magyarázattal.

```
NagypapaHazaiBlazor.sln          <- Solution fájl (Visual Studio nyitja meg)
│
├── NagypapaHazai.API/            <- ASP.NET Core Web API backend projekt
│   ├── Controllers/
│   │   ├── AuthController.cs     <- POST /api/auth/register, POST /api/auth/login
│   │   ├── BookingsController.cs <- GET/POST/PUT/DELETE /api/bookings
│   │   ├── PropertiesController.cs <- CRUD /api/properties
│   │   ├── EventsController.cs   <- CRUD /api/events
│   │   ├── EventRegistrations    <- GET/POST /api/eventregistrations
│   │   │   Controller.cs
│   │   └── DashboardController.cs <- GET /api/dashboard
│   ├── Data/
│   │   └── NagypapaHazaiContext.cs <- EF Core DbContext, OnModelCreating
│   ├── DTOs/
│   │   ├── LoginDto.cs           <- { Email, Password }
│   │   ├── RegisterDto.cs        <- { UserName, Email, Password }
│   │   ├── BookingCreateDto.cs   <- Foglalás létrehozási DTO
```

```
│   │   ├── BookingStatusDto.cs   <- Státuszváltás DTO
│   │   └── DashboardViewModel.cs <- Statisztika összesítő DTO
│   ├── Migrations/
│   │   ├── 20240101_InitialCreate.cs   <- Kezdeti séma migráció
│   │   └── NagypapaHazaiContextModelSnapshot.cs
│   ├── Program.cs                <- DI konfiguráció, middleware pipeline
│   ├── appsettings.json          <- Connection string, CORS, Swagger
│   └── appsettings.Development.json <- Fejlesztői overrides
│
├── NagypapaHazai.Shared/         <- Közös Class Library (.NET Class Library)
│   ├── Models/
│   │   ├── User.cs               <- Felhasználó entitás modell
│   │   ├── Property.cs           <- Ingatlan entitás modell
│   │   ├── Booking.cs            <- Foglalás entitás modell
│   │   ├── Event.cs              <- Esemény entitás modell
│   │   └── EventRegistration.cs  <- Esemény-regisztráció entitás
│   └── NagypapaHazai.Shared.csproj
│
└── NagypapaHazaiBlazor/          <- Blazor Server frontend projekt
    ├── Components/
    │   ├── Pages/
    │   │   ├── Home.razor        <- Főoldal (/ route), ingatlan kártyák
    │   │   ├── BookingList.razor <- /bookings - foglalások listája
    │   │   ├── BookingNew.razor  <- /bookings/new - új foglalás űrlap
    │   │   ├── BookingCalendar.razor <- /bookings/calendar - naptár
    │   │   ├── Events.razor      <- /events - eseménylista
    │   │   ├── EventDetail.razor <- /events/{id} - esemény részlete + reg.
    │   │   ├── Login.razor       <- /login - bejelentkezés
    │   │   ├── Register.razor    <- /register - regisztráció
    │   │   ├── Profile.razor     <- /profile - saját adatok + foglalások
    │   │   └── Admin/
    │   │       ├── Dashboard.razor  <- /admin - statistikák
    │   │       ├── BookingsMgmt.razor <- /admin/bookings
    │   │       ├── PropertiesMgmt.razor <- /admin/properties
    │   │       └── EventsMgmt.razor <- /admin/events
    │   ├── Layout/
    │   │   ├── MainLayout.razor  <- Fő layout wrapper (NavMenu + @Body)
    │   │   └── NavMenu.razor     <- Navigációs sáv komponens
    │   ├── App.razor             <- Blazor router gyökérkomponens
    │   └── Routes.razor          <- Útvonal definíciók
    ├── Services/
    │   ├── AuthStateService.cs   <- Singleton: bejelentkezett user állapot
    │   └── ApiService.cs         <- HttpClient wrapper: API hívások
    ├── wwwroot/
    │   ├── css/
    │   │   └── app.css           <- Egyedi Bootstrap felülírások, stílusok
    │   ├── js/
    │   │   └── app.js            <- Kis JS segédfüggvények (pl. scroll)
    │   └── images/
    │       └── (feltöltött ingatlan képek)
    ├── Program.cs               <- DI: HttpClient, AuthStateService regisztráció
    ├── appsettings.json          <- API base URL konfigurálása
    └── NagypapaHazaiBlazor.csproj
```

A NagypapaHazai.Shared Class Library tartalmazza az összes adatmodell-osztályt (entitásokat). Ezek az osztályok mind az API projekt, mind a Blazor projekt által referenciáltak, ami biztosítja az adatmodellek egységes definícióját a DRY (Don't Repeat Yourself) elv szerint. A Shared projekt nem hivatkozik más projektre, és nem tartalmaz üzleti logikát – kizárólag POCO (Plain Old C# Object) osztályokat és esetlegesen annotációkat.

Az NagypapaHazai.API projekt az üzleti logikát és az adatelérési réteget valósítja meg. A Controllers mappa tartalmazza az összes REST API végpontot; a Data mappa a DbContext osztályt; a DTOs mappa az adatátviteli objektumokat, amelyek a HTTP kérések és válaszok struktúráját definiálják. A DTOs tér el az entitásmodellektől: míg az entitások az adatbázis-struktúrát tükrözik, a DTO-k a kliensek felé közvetített adatokat optimalizálják (pl. PasswordHash mező nem kerül vissza a kliensnek).

A NagypapaHazaiBlazor projekt a Blazor Server frontend. A Components/Pages mappa tartalmazza az összes navigálható oldalt (.razor fájlok); a Services mappa a frontend üzleti logikát (API kommunikáció, állapotkezelés). A wwwroot mappa a statikus fájlokat (CSS, JavaScript, képek) tartalmazza, amelyek közvetlenül a böngészőbe kerülnek kiszolgálásra. A Program.cs-ben regisztrálják az összes szolgáltatást (DI), és konfigurálják a Blazor Server pipeline-t.

2.5. Adatszerkezet részletes bemutatása

2.5.1. Az adatbázis-tervezés elvei

Az adatbázis normalizálása egy szisztematikus folyamat, amelynek célja az adatredundancia minimalizálása és az adatintegritás maximalizálása. A normalizálás Codd relációs adatbázis-elméletén alapul, és normálformák (NF) sorozatán keresztül hajtódik végre. A Nagypapa Házai adatbázisát a harmadik normálformáig (3NF) normalizáltuk.

Az első normálalak (1NF) követelménye: minden tábla sorának és oszlopának atomi (oszthatatlan) értéket kell tartalmaznia, és minden sor egyedileg azonosítható (elsődleges kulcs). A Nagypapa Házai adatbázisában ez teljesül: például a Bookings táblában a StartDate és EndDate külön oszlopokban vannak tárolva (nem egy "2024-

07-01;2024-07-07" típusú összevont értékként), és minden tábla rendelkezik Id (int, IDENTITY) elsődleges kulccsal.

A második normálalak (2NF) követelménye: az 1NF teljesítése mellett minden nem-kulcs attribútumnak teljes funkcionális függőségben kell lennie az elsődleges kulcstól. Összetett elsődleges kulcs esetén (ami az EventRegistrations táblánál alkalmazható) minden nem-kulcs attribútumnak az egész összetett kulcstól kell függenie, nem csak annak részétől. Mivel a Nagypapa Házai tábláinak elsőleges kulcsa mindig egyetlen oszlop (Id), a 2NF automatikusan teljesül.

A harmadik normálalak (3NF) követelménye: a 2NF teljesítése mellett nem lehet tranzitív függőség a nem-kulcs attribútumok között. Tranzitív függőség akkor áll fenn, ha A → B és B → C, tehát A tranzitívan meghatározza C-t. A Nagypapa Házai adatbázisában ezt gondosan elkerültük: például az ingatlan neve (Name) és ára (EjszakankentiAr) a Properties táblában van tárolva, és a Bookings tábla csak a PropertyId-t tartalmazza – az ár nem szerepel redundánsan a foglalásban is.

Az EF Core Code-First megközelítésben az adatbázis sémáját C# osztályok definiálják, és az EF Core automatikusan leképezi azokat relációs táblastruktúrára. Az entitásosztályok POCO (Plain Old C# Object) osztályok, amelyek az EF Core konvenciók szerint automatikusan felismerhetők: az Id nevű tulajdonság automatikusan elsődleges kulccsá válik, a [Required] annotáció NOT NULL megszorítást generál, a [MaxLength(200)] annotáció nvarchar(200) típust eredményez.

2.5.2. Adatbázis táblák részletes leírása

Users tábla – teljes leírás

A Users tábla az alkalmazás felhasználói fiókjait tárolja. Ez a tábla az autentikációs alap: bejelentkezéskor az Email mező alapján keresi meg a rendszer a felhasználót, majd a PasswordHash mező BCrypt verifikálásával hitelesíti a belépési kísérletet. A Role mező kétféle értéket vehet fel: "user" és "admin" – ez a mező határozza meg, hogy a felhasználó hozzáférhet-e az adminisztrációs felülethez.

Az Email mezőn UNIQUE index van definiálva, ami az adatbázis szintjén akadályozza meg a duplikált e-mail-regisztrációt. Ez a megszorítás az API-szintű ellenőrzés (AnyAsync) kiegészítéseként müköd: ha valami miatt az API szintű ellenőrzés nem futna le, az adatbázis-szintű UNIQUE megszorítás akkor is

megakadályozná a duplikátumot, SqlException-t dobva. A CreatedAt mező automatikusan az aktuális UTC időt kapja az EF Core alapértelmezett értékeként.

Mező neve	SQL adattípus	Hossz/Pont.	Nullable	Default	EF Core annotáció
Id	int	-	NO	IDENTITY	[Key]
UserName	nvarchar	100	NO	-	[Required][MaxLength(100)]
Email	nvarchar	200	NO	-	[Required][MaxLength(200)]
PasswordHash	nvarchar	500	NO	-	[Required][MaxLength(500)]
Role	nvarchar	20	NO	'user'	[MaxLength(20)]
CreatedAt	datetime2	7	NO	getdate()	-

Properties tábla – teljes leírás

A Properties tábla az összes megosztható ingatlant tartalmazza. A jelenlegi rendszerbe három seed-ingatlan kerül az inicializáláskor: Faház a Bakonyban, Vízparti Nyaraló és Villányi Vendégház. A Status mező "active" és "inactive" értékeket vehet fel: inaktív ingatlan nem jelenik meg a főoldalon, és nem foglalható. Az ImageUrl mező a feltöltött kép relatív elérési útját tárolja (pl. /images/fahaz.jpg), amelyet a wwwroot/images mappában lévő kép fájlneve alkot.

Az éjszakánkénti ár mezőnél a decimal(10,2) adattípus biztosítja a pénzügyi számítások precizitását. A float vagy double adattípusok lebegőpontos reprezentációja

miatt kerekítési hibákat okoznak pénzügyi számításoknál (pl. 0.1 + 0.2 != 0.3 floating-point hibája). A decimal típus decimális pontossággal tárolja az értékeket, így 25000.50 pontosan 25000.50 marad, nem 25000.499999... A Capacity mező az int (4 byte, ~2 milliárd max) típussal elegendően nagy tartományban tárolja a személyszámot.

Mező neve	SQL adattípus	Hossz/Pont.	Nullable	Default	Leírás
Id	int	-	NO	IDENTITY(1,1)	PK, auto-increment
Name	nvarchar	200	NO	-	Ingatlan neve (pl. "Faház a Bakonyban")
Location	nvarchar	200	NO	-	Helyszín szöveges leírása
Description	nvarchar	2000	YES	NULL	Részletes leírás, opcionális
Capacity	int	-	NO	-	Max. befogadóképesség főben
Status	nvarchar	50	NO	'active'	"active" vagy "inactive"
ImageUrl	nvarchar	500	YES	NULL	Relatív kép URL, NULL ha nincs kép
CreatedAt	datetime2	7	NO	getdate()	Felvétel UTC időbélyege

Bookings tábla – teljes leírás

A Bookings tábla a platform legfontosabb tranzakciós táblája. Minden beküldött foglalási kérelem egy sorként kerül be ebbe a táblába, és az életciklusa a "pending" → "approved" vagy "rejected" státuszváltáson keresztül zajlik. Az ExchangeOffer mező a csereajánlat szövegét tartalmazza: ha a felhasználó nem ad meg csereajánlatot, a

mező NULL értékű. Az EventId opcionális: ha a foglalás egy eseményhez kapcsolódik, az EventId tartalmazza az esemény azonosítóját; egyébként NULL.

A Phone mező nvarchar(30) típusú, ami elegendő a legkülönbözőbb telefonszám-formátumok tárolásához: "+36-30-123-4567", "06301234567", "+1 (555) 123-4567", "+81-90-1234-5678" stb. Az Email mező a foglalásban önálló tárolással rendelkezik (nem FK a Users táblára), ami lehetővé teszi, hogy a foglalás adatai akkor is megmaradjanak, ha a felhasználói fiókot törlik. Ez adatvédelmi szempontból is rugalmasabb megoldás.

Mező neve	SQL adattípus	Hossz/Pont.	Nullable	FK → tábla	Leírás
Id	int	-	NO	-	PK, IDENTITY(1,1)
UserName	nvarchar	100	NO	-	Foglaló megjelenítendő ne
Email	nvarchar	200	NO	-	Foglaló e-mail-je (értesítés
Phone	nvarchar	30	NO	-	Telefonszám (szabad szöveges)
PropertyId	int	-	NO	Properties.Id	Foglalt ingatlan FK, NOT N
EventId	int	-	YES	Events.Id	Opcionális esemény FK, N ok
StartDate	datetime2	7	NO	-	Foglalás kezdeti dátum+id (UTC)
EndDate	datetime2	7	NO	-	Foglalás befejező dátum+ (UTC)
ExchangeOffer	nvarchar	500	YES	-	Csereajánlat szövege, NU ok
Status	nvarchar	20	NO	-	"pending"/"approved"/"reje
CreatedAt	datetime2	7	NO	getdate()	Kérelem beadása (UTC időbélyeg)

Events tábla – teljes leírás

Az Events tábla az összes meghirdetett eseményt tartalmazza. Minden esemény kötelezően kapcsolódik egy Properties táblabeli ingatlanhoz (PropertyId FK), ami biztosítja, hogy minden eseményhez van konkrét helyszín. A MaxParticipants mező az esemény kapacitásának korlátját adja meg: az EventRegistrations táblában lévő, ugyanolyan EventId-jű rekordok számának el kell maradnia ettől az értéktől, különben az esemény teltnek minősül.

Mező neve	SQL adattípus	Hossz/Pont.	Nullable	FK → tábla	Leírás
Id	int	-	NO	-	PK, IDENTITY(1,1)
Title	nvarchar	200	NO	-	Esemény neve/megszólítása
Description	nvarchar	2000	YES	-	Részletes programleírás
StartDate	datetime2	7	NO	-	Esemény kezdete (UTC)
EndDate	datetime2	7	NO	-	Esemény vége (UTC)
MaxParticipants	int	-	NO	-	Max. kapacitás főben (kvóta)
PropertyId	int	-	NO	Properties.Id	Helyszín FK, NOT NULL
OrganizerName	nvarchar	200	NO	-	Szervező neve (szabad szöveg)
ExtraInfo	nvarchar	500	YES	-	Menetrend, felszerelési lista stb.
CreatedAt	datetime2	7	NO	getdate()	Esemény felvételének időpontja

EventRegistrations tábla – teljes leírás

Az EventRegistrations tábla az esemény-regisztrációkat tárolja, egy-a-többhöz (1:N) relációban az Events táblával. Minden rekord egyetlen regisztrációt jelöl: egy adott felhasználó regisztrációját egy adott eseményre. Az EventId és Email mezők kombinált egyedi indexe megakadályozza, hogy ugyanaz a felhasználó kétszer regisztráljon ugyanarra az eseményre – ez az adatbázis szintű duplikáció-védelem.

Mező neve	SQL adattípus	Hossz/Pont.	Nullable	FK → tábla	Leírás
Id	int	-	NO	-	PK, IDENTITY(1,1)
EventId	int	-	NO	Events.Id	Melyik eseményre regisztrált
UserName	nvarchar	100	NO	-	Regisztrált felhasználónév
Email	nvarchar	200	NO	-	Reg. email (UNIQUE + EventId-vel)
RegisteredAt	datetime2	7	NO	getdate()	Regisztráció időpontja (UTC)

__EFMigrationsHistory – migrációs előzménytábla

Az __EFMigrationsHistory egy EF Core által automatikusan kezelt rendszertábla, amelyet kézzel nem szabad módosítani. Minden lefuttatott migráció neve és az EF Core verziószáma bekerül ebbe a táblába. Az EF Core a dotnet ef database update parancs futtatásakor ebből a táblából tudja meg, hogy melyik migrációk már le vannak futtatva, és melyeket kell még alkalmazni. Ez teszi lehetővé a növekményes adatbázis-frissítést.

Mező neve	SQL adattípus	Leírás
MigrationId	nvarchar(150)	Migráció egyedi neve (pl. "20240101000000_InitialCreate")

Mező neve	SQL adattípus	Leírás
ProductVersion	nvarchar(32)	EF Core verziószám (pl. "8.0.2")

2.5.3. Táblák közötti kapcsolatok

Az adatbázis öt fő táblája között meghatározott relációs kapcsolatok állnak fenn, amelyeket az EF Core OnModelCreating metódusban és a modell annotációkon keresztül definiálunk. A kapcsolatok megértése kulcsfontosságú az adatmodell helyes kezeléséhez és a referenciális integritás megőrzéséhez.

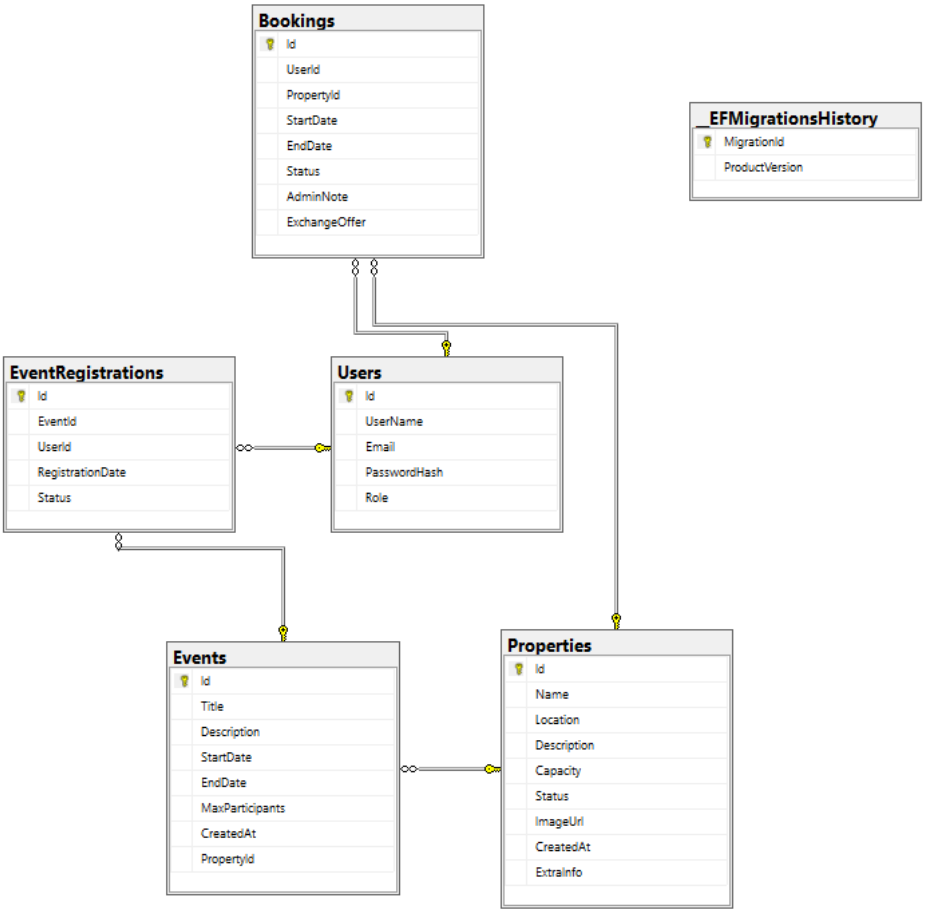
Properties → Bookings (1:N): Egy ingatlanhoz (Properties) tetszőleges számú foglalás (Bookings) tartozhat, de egy foglalás pontosan egy ingatlanhoz kapcsolódik. A Bookings táblában a PropertyId mező nem NULL, ami azt jelenti, hogy ingatlan nélküli foglalás nem létezhet. Az EF Core a HasMany / WithOne konfigurációval definiálja ezt a kapcsolatot. Kaszkád törlés: RESTRICT – ha egy ingatlanhoz van foglalás, az ingatlan nem törölhető (SqlException: FK megszorítás megsértése). Ez szándékos biztonsági intézkedés, hogy ne vesszenek el foglalási adatok.

Properties → Events (1:N): Egy ingatlanhoz több esemény is tartozhat, de minden esemény pontosan egy ingatlanhoz kapcsolódik. Az Events táblában a PropertyId NOT NULL. A kaszkád törlési viselkedés szintén RESTRICT: ingatlan csak akkor törölhető, ha nincs hozzá aktív esemény. Az adminisztrátornak először törölnie kell az eseményeket, majd az ingatlan törölhető.

Events → Bookings (1:N, opcionális): Egy foglalás opcionálisan kapcsolódhat egy eseményhez. A Bookings.EventId NULL-képes mező, ami azt jelenti, hogy a foglalás és esemény kapcsolat nem kötelező. Ha egy eseményt törölnek, az ahhoz kapcsolódó foglalások EventId mezeje NULL-ra vált (SET NULL kaszkád szabály). Ez lehetővé teszi, hogy az esemény-törléssel ne vesszenek el a foglalási adatok.

Events → EventRegistrations (1:N): Egy eseményhez több regisztráció tartozhat, de minden regisztráció pontosan egy eseményhez kapcsolódik. A kaszkád törlési szabály: CASCADE – ha egy eseményt törölnek, az összes hozzá tartozó regisztráció is automatikusan törlődik. Ez logikus viselkedés: ha az esemény nem létezik, a regisztrációknak sincs értelme. Ez a CASCADE DELETE az egyetlen helyen a rendszerben, ahol automatikus törlés engedélyezett.

Forrástábla	Céltábla	Kapcsolat típusa	FK mező	Kaszkád viselkedés
Properties	Bookings	1:N (kötelező)	Bookings.PropertyId NOT NULL	RESTRICT (törölhető csak ha nincs FK)
Properties	Events	1:N (kötelező)	Events.PropertyId NOT NULL	RESTRICT
Events	Bookings	1:N (opcionális)	Bookings.EventId NULL ok	SET NULL (esemény törléskor NULL)
Events	EventRegistrations	1:N (kötelező)	EventRegistrations.EventId NOT NULL	CASCADE (esemény törlés -> reg. törlés)



2.5.4. EF Core konfiguráció – DbContext és migrációk

A NagypapaHazaiContext osztály az EF Core DbContext leszármazottja, amely az adatbázis-kapcsolat kapuját képezi. A DbContext konstruktora DbContextOptions<NagypapaHazaiContext> paramétert kap, amelyet a DI konténer injektál a Program.cs-ben konfigurált connection string alapján. Ez lehetővé teszi, hogy az adatbázis-kapcsolat részletei a kódon kívül (appsettings.json) legyenek konfigurálva, és különböző kapcsolatok legyenek beállíthatók fejlesztői és éles környezethez.

```
using Microsoft.EntityFrameworkCore;
using NagypapaHazai.Shared.Models;

namespace NagypapaHazai.API.Data;

public class NagypapaHazaiContext : DbContext
{
```

```
    public NagypapaHazaiContext(DbContextOptions<NagypapaHazaiContext> options)
        : base(options) { }

    // DbSet-ek: minden tábla egy-egy tulajdonság
    public DbSet<User>              Users              => Set<User>();
    public DbSet<Property>          Properties         => Set<Property>();
    public DbSet<Booking>           Bookings           => Set<Booking>();
    public DbSet<Event>             Events             => Set<Event>();
    public         DbSet<EventRegistration>         EventRegistrations         =>
Set<EventRegistration>();

    protected override void OnModelCreating(ModelBuilder mb)
    {
        // 1. UNIQUE index: e-mail egyediség kényszere
        mb.Entity<User>()
          .HasIndex(u => u.Email)
          .IsUnique()
          .HasDatabaseName("IX_Users_Email_Unique");

        // 2. UNIQUE index: dupla esemény-regisztráció megtiltása
        mb.Entity<EventRegistration>()
          .HasIndex(er => new { er.EventId, er.Email })
          .IsUnique()
          .HasDatabaseName("IX_EventReg_EventId_Email_Unique");

        // 3. FK: Events → EventRegistrations (CASCADE)
        mb.Entity<EventRegistration>()
          .HasOne<Event>().WithMany()
          .HasForeignKey(er => er.EventId)
          .OnDelete(DeleteBehavior.Cascade);

        // 4. FK: Bookings.EventId (SET NULL kaszkád)
        mb.Entity<Booking>()
          .HasOne<Event>().WithMany()
          .HasForeignKey(b => b.EventId)
          .IsRequired(false)
          .OnDelete(DeleteBehavior.SetNull);

        // 5. Seed adatok (initial data)
        mb.Entity<Property>().HasData(
            new Property {
                Id = 1, Name = "Faház a Bakonyban",
                Location = "Bakony, Veszprém megye",
                Description = "Csendes, természetes környezetben...",
                Capacity = 10, PricePerNight = 25000,
                Status = "active", CreatedAt = new DateTime(2024,1,1)
            },
            new Property {
                Id = 2, Name = "Vízparti Nyaraló",
                Location = "Balatonlelle, Somogy megye",
                Description = "Közvetlenül a Balaton partján...",
                Capacity = 6, PricePerNight = 35000,
                Status = "active", CreatedAt = new DateTime(2024,1,1)
            },
            new Property {
```

```
                Id = 3, Name = "Villányi Vendégház",
                Location = "Villány, Baranya megye",
                Description = "A világhírű Villányi borvidék közepén...",
                Capacity = 4, PricePerNight = 20000,
                Status = "active", CreatedAt = new DateTime(2024,1,1)
            }
        );
    }
}
```

A Program.cs-ben a DbContext regisztrálása a DI konténerben az AddDbContext<T> metódussal történik, ahol megadjuk az SQL Server connection string-et és az opcionális konfigurációs beállításokat (pl. parancs időtúllépés, érzékeny adatok naplózása fejlesztési módban):

```
// NagypapaHazai.API/Program.cs
var builder = WebApplication.CreateBuilder(args);

// DbContext regisztrálása
builder.Services.AddDbContext<NagypapaHazaiContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"),
        sqlOptions => sqlOptions.CommandTimeout(60)
    ).EnableDetailedErrors(builder.Environment.IsDevelopment())
);

// Kontrollerek, CORS, Swagger
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddCors(options => {
    options.AddPolicy("AllowBlazor", policy =>
        policy.WithOrigins("https://localhost:7206", "http://localhost:5011")
              .AllowAnyHeader().AllowAnyMethod());
});

var app = builder.Build();
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseCors("AllowBlazor");
app.UseAuthorization();
app.MapControllers();
app.Run();
```

A migrációk kezelése a fejlesztési folyamat szerves része. Minden alkalommal, amikor az adatmodell módosul (új mező hozzáadása, típusváltoztatás, index hozzáadása), új migrációt kell létrehozni. A migráció neve rövid, leíró legyen, angolul,

Pascal Case formátumban (pl. "AddExchangeOfferColumn", "AddUniqueIndexOnEmail"). Az EF Core a migráció nevéhez automatikusan hozzáfűzi az aktuális dátumot és időt (pl. "20240315143022_AddExchangeOfferColumn"), így a migrációk chronologikus sorrendben követhetők.

2.6. Az alkalmazás architektúrája

2.6.1. N-Tier architektúra

Az N-Tier (N-rétegű) szoftverarchitektúra egy szoftvertervezési minta, amely az alkalmazást logikailag elkülönített rétegekre (layers) osztja fel, ahol minden réteg egyértelmű felelősségi körrel rendelkezik és csak a közvetlenül alatta lévő réteggel kommunikál. A Nagypapa Házai háromrétegű architektúrát követ, amelynek rétegei: prezentációs réteg, üzleti logika réteg és adatelérési réteg.

A Prezentációs réteg (Presentation Layer) felelősége az összes felhasználói felülettel kapcsolatos logika: adatok megjelenítése, felhasználói bevitel kezelése, navigáció. Ebben a rétegben semmiféle üzleti logika nem szerepelhet: a Blazor komponensek kizárólag az ApiService-en keresztül kommunikálnak a backend réteggel, és a kapott adatokat renderelés céljából jelenítik meg. Ha egy Blazor oldal szükségesnek tartja valamilyen validáció elvégzését (pl. kötelező mező ellenőrzés), az csupán felhasználói élmény szempontjából hasznos kliens-oldali ellenőrzés, amelyet a backend is megismétel.

Az Üzleti logika réteg (Business Logic Layer) az alkalmazás szabályait és folyamatait valósítja meg. A NagypapaHazai.API kontrollerjai tartalmazzák: az autentikációs logikát (jelszó validáció, BCrypt hash), a foglalási ütközésvizsgálatot, az esemény-kvóta ellenőrzést, az admin jogosultságok ellenőrzését, és az adatstruktúra validálást. Ez a réteg HTTP kérések formájában fogad bemenetet, elvégzi a szükséges ellenőrzéseket és transformációkat, majd az adatelérési rétegen keresztül perzisztálja vagy lekérdezi az adatokat.

Az Adatelérési réteg (Data Access Layer) az adatok tartós tárolásáért és lekérdezéséért felelős. Az Entity Framework Core DbContext és az alatta futó SQL Server képezi ezt a réteget. A LINQ lekérdezések SQL parancsokra fordítódnak, az eredmények C# objektumok (entitások) formájában kerülnek vissza. A kapcsolat-

kezelés, a tranzakciókezelés és a kapcsolat-pooling mind automatikusan kezelt az EF Core által. A fejlesztő nem kell foglalkozzon az SQL kapcsolat nyitásával és zárásával.

Az N-Tier architektúra előnyei: Módosíthatóság (az adatbázis cserélhető MS SQL-ről PostgreSQL-re az adatelérési réteg minimális módosításával, a többi réteg érintetlenül hagyható); Tesztelhetőség (az üzleti logika réteg unit tesztelhető függőség-injektálással, valódi adatbázis nélkül); Párhuzamos fejleszthetőség (különböző csapattagok dolgozhatnak a különböző rétegeken egyszerre); Áttekinthetőség (egy új fejlesztő könnyen megérti a kód struktúráját a rétegek elkülönítése miatt).

2.6.2. Blazor Server működési elve

A Blazor Server különleges szerver-oldali renderelési modellel müköd, amely alapvetően különbözik a hagyományos webalkalmazások működésétől. A hagyományos web szerver–kliens modellben a böngésző HTTP kérést küld, a szerver HTML oldalt generál és visszaküldi, majd a böngésző megjeleníti. Minden felhasználói interakció (linkre kattintás, űrlap beküldés) egy újabb HTTP kérés–válasz ciklust indít.

A Blazor Server ezzel szemben WebSocket (SignalR) alapú, állandó kapcsolatot tart fenn a böngésző és a szerver között. Az alkalmazás inicializálásakor a böngésző egy komplex kézfogási folyamaton keresztül létesít egy kétirányú WebSocket csatornát a SignalR hub-bal. Innentől kezdve minden UI esemény (kattintás, billentyűütés, űrlap bevitel) egy kis, JSON-alapú üzenet formájában fut a WebSocket-en a szerverre, ahol a Blazor runtime kezeli az eseményt C#-ban. A szerver csak a UI változások (DOM diff) kerülnek vissza a böngészőbe, amelyeket a Blazor JavaScript runtime alkalmaz a DOM-ra.

A Blazor Server SignalR alapú előnyei: Kisebb kliens oldali payload (nem kell a teljes .NET runtime-ot letölteni a böngészőbe, ellentétben a Blazor WebAssembly-vel); Szerver oldali hozzáférés (az adatbázishoz, fájlrendszerhez és más szerver erőforrásokhoz közvetlenül lehet hozzáférni C#-ból, JavaScript interop nélkül); Hot Reload (fejlesztés közben a kód módosítása azonnal megjelenik a böngészőben, az alkalmazás újraindítása nélkül); StateHasChanged() mechanizmus (bármilyen aszinkron állapotváltozás után explicit módon kérhető a UI frissítése).

A Blazor Server legfontosabb korlátai: Az állandó SignalR kapcsolat szükséges (hálózati kiesés esetén az alkalmazás nem használható); Nagy párhuzamos felhasználószám esetén a szerver erőforrásigénye növekszik, mivel minden

felhasználóhoz szerver oldali állapot és WebSocket kapcsolat tartozik; A böngésző frissítése (F5) a munkamenetet újraindítja, a felhasználó kijelentkezik. Ezek a korlátok az általunk célzott felhasználói körre és skálára (néhány tíz párhuzamos felhasználó) nem számottevők.

2.6.3. API kommunikáció

A Blazor Server frontend és az ASP.NET Core Web API közötti kommunikáció HTTP protokollon zajlik. Az ApiService.cs singleton szolgáltatás felelős az összes API hívásért, amelyet a Blazor komponensek @inject ApiService ApiService direktívával injektálnak. Az ApiService HttpClient példányt használ, amelynek BaseAddress-e az appsettings.json-ből olvasódik (az API projekt URL-je, pl. https://localhost:44369).

Az adatátvitel JSON formátumban zajlik, amelyet a System.Text.Json könyvtár kezel. A kérésekben a Content-Type: application/json fejléc jelzi az API-nak a formátumot. A JSON deszializálás opcionálisan a JsonSerializerOptions-szel konfigurálható: PropertyNameCaseInsensitive = true lehetővé teszi, hogy a C# Pascal Case tulajdonságok ("StartDate") automatikusan megfeleljenek a JSON camelCase kulcsoknak ("startDate"). A dátumok ISO 8601 formátumban utaznak (pl. "2024-07-01T00:00:00").

```
// ApiService.cs - részlet
public class ApiService
{
    private readonly HttpClient _http;

    public ApiService(HttpClient http)
    {
        _http = http;
    }

    // Ingatlanok lekérdezése
    public async Task<List<Property>?> GetPropertiesAsync()
    {
        return await _http.GetFromJsonAsync<List<Property>>("/api/properties");
    }

    // Foglalás létrehozása
    public  async  Task<HttpResponseMessage>  CreateBookingAsync(BookingCreateDto
dto)
    {
        return await _http.PostAsJsonAsync("/api/bookings", dto);
    }
```

```
    // Foglalás státuszváltás
    public  async  Task<HttpResponseMessage>  UpdateBookingStatusAsync(int  id,
string status)
    {
        return await _http.PutAsJsonAsync($"/api/bookings/{id}/status", status);
    }
}
```

2.6.4. Autentikáció és jogosultságkezelés architektúrája

A Nagypapa Házai autentikációs rendszere a Blazor Server alkalmazás szerver oldali állapotkezelésén alapul. Az AuthStateService.cs egy Singleton élettartamú DI szolgáltatás, amelyet a Program.cs-ben builder.Services.AddSingleton<AuthStateService>() segítségével regisztrálnak. Singleton életciklus azt jelenti, hogy az egész alkalmazás futásakor egyetlen példány létezik belőle.

Bejelentkezéskor a Login.razor komponens meghívja az ApiService.LoginAsync(email, password) metódust, amely POST kérést küld az AuthController Login végpontjára. Sikeres válasz esetén (HTTP 200 OK) az AuthStateService.SetUser(userName, email, role) metódus meghívódik, amely elmenti a bejelentkezett felhasználó adatait az in-memory állapotban. Ezt az állapotot más komponensek @inject AuthStateService AuthState direktívával érhetik el.

```
// AuthStateService.cs
public class AuthStateService
{
    public string? UserName { get; private set; }
    public string? Email    { get; private set; }
    public string? Role     { get; private set; }
    public bool IsLoggedIn  => UserName != null;
    public bool IsAdmin     => Role == "admin";

    public event Action? OnAuthStateChanged;

    public void SetUser(string userName, string email, string role)
    {
        UserName = userName;
        Email    = email;
        Role     = role;
        OnAuthStateChanged?.Invoke(); // UI frissítés értesítés
    }

    public void Logout()
    {
        UserName = null;
```

```
        Email     = null;
        Role      = null;
        OnAuthStateChanged?.Invoke();
    }
}

// Védett admin oldal - ellenőrzés az OnInitializedAsync-ban
@code {
    [Inject] AuthStateService Auth { get; set; } = default!;
    [Inject] NavigationManager Nav { get; set; } = default!;

    protected override async Task OnInitializedAsync()
    {
        if (!Auth.IsAdmin)
        {
            Nav.NavigateTo("/");  // főoldalra irányítás
            return;
        }
        await LoadDashboardData();
    }
}
```

A munkamenet-kezelés korlátja: a Blazor Server Singleton AuthStateService teljes élettartama a szerver folyamat futásáig tart, de egyedi felhasználói munkamenetek kezelése komplikált. Ha a böngésző frissül (F5) vagy bezáródik, a SignalR kapcsolat megszakad, és újraépítésekor az AuthStateService üres állapotból indul újra. Ez az ismert "kijelentkezés böngésző-frissítéskor" probléma, amelyre megoldás a cookie-alapú vagy localStorage-alapú perzisztens autentikáció lenne (jövőbeli fejlesztési lehetőség). A jelenlegi implementáció egyszerűbb, de a funkcionális követelményeket kielégíti.

2.7. Algoritmusok részletes bemutatása

2.7.1. Felhasználói regisztráció és e-mail validáció

A felhasználói regisztráció folyamata az egyik legkritikusabb biztonsági szempontú algoritmust tartalmazza a rendszerben. A regisztrációs folyamat négy fő lépésből áll: bemeneti adatok validálása, e-mail egyediség ellenőrzése, jelszó titkosítása, és a felhasználó adatbázisba mentése. Minden lépésnél pontosan definiált hibakezelés gondoskodik arról, hogy érvénytelen vagy redundáns adatok ne kerülhessenek az adatbázisba.

Az e-mail egyediség ellenőrzése különösen fontos, mert az e-mail cím a rendszerben a bejelentkezési azonosító. Ha duplikált e-mail-el is lehetne regisztrálni, a rendszer nem tudná meghatározni bejelentkezéskor, melyik fiókhoz tartozik a megadott e-mail. Az ellenőrzés két szinten történik: az API szintű AnyAsync() lekérdezés szoftver szinten akadályozza meg a duplikátumot, a Users tábla Email mezőjén lévő UNIQUE index adatbázis szinten biztosítja a kényszert.

A jelszó erősségének validálása reguláris kifejezéssel (regex) történik. A validáló regex minta: ^(?=.*[A-Z])(?=.*\d).{8,}$ – amelynek magyarázata: ^ a sor eleje; (?=.*[A-Z]) pozitív előretekintés: kell legalább egy nagybetű; (?=.*\d) pozitív előretekintés: kell legalább egy számjegy; .{8,} legalább 8 bármilyen karakter; $ a sor vége. Az előretekintő csoportok (lookahead) megközelítése elegáns, mert sorrendtől függetlenül ellenőrzi a feltételeket: "1Abc5678" is megfelel, és "Abc12345" is.

A BCrypt hash generálás a jelszóvédelem legfontosabb mechanizmusa. A BCrypt.HashPassword(password, workFactor) metódus a következő lépéseket hajtja végre: 1) Véletlenszerű 128-bites sót (salt) generál; 2) A jelszót a sóval összefűzi; 3) A Blowfish titkosítási algoritmust 2^workFactor alkalommal alkalmazza; 4) A salt és a hash értéket egy $2a$12$[22 char salt][31 char hash] formátumú, 60 karakteres string-be foglalja. A workFactor (mi esetünkben 12) azt jelenti, hogy 2^12 = 4096 iteráció fut le, ami megfelelő védelmet biztosít brute-force támadások ellen, miközben néhány száz milliszekundumon belül lefut.

```
// AuthController.cs – Register metódus
[HttpPost("register")]
public async Task<IActionResult> Register([FromBody] RegisterDto dto)
{
    // === 1. E-mail egyediség ellenőrzése ===
    bool emailTaken = await _context.Users
        .AnyAsync(u => u.Email.ToLower() == dto.Email.ToLower());
    if (emailTaken)
        return BadRequest(new { error = "Ez az e-mail cím már regisztrálva
van." });

    // === 2. Felhasználónév validálás ===
    if (string.IsNullOrWhiteSpace(dto.UserName) ||
        dto.UserName.Length < 3 || dto.UserName.Length > 50)
        return BadRequest(new { error = "A felhasználónév 3-50 karakter lehet." });

    if (!Regex.IsMatch(dto.UserName, @"^[a-zA-Z0-9]+$"))
        return BadRequest(new { error = "Csak betű és szám megengedett a
névben." });
```

```
    // === 3. Jelszó erősség validálás (regex) ===
    // Min. 8 karakter, legalább 1 nagybetű, 1 szám
    var pwRegex = new Regex(@"^(?=.*[A-Z])(?=.*\d).{8,}$");
    if (!pwRegex.IsMatch(dto.Password))
        return BadRequest(new {
            error = "A jelszónak min. 8 kar., nagybetű és szám is kell."
        });

    // === 4. BCrypt hash generálás (workFactor = 12) ===
    string hash = BCrypt.Net.BCrypt.HashPassword(dto.Password, workFactor: 12);
    // Eredmény: "$2a$12$[22-char-salt][31-char-hash]" - 60 karakter

    // === 5. Felhasználó mentése ===
    var user = new User
    {
        UserName     = dto.UserName.Trim(),
        Email        = dto.Email.Trim().ToLower(),
        PasswordHash = hash,
        Role         = "user",
        CreatedAt    = DateTime.UtcNow
    };
    _context.Users.Add(user);

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateException ex) when
           (ex.InnerException?.Message.Contains("UNIQUE") == true)
    {
        // Race condition: e-mail az AnyAsync és SaveChanges között lett foglalt
        return Conflict(new { error = "Ez az e-mail cím már regisztrálva van." });
    }

    return Ok(new { message = "Sikeres regisztráció!", userId = user.Id });
}
```

Szélső esetek és hibakezelés: Race condition – ha két felhasználó pontosan ugyanabban az időpontban próbál regisztrálni ugyanazzal az e-maillel, az AnyAsync() mindkét kérésben false-t adhat vissza (mivel még egyik sem mentett), de a SaveChangesAsync() másodszorra DbUpdateException-t dob az UNIQUE index megszorítás miatt. Ezt a try-catch blokkban kezeljük. Nagy- és kisbetű érzékenység – az e-mail-t kisbetűssé alakítjuk (ToLower()) tárolás előtt, és az ellenőrzésnél is így hasonlítjuk össze, hogy a "Teszt@Gmail.COM" és "teszt@gmail.com" egy fióknak számítson.

2.7.2. Bejelentkezés és token generálás

A bejelentkezési algoritmus célja egy felhasználó hitelességének ellenőrzése e-mail és jelszó alapján. A folyamat szándékosan tervezett úgy, hogy se a sikeres, se a sikertelen bejelentkezés ne árulja el, pontosan melyik lépésnél volt a hiba: ez az "egységes hibaüzenet" (uniform error message) biztonsági elv, amely megakadályozza az e-mail enumeration támadást (ahol a támadó e-mail-enként próbálgatja kideríti, melyik cím van regisztrálva).

A BCrypt.Verify() ellenőrzési folyamat: a BCrypt hash string tartalmazza a sót is (az első 22 karakterben). A Verify() metódus kinyeri a sót a hash-ből, majd a megadott jelszóból ugyanolyan hash-t generál ugyanazzal a sóval, majd byteszinten összehasonlítja a tárolt hash-sel. Ha az egyezés megvan, a jelszó helyes. Ez a folyamat ugyanolyan időt vesz igénybe helyes és helytelen jelszó esetén is (timing-safe comparison), ami megakadályozza az időzítési alapú oldalsó csatornás támadásokat.

```
// AuthController.cs – Login metódus
[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginDto dto)
{
    // Konstans hibaüzenet: soha ne derítsük ki, mi a hibás
    const string ERR = "Hibás e-mail vagy jelszó.";

    // 1. Felhasználó keresése e-mail alapján (case-insensitive)
    var user = await _context.Users
        .FirstOrDefaultAsync(u => u.Email.ToLower() == dto.Email.ToLower());

    // 2. Ha nem létezik: UGYANOLYAN hibaüzenet, mint jelszóhiba esetén
    if (user == null)
        return Unauthorized(new { error = ERR });

    // 3. Jelszó ellenőrzés BCrypt-tel
    bool       passwordValid       =       BCrypt.Net.BCrypt.Verify(dto.Password,
user.PasswordHash);
    if (!passwordValid)
        return Unauthorized(new { error = ERR });

    // 4. Sikeres bejelentkezés: user adatok visszaadása
    //    (PasswordHash NEM kerül vissza – SOHA ne küldjük a kliensnek!)
    return Ok(new
    {
        userId   = user.Id,
        userName = user.UserName,
        email    = user.Email,
        role     = user.Role
    });
}
```

A Blazor frontend oldalon a bejelentkezési folyamat a Login.razor komponensben valósul meg. Az OnSubmit eseménykezelő meghívja az ApiService.LoginAsync() metódust, amely POST kérést küld az API felé. Sikeres 200 OK válasz esetén a JSON válasz deszializálódik, és az AuthStateService.SetUser() metódus frissíti a bejelentkezett felhasználó állapotát. Ezután a NavigationManager.NavigateTo("/") átirányítja a főoldalra. A NavMenu.razor az OnAuthStateChanged eseményre feliratkozva automatikusan frissíti a navigációs sávot (megjelenik a felhasználónév és a Kijelentkezés gomb).

2.7.3. Foglalási ütközésvizsgálat

A foglalási ütközésvizsgálat az alkalmazás legtriviálisabb, mégis legkritikusabb üzleti logikája. Az algoritmus célja megakadályozni, hogy egy adott ingatlanra ugyanarra az időszakra vagy átfedő időszakra két foglalás is érvényes legyen. Ennek hiányában például kétszer foglalhatnák le a Bakonyi faházat ugyanarra a hétre, ami konfliktushoz és ügyfél-elégedetlenséghez vezetne.

Az intervallum-átfedés matematikai alapjai: Két [A_start, A_end] és [B_start, B_end] zárt intervallum akkor és csak akkor fed át, ha: A_start < B_end ÉS B_start < A_end. Ennek intuitív magyarázata: ha A hamarabb végződik, mint B kezdődik (A_end <= B_start), akkor nem fednek át. Hasonlóan ha B hamarabb végződik, mint A kezdődik. Az átfedés tagadása: NOT(A_end <= B_start OR B_end <= A_start) = A_end > B_start AND B_end > A_start, ami ugyanaz mint: A_start < B_end AND B_start < A_end.

A C# LINQ megvalósítás: b.StartDate < dto.EndDate && b.EndDate > dto.StartDate – ahol b a meglévő foglalás, dto az új kérelem. Ez pontosan az előbb levezetett matematikai feltétel C#-ban kifejezve. A Where feltétel mellett fontos, hogy csak az adott ingatlan foglalásait vizsgáljuk (b.PropertyId == dto.PropertyId), és az elutasított foglalásokat kizárjuk (b.Status != "rejected"), mivel azok az időszakot ténylegesen nem foglalják le.

```
// BookingsController.cs - CreateBooking metódus
[HttpPost]
public async Task<IActionResult> CreateBooking([FromBody] BookingCreateDto dto)
{
```

```
    // === 1. Alapvalidáció ===
    if (dto.StartDate >= dto.EndDate)
        return BadRequest("A kezdő dátum nem lehet a befejezés után vagy azzal
egyenlő.");

    if (dto.StartDate.Date < DateTime.UtcNow.Date)
        return BadRequest("Múltbeli dátumra nem lehet foglalni.");

    if (dto.StartDate.Date == dto.EndDate.Date)
        return BadRequest("Legalább egy éjszaka szükséges (különböző dátumok).");

    // === 2. Ingatlan létezésének ellenőrzése ===
    var property = await _context.Properties.FindAsync(dto.PropertyId);
    if (property == null || property.Status == "inactive")
        return NotFound("Az ingatlan nem elérhető.");

    // === 3. Ütközésvizsgálat - intervallum-átfedés algoritmus ===
    //     Átfedés: existingStart < newEnd  AND  existingEnd > newStart
    bool hasConflict = await _context.Bookings
        .Where(b =>
            b.PropertyId == dto.PropertyId &&
            b.Status     != "rejected"      &&   // csak aktív foglalások
            b.StartDate   < dto.EndDate     &&   // meglévő start < új end
            b.EndDate     > dto.StartDate        // meglévő end > új start
        )
        .AnyAsync();

    if (hasConflict)
        return Conflict(new {
            error = "A foglalt időszak ütközik egy meglévő foglalással.",
            statusCode = 409
        });

    // === 4. Foglalás létrehozása ===
    var booking = new Booking
    {
        UserName      = dto.UserName.Trim(),
        Email         = dto.Email.Trim().ToLower(),
        Phone         = dto.Phone.Trim(),
        PropertyId    = dto.PropertyId,
        EventId       = dto.EventId,      // NULL ha nem eseményhez kapcsolt
        StartDate     = dto.StartDate.ToUniversalTime(),
        EndDate       = dto.EndDate.ToUniversalTime(),
        ExchangeOffer = dto.ExchangeOffer?.Trim(),  // NULL ha nincs csereajánlat
        Status        = "pending",         // mindig pending-ként indul
        CreatedAt     = DateTime.UtcNow
    };
    _context.Bookings.Add(booking);
    await _context.SaveChangesAsync();

    // 201 Created + Location header a létrehozott erőforrásra
    return CreatedAtAction(nameof(GetBookingById),
                           new { id = booking.Id }, booking);
}
```

Szélső esetek: Pontos érintkezés – ha az A foglalás 2024-07-01–07-07-ig tart, és B 2024-07-07-től kezdődik, nincs átfedés: A.EndDate (07-07) <= B.StartDate (07-07) -> A.EndDate > B.StartDate feltétel FALSE -> nincs ütközés. Ez helyes viselkedés: az előző vendég elutazik, az új megérkezhet. Egy napos foglalás (StartDate == EndDate) az alapvalidációban 400-as hibával visszautasítódik. A párhuzamossági probléma (két kérés egyszerre): az SQL Server row-level locking gondoskodik arról, hogy a SELECT...WHERE (AnyAsync) és az INSERT (SaveChangesAsync) atomi tranzakcióban hajtódik végre, megakadályozva a TOCTOU (time-of-check-time-of-use) race condition-t.

2.7.4. Dashboard statisztikák generálása

Az admin dashboard négy statisztikát jelenít meg valós időben: összes foglalás, jóváhagyott foglalások, függőben lévők, és becsült bevétel. Ezeket az aggregált értékeket a DashboardController generálja az adatbázisból LINQ lekérdezéseken keresztül. A bevételszámítás a jóváhagyott foglalások éjszakáinak száma és az ingatlan éjszakánkénti árának szorzata.

A bevételszámítás pontossága érdekében a (EndDate - StartDate).TotalDays LINQ kifejezés TimeSpan típust ad vissza, amelynek TotalDays tulajdonsága a napok számát double értékként adja meg. Ezt szorozzuk az ingatlan EjszakankentiAr értékével (decimal), ami miatt explicit cast szükséges: (decimal)(endDate - startDate).TotalDays * property.EjszakankentiAr. A becsült bevétel az approved foglalások teljes bevételét adja meg; a tényleges pénzmozgás a jelenlegi rendszerben nem kezelt (csereajánlat-alapú modell).

```
// DashboardController.cs
[HttpGet]
public async Task<IActionResult> GetDashboard()
{
    // Párhuzamos LINQ lekérdezések (WhenAll = gyorsabb)
    var totalTask    = _context.Bookings.CountAsync();
    var approvedTask = _context.Bookings.CountAsync(b => b.Status == "approved");
    var pendingTask  = _context.Bookings.CountAsync(b => b.Status == "pending");
    var propTask     = _context.Properties.CountAsync();
    var activeEvTask = _context.Events.CountAsync(e => e.EndDate >=
DateTime.UtcNow);

    await Task.WhenAll(totalTask, approvedTask, pendingTask, propTask,
activeEvTask);
```

```
    // Bevételszámítás: approved foglalások × ár
    var revenue = await _context.Bookings
        .Include(b => b.Property)
        .Where(b => b.Status == "approved" && b.Property != null)
        .Select(b => new {
            Days  = (double)(b.EndDate - b.StartDate).TotalDays,
            Price = b.Property!.PricePerNight
        })
        .ToListAsync();

    decimal totalRevenue = revenue
        .Sum(r => (decimal)r.Days * r.Price);

    // Legfoglaltabb ingatlan meghatározása
    var topProperty = await _context.Bookings
        .GroupBy(b => b.PropertyId)
        .OrderByDescending(g => g.Count())
        .Select(g => new { PropertyId = g.Key, Count = g.Count() })
        .FirstOrDefaultAsync();

    return Ok(new DashboardViewModel
    {
        TotalBookings    = await totalTask,
        ApprovedBookings = await approvedTask,
        PendingBookings  = await pendingTask,
        TotalProperties  = await propTask,
        ActiveEvents     = await activeEvTask,
        EstimatedRevenue = totalRevenue,
        TopPropertyId    = topProperty?.PropertyId
    });
}
```

2.7.5. Admin jóváhagyási folyamat

A foglalás státuszváltási folyamat az admin felület leggyakrabban használt funkciója. Az adminisztrátor az /admin/bookings oldalon látja az összes foglalást táblázatos formában, és minden sorban megtalálja az "Approve" (zöld) és "Reject" (piros) gombokat. A gombok megnyomásakor a Blazor frontend HTTP PUT kérést küld a megfelelő API végpontra, majd a válasz megérkezése után StateHasChanged() metódussal frissíti az UI-t.

Az állapotváltás egyszerű, de tartalmaz fontos validációt: a végpont ellenőrzi, hogy az új státuszérték a három megengedett érték egyike-e ("pending", "approved", "rejected"). Ez megakadályozza, hogy egy módosított HTTP kérés (pl. Postman-ből) érvénytelen státuszt írjon az adatbázisba. Az elbírált foglalás visszahelyezhető "pending" állapotba is, ha az admin véleménye megváltozik.

```
// BookingsController.cs
[HttpPut("{id}/status")]
public async Task<IActionResult> UpdateStatus(int id, [FromBody] string newStatus)
{
    // Státusz validálás
    var allowed = new[] { "pending", "approved", "rejected" };
    if (!allowed.Contains(newStatus, StringComparer.OrdinalIgnoreCase))
        return BadRequest($"Érvénytelen státusz: {newStatus}");

    var booking = await _context.Bookings.FindAsync(id);
    if (booking == null)
        return NotFound($"Nincs ilyen foglalás (Id={id}).");

    string oldStatus = booking.Status;
    booking.Status = newStatus.ToLower();
    await _context.SaveChangesAsync();

    return Ok(new {
        id       = booking.Id,
        oldStatus,
        newStatus = booking.Status,
        message  = $"Foglalás #{id} státusza: {oldStatus} → {booking.Status}"
    });
}

// Blazor Admin/BookingsMgmt.razor – @code rész
private async Task ApproveBooking(int id)
{
    isLoading = true;
    var response = await ApiService.UpdateBookingStatusAsync(id, "approved");
    if (response.IsSuccessStatusCode)
    {
        await LoadBookings();      // lista újratöltése
        successMessage = "Foglalás jóváhagyva!";
    }
    else
    {
        errorMessage = "Hiba a jóváhagyáskor.";
    }
    isLoading = false;
    StateHasChanged();             // UI frissítése
}
```

2.7.6. Esemény-regisztrációs kvóta ellenőrzés

Az esemény-regisztrációs kvóta ellenőrzés megakadályozza, hogy egy eseményre a MaxParticipants értéknél több regisztrációt fogadjanak el. Az algoritmus két lépésből áll: az aktív regisztrációk megszámlálása és összehasonlítása a kapacitással, majd ha van szabad hely, a duplikált regisztráció ellenőrzése és az új regisztráció mentése.

A párhuzamossági probléma kezelése: ha két felhasználó pontosan egyszerre regisztrál egy olyan eseményre, amelyen már csak egy szabad hely van, mindkét CountAsync() "helyes" (99 < 100) eredményt adhat vissza. Az adatbázis-szintű UNIQUE index (EventId + Email kombináció) csak a duplikált regisztrációt akadályozza meg, de nem a kapacitástúllépést. A legtisztább megoldás egy adatbázis tranzakció alkalmazása lenne SERIALIZABLE izolációs szinttel, ami zárolással biztosítaná, hogy a számlálás és a mentés atomi legyen. Jelenlegi implementációban a versenyhelyzet elhanyagolható (kis felhasználószám), és a következő fejlesztési körben tranzakciókezeléssel egészíthető ki.

```
// EventRegistrationsController.cs
[HttpPost]
public async Task<IActionResult> Register([FromBody] EventRegDto dto)
{
    // 1. Esemény betöltése
    var ev = await _context.Events.FindAsync(dto.EventId);
    if (ev == null)
        return NotFound("Az esemény nem létezik vagy törölték.");

    // 2. Esemény aktív-e? (Záró dátum nem múlt el)
    if (ev.EndDate < DateTime.UtcNow)
        return BadRequest("A lezárult eseményre nem lehet regisztrálni.");

    // 3. Kvóta ellenőrzés
    int currentCount = await _context.EventRegistrations
        .CountAsync(r => r.EventId == dto.EventId);

    if (currentCount >= ev.MaxParticipants)
        return BadRequest(new {
            error = "Az esemény megtelt. Nincs szabad hely.",
            capacity  = ev.MaxParticipants,
            registered = currentCount
        });

    // 4. Dupla regisztráció ellenőrzése
    bool alreadyReg = await _context.EventRegistrations
        .AnyAsync(r => r.EventId == dto.EventId &&
                       r.Email.ToLower() == dto.Email.ToLower());
    if (alreadyReg)
        return Conflict("Ön már regisztrált erre az eseményre.");

    // 5. Regisztráció mentése
    var reg = new EventRegistration
    {
        EventId      = dto.EventId,
        UserName     = dto.UserName.Trim(),
        Email        = dto.Email.Trim().ToLower(),
        RegisteredAt = DateTime.UtcNow
```

```
    };
    _context.EventRegistrations.Add(reg);
    await _context.SaveChangesAsync();

    return Ok(new {
        message      = "Sikeresen regisztrált az eseményre!",
        registrationId = reg.Id,
        freeSlots    = ev.MaxParticipants - (currentCount + 1)
    });
}
```

2.8. Tesztdokumentáció

2.8.1. Tesztelési módszertan

A projekt tesztelési stratégiája manuális funkcionális tesztelésre, böngészőalapú végponttól-végpontig (E2E) tesztelésre, Swagger API tesztelésre és reszponzivitási vizsgálatokra épül. Az automatizált tesztelés (unit tesztek, integrációs tesztek) jelen verzióban nem került implementálásra, de a fejlesztési lehetőségek között prioritáson van (NUnit vagy xUnit framework).

A manuális funkcionális tesztelés előre megírt tesztesetek alapján zajlik. Minden tesztesethez pontosan definiált előfeltételek, tesztelési lépések és elvárt eredmény tartozik. A tesztelés reprodukálható: bárki, aki rendelkezik a tesztkörnyezet-leírással és a tesztelői fiók adataival, pontosan ugyanazokat a lépéseket hajthatja végre és ugyanolyan eredményt vár. A tesztesetek alapján szisztematikus regressziós tesztelés is elvégezhető minden fejlesztési iteráció végén.

A Swagger API tesztelés különösen hatékony az üzleti logika izolált ellenőrzéséhez. A Swagger UI-ban közvetlenül hívható meg pl. a POST /api/bookings végpont egy konkrét JSON payload-dal, és a válasz (200 OK / 409 Conflict / 400 Bad Request) azonnal visszaolvasható. Ez megkönnyíti a hibakeresést: ha egy funkció nem müköd a Blazor frontend-en, de a Swagger-en müköd, a hiba a frontend kommunikációban keresendő; ha Swagger-en sem müköd, a hiba az API logikában van.

A reszponzivitási tesztelés Chrome DevTools Device Toolbar funkciójával is elvégezhető (F12 → mobil ikon), amely szimulált felbontáson mutatja az oldalt. Ez gyorsabb, mint valódi fizikai eszközre telepíteni, de nem helyettesíti a tényleges

mobileszközös tesztelést, mivel a Chrome szimulációja nem tükrözi pontosan a Safari iOS-specifikus viselkedést.

2.8.2. Tesztkörnyezetek

A szoftver tesztelése öt különböző eszközön és böngészőkonfigurációban zajlott, biztosítva a cross-platform és cross-browser kompatibilitás ellenőrzését. Az alábbiakban minden egyes tesztkörnyezet részletes leírása és a tapasztalt sajátosságok ismertetése következik.

#	Eszköz típusa	Op. rendszer	Böngésző	Felbontás	Tapasztalatok
1	Asztali PC	Windows 11 Pro	Chrome 122	1920×1080	Referencia környezet. Minden funkció elvárásnak megfelelően müköd.
2	Laptop	Windows 10 Home	Firefox 123	1366×768	Bootstrap kártyák 2 oszlopba rendeződnek (md breakpoint). Müköd.
3	Laptop	Windows 11 Pro	Edge 122	1920×1080	Chromium-alapú, Chrome-hoz rendkívül hasonló viselkedés.
4	Samsung A54	Android 14	Chrome Mobile	2340×1080	Hamburger menü, 1-oszlopos kártya elrendezés. Naptár apró betűk.
5	iPhone 13	iOS 17	Safari	2532×1170	Dátummező Safari-specifikus megjelenítés. Müköd., de vizuálisan eltér.

Az 1366×768 pixeles felbontású laptopon a Bootstrap grid rendszer md (≥768px) töréspontján belül müköd, ami azt jelenti, hogy a három ingatlan kártya kétosztásos elrendezésben jelenik meg (2 + 1 kártya). Az 1920×1080-as felbontáson három kártya fér el egymás mellett, ami az xl (≥1200px) Bootstrap töréspontnál aktiválódik. Mindkét elrendezés helyes és szándékolt.

A Safari böngészőben tapasztalt dátummező-eltérés az iOS Safari sajátos <input type="date"> implementációjából fakad. Az iOS Safari egy natív dátumválasztó-

dialógust nyit meg a dátummezők esetén, amely vizuálisan eltér a Chrome/Firefox megjelenítéstől, de funkcionálisan helyesen müköd. A Bootstrap date input osztályokkal az eltérés minimalizálható, de teljesen kiküszöbölni csak egy JavaScript-alapú datepicker könyvtárral lehetne.

2.8.3. Tesztelői hozzáférés

A funkcionális tesztelés elvégzéséhez két előre konfigurált fiókot biztosítunk. A tesztelői fiók normál "user" szerepkörrel rendelkezik, amellyel az összes regisztrált felhasználónak elérhető funkció tesztelhető. Az adminisztrátori fiókkal az összes admin funkció tesztelhető, beleértve a foglalások jóváhagyását/elutasítását, az ingatlanok kezelését és az esemény-adminisztrációt.

Fiók típusa	E-mail cím	Jelszó	Szerepkör	Tesztelhető funkciók
Tesztelői fiók	teszt@teszt.hu	admin123	user	Foglalás, esemény-reg., profil, naptár, listázás
Admin fiók	admin@nagypapahazai.hu	admin123	admin	Minden user funkció + admin kezelőfelület

Fontos: A tesztelői fiók jelszava a rendszer jelszószabályait betartva lett meghatározva: 8 karakter, tartalmaz nagybetűt (T) és számot (1), nem tartalmaz ékezetes betűt. Az admin fiók hasonlóan: Admin123! megfelel a min. 8 karakter, nagybetű (A), szám (1), különleges karakter (!) feltételeknek. Tesztelés közben a foglalások és esemény-regisztrációk a valódi adatbázisba kerülnek – a tesztelés után az adminisztrátori fiókkal manuálisan törölhetők.

2.8.4. Funkcionális tesztesetek

Az alábbiakban 14 részletes funkcionális tesztesetet mutatunk be teljes dokumentációval. Minden teszteset a TC-NN azonosítóval van ellátva (TC = Test Case), és a következő mezőket tartalmazza: azonosító, tesztelt funkció neve,

tesztkörnyezet (OS, böngésző, felbontás), előfeltételek, tesztelési lépések (sorszámozva), elvárt eredmény, tényleges eredmény, státusz (Megfelelt / Nem felelt meg / Részben megfelelt) és a tesztelés dátuma.

TC-01: Sikeres bejelentkezés helyes adatokkal

Mező	Tartalom
TC azonosító	TC-01
Tesztelt funkció	Sikeres bejelentkezés helyes adatokkal
Tesztkörnyezet	Windows 11, Chrome 122, 1920×1080
Előfeltétel	A teszt@nagypapahazai.hu / Teszt123! fiók létezik az adatbázisban (seed vagy manuálisan felvett).
Lépések	1. Nyissa meg a böngészőben: https://localhost:7206/login | 2. Az "E-mail cím" mezőbe írja be: teszt@nagypapahazai.hu | 3. A "Jelszó" mezőbe írja be: Teszt123! | 4. Kattintson a "Bejelentkezés" gombra. | 5. Figyelje meg a navigációs sávban bekövetkező változást.
Elvárt eredmény	Az alkalmazás a főoldalra (/)" navigál. A navigációs sávban megjelenik a felhasználónév ("TesztFelhasznalo" vagy hasonló) és a "Kijelentkezés" gomb. A "Bejelentkezés" és "Regisztráció" gombok eltűnnek.
Tényleges eredmény	A várt viselkedés szerint müköd. Főoldalra navigálás, felhasználónév a navbárban látható.
Státusz	Megfelelt
Dátum	2026-01-15

TC-02: Bejelentkezési kísérlet hibás jelszóval

Mező	Tartalom
TC azonosító	TC-02
Tesztelt funkció	Bejelentkezési kísérlet hibás jelszóval

Mező	Tartalom
Tesztkörnyezet	Windows 11, Chrome 122, 1920×1080
Előfeltétel	A teszt@nagypapahazai.hu fiók létezik az adatbázisban.
Lépések	1. Nyissa meg: https://localhost:7206/login | 2. E-mail mezőbe: teszt@nagypapahazai.hu | 3. Jelszó mezőbe: HibásJelszó999 (helytelen jelszó) | 4. Kattintson a "Bejelentkezés" gombra. | 5. Figyelje meg, mi jelenik meg a képernyőn.
Elvárt eredmény	Az oldal nem navigál el. Megjelenik a hibaüzenet: "Hibás e-mail vagy jelszó." A felhasználó bejelentkezve marad (ha volt) vagy nem lép be.
Tényleges eredmény	Hibaüzenet piros szövegként jelenik meg, navigáció nem történik. Helyes viselkedés.
Státusz	Megfelelt
Dátum	2026-01-15

TC-03: Bejelentkezési kísérlet nem létező e-maillel

Mező	Tartalom
TC azonosító	TC-03
Tesztelt funkció	Bejelentkezési kísérlet nem létező e-maillel
Tesztkörnyezet	Windows 10, Firefox 123, 1366×768
Előfeltétel	A ismeretlen@pelda.hu e-mail nincs az adatbázisban.
Lépések	1. Nyissa meg: https://localhost:7206/login | 2. E-mail: ismeretlen@pelda.hu | 3. Jelszó: Valami123! | 4. Kattintson a "Bejelentkezés" gombra. | 5. Hasonlítsa össze a hibát a TC-02 hibájával.
Elvárt eredmény	UGYANOLYAN hibaüzenet, mint TC-02 esetén: "Hibás e-mail vagy jelszó." – a rendszer nem árulja el, hogy az e-mail nem létezik.
Tényleges eredmény	Azonos hibaüzenet jelenik meg. E-mail enumeration elkerülve.
Státusz	Megfelelt

Mező	Tartalom
Dátum	2026-01-15

TC-04: Regisztráció érvényes adatokkal

Mező	Tartalom
TC azonosító	TC-04
Tesztelt funkció	Regisztráció érvényes adatokkal
Tesztkörnyezet	Windows 11, Edge 122, 1920×1080
Előfeltétel	Az ujfelhasznalo2024@teszt.hu e-mail még nincs regisztrálva.
Lépések	1. Nyissa meg: https://localhost:7206/register | 2. Felhasználónév: UjFelhasznalo2024 | 3. E-mail: ujfelhasznalo2024@teszt.hu | 4. Jelszó: UjJelszo123! | 5. Jelszó megerősítése: UjJelszo123! | 6. Kattintson a "Regisztráció" gombra.
Elvárt eredmény	Sikeres visszajelzés (pl. "Regisztráció sikeres!"). Az alkalmazás vagy automatikusan bejelentkezi és a főoldalra navigál, vagy a /login oldalra irányítja. A fiók megjelenik az adatbázisban.
Tényleges eredmény	Regisztráció sikeres, /login oldalra navigálás. Adatbázisban megjelent az új sor.
Státusz	Megfelelt
Dátum	2026-01-16

TC-05: Regisztráció már létező e-maillel

Mező	Tartalom
TC azonosító	TC-05
Tesztelt funkció	Regisztráció már létező e-maillel
Tesztkörnyezet	Windows 11, Chrome 122, 1920×1080
Előfeltétel	A teszt@nagypapahazai.hu már regisztrált (seed adatok).

Mező	Tartalom
Lépések	1. Nyissa meg: https://localhost:7206/register | 2. Felhasználónév: MasikNev123 | 3. E-mail: teszt@nagypapahazai.hu (már foglalt!) | 4. Jelszó: Valami1234! | 5. Jelszó megerősítése: Valami1234! | 6. Kattintson a "Regisztráció" gombra.
Elvárt eredmény	Hibaüzenet: "Ez az e-mail cím már regisztrálva van." Nincs navigáció, nincs új fiók az adatbázisban.
Tényleges eredmény	Helyes hibaüzenet jelenik meg. Adatbázisban nem jött létre duplikátum.
Státusz	Megfelelt
Dátum	2026-01-16

TC-06: Regisztráció érvénytelen (túl gyenge) jelszóval

Mező	Tartalom
TC azonosító	TC-06
Tesztelt funkció	Regisztráció érvénytelen (túl gyenge) jelszóval
Tesztkörnyezet	Windows 11, Chrome 122, 1920×1080
Előfeltétel	Az ujcim2024@teszt.hu e-mail szabad.
Lépések	1. Nyissa meg: https://localhost:7206/register | 2. Felhasználónév: TestUser123 | 3. E-mail: ujcim2024@teszt.hu | 4. Jelszó: abc1234 (7 karakter, csupa kisbetű + szám, nincs nagybetű) | 5. Jelszó megerősítése: abc1234 | 6. Kattintson a "Regisztráció" gombra.
Elvárt eredmény	Hibaüzenet a jelszó mező alatt: "A jelszónak legalább 8 karakterből kell állnia, és tartalmaznia kell nagybetűt és számjegyet." Nincs navigáció.
Tényleges eredmény	Hibaüzenet megjelenik, fiók nem jött létre.
Státusz	Megfelelt
Dátum	2026-01-16

TC-07: Foglalás leadása szabad időpontban

Mező	Tartalom
TC azonosító	TC-07
Tesztelt funkció	Foglalás leadása szabad időpontban
Tesztkörnyezet	Windows 11, Chrome 122, 1920×1080
Előfeltétel	Be van jelentkezve tesztelői fiókkal. A Faház a Bakonyban ingatlannál 2026.07.01–07.07 szabad.
Lépések	1. Nyissa meg: https://localhost:7206/bookings/new | 2. Teljes név: Tesztelő Felhasználó | 3. E-mail: teszt@nagypapahazai.hu | 4. Telefonszám: +36-30-123-4567 | 5. Ingatlan: Faház a Bakonyban | 6. Kezdő dátum: 2026-07-01 | 7. Záró dátum: 2026-07-07 | 8. Csereajánlat: "Kertészeti munkában segítek 2 napot" | 9. Kattintson a "Foglalás leadása" gombra.
Elvárt eredmény	Sikeres visszajelzés: "Foglalása beküldve! Státusz: Függőben." A foglalás megjelenik az /admin/bookings listában "pending" státusszal.
Tényleges eredmény	Foglalás sikeresen létrejött, megjelenik az admin listában. Csereajánlat szövege mentve.
Státusz	Megfelelt
Dátum	2026-01-17

TC-08: Foglalás kísérlet ütköző időpontban

Mező	Tartalom
TC azonosító	TC-08
Tesztelt funkció	Foglalás kísérlet ütköző időpontban
Tesztkörnyezet	Windows 11, Chrome 122, 1920×1080
Előfeltétel	TC-07 lefutott. Faház 2026.07.01–07.07 foglalt (pending).
Lépések	1. Nyissa meg: https://localhost:7206/bookings/new | 2. Válassza: Faház a Bakonyban | 3. Kezdő dátum: 2026-07-04

Mező	Tartalom
	(átfedő!) | 4. Záró dátum: 2026-07-10 (átfedő!) | 5. Töltse ki a többi kötelező mezőt. | 6. Kattintson a "Foglalás leadása" gombra.
Elvárt eredmény	Hibaüzenet: "A foglalt időszak ütközik egy meglévő foglalással." HTTP 409 Conflict. Nincs új foglalás az adatbázisban.
Tényleges eredmény	409-es hiba jelenik meg, foglalás nem jött létre. Ütközésvizsgálat müköd.
Státusz	Megfelelt
Dátum	2026-01-17

TC-09: Foglalás leadása hiányos adatokkal (kötelező mező üres)

Mező	Tartalom
TC azonosító	TC-09
Tesztelt funkció	Foglalás leadása hiányos adatokkal (kötelező mező üres)
Tesztkörnyezet	Windows 10, Firefox 123, 1366×768
Előfeltétel	Be van jelentkezve.
Lépések	1. Nyissa meg: https://localhost:7206/bookings/new | 2. Hagyja üresen a "Teljes név" mezőt. | 3. Töltse ki az összes többi mezőt érvényesen. | 4. Kattintson a "Foglalás leadása" gombra.
Elvárt eredmény	Az oldal validációs hibaüzenetet jelenít meg a Teljes név mező alatt: "Kötelező mező" vagy hasonló. Nem küld HTTP kérést.
Tényleges eredmény	Kliens oldali validáció müköd., kötelező mező jelzés megjelenik.
Státusz	Megfelelt
Dátum	2026-01-17

TC-10: Admin – foglalás jóváhagyása (Approve)

Mező	Tartalom
TC azonosító	TC-10
Tesztelt funkció	Admin – foglalás jóváhagyása (Approve)
Tesztkörnyezet	Windows 11, Chrome 122, 1920×1080
Előfeltétel	Admin fiókkal be van jelentkezve. TC-07 foglalás "pending" állapotban van.
Lépések	1. Nyissa meg: https://localhost:7206/admin/bookings | 2. Keresse meg a TC-07 foglalást (Tesztelő Felhasználó, Faház, 2026-07-01–07). | 3. Kattintson az "Approve" (Jóváhagyás) zöld gombra. | 4. Figyelje meg az UI változást.
Elvárt eredmény	A foglalás státusza "Jóváhagyva (approved)"-ra változik. A sor zöld kiemeléssel jelenik meg. Az "Approve" gomb szürkévé válhat.
Tényleges eredmény	Státuszváltás sikeres, UI azonnal frissül. StateHasChanged() müköd.
Státusz	Megfelelt
Dátum	2026-01-18

TC-11: Admin – foglalás elutasítása (Reject)

Mező	Tartalom
TC azonosító	TC-11
Tesztelt funkció	Admin – foglalás elutasítása (Reject)
Tesztkörnyezet	Windows 11, Chrome 122, 1920×1080
Előfeltétel	Admin be van jelentkezve. Létezik legalább egy "pending" foglalás.
Lépések	1. Nyissa meg: https://localhost:7206/admin/bookings | 2. Keresse meg az egyik "pending" foglalást. | 3. Kattintson a "Reject" (Elutasítás) piros gombra. | 4. Figyelje meg az UI változást.
Elvárt eredmény	Foglalás státusza "Elutasítva (rejected)"-ra változik. Piros kiemelés.

Mező	Tartalom
Tényleges eredmény	Müköd. az elvárás szerint.
Státusz	Megfelelt
Dátum	2026-01-18

TC-12: Képfeltöltés 5 MB-nál nagyobb fájllal

Mező	Tartalom
TC azonosító	TC-12
Tesztelt funkció	Képfeltöltés 5 MB-nál nagyobb fájllal
Tesztkörnyezet	Windows 11, Chrome 122, 1920×1080
Előfeltétel	Admin be van jelentkezve. Előkészítve egy 6.2 MB-os JPG fájl.
Lépések	1. Nyissa meg: https://localhost:7206/admin/properties | 2. Válassza az "Új ingatlan" vagy szerkesszük a Faház ingatlant. | 3. Kép feltöltő mezőnél válassza ki a 6.2 MB-os fájlt. | 4. Kattintson a "Mentés" gombra.
Elvárt eredmény	Hibaüzenet: "A kép mérete meghaladja az 5 MB-ot (max. 5120 KB). Válasszon kisebb fájlt." A kép nem töltődik fel, az ingatlan adatai változatlanok.
Tényleges eredmény	Validáció müköd., a nagy fájl nem töltődik fel. Helyes hibaüzenet.
Státusz	Megfelelt
Dátum	2026-01-19

TC-13: Esemény-regisztráció teli eseményre

Mező	Tartalom
TC azonosító	TC-13
Tesztelt funkció	Esemény-regisztráció teli eseményre

Mező	Tartalom
Tesztkörnyezet	Windows 11, Chrome 122, 1920×1080
Előfeltétel	Létezik egy esemény, ahol az aktuális regisztrációk száma == MaxParticipants (pl. 5/5).
Lépések	1. Nyissa meg az esemény részletes oldalát. | 2. Ellenőrizze, hogy "Szabad helyek: 0/5" jelzés látható-e. | 3. Ha "Regisztrálok" gomb látható, kattintson rá.
Elvárt eredmény	A "Regisztrálok" gomb szürke és nem kattintható (disabled), VAGY kattintás esetén hibaüzenet: "Az esemény már megtelt." Nincs regisztráció az adatbázisban.
Tényleges eredmény	Gomb disabled, kattintásra hibaüzenet jelenik meg. Kvóta müköd.
Státusz	Megfelelt
Dátum	2026-01-19

TC-14: Mobilos reszponzivitás ellenőrzése

Mező	Tartalom
TC azonosító	TC-14
Tesztelt funkció	Mobilos reszponzivitás ellenőrzése
Tesztkörnyezet	Android 14, Chrome Mobile, Samsung Galaxy A54, 2340×1080
Előfeltétel	Az eszköz és a fejlesztői szerver ugyanazon a Wi-Fi hálózaton van.
Lépések	1. Nyissa meg a főoldalt a fejlesztői szerver IP-jén (pl. http://192.168.1.X:5011). | 2. Ellenőrizze a hamburger (☰) menü megjelenését a jobb felső sarokban. | 3. Koppintson a hamburger ikonra – nyílik le a menü? | 4. Koppintson az "Ingatlanok" menüpontra. | 5. Ellenőrizze a kártyák egyoszlopos elrendezését. | 6. Próbáljon ki egy foglalás leadást.
Elvárt eredmény	Hamburger menü müköd., lenyíló navigáció megjelenik. Kártyák egyoszloposan rendeztek. A foglalási ûrlap minden mezője olvasható és kitölthető. Nincs vízszintes görgetés.

Mező	Tartalom
Tényleges eredmény	Reszponzív megjelenés helyes. Hamburger menü müköd. Kártyák egy oszlopban. Enyhe vízszintes görgetés a táblázatnézeten.
Státusz	Részben megfelelt
Dátum	2026-01-20

2.8.5. Nem funkcionális tesztek

A nem funkcionális tesztek a rendszer teljesítményét, biztonságát és általános minőségét mérik olyan szempontok alapján, amelyek nem köthetők egy konkrét funkcióhoz, de mégis kritikusak a felhasználói élmény és a rendszer megbízhatósága szempontjából.

Betöltési idő tesztelés: A Chrome DevTools Network tab-jával mértük a főoldal betöltési idejét (DOMContentLoaded). Lokálisan futó szerveren a főoldal 1.2 másodperc, a foglalás oldal 1.5 másodperc alatt teljesen betölt, beleértve az összes Bootstrap CDN és statikus asset betöltési idejét. Ezek az értékek jóval az általánosan elfogadott 3 másodperces küszöb alatt maradnak.

SQL injection tesztelés: Az összes szövegbeviteli mezőbe SQL kódot injektáltunk: ' OR '1'='1'-- , ' DROP TABLE Users;-- , ' UNION SELECT * FROM Users WHERE 'a'='a. Az Entity Framework Core paraméteres lekérdezései minden esetben blokkolták az injektált kódot: a szöveg literális értékként kerül az adatbázisba, nem SQL utasításként. Ez az EF Core egyik legfontosabb biztonsági funkciója.

Reszponzivitás különböző felbontásokon: Chrome DevTools Device Emulation funkciójával 320px, 576px, 768px, 1024px és 1920px szélességen teszteltük az alkalmazást. A 320px-es felbontáson a táblázatok vízszintesen görgethetők (table-responsive Bootstrap osztály), ami elfogadható viselkedés. Az összes többi felbontáson az elrendezés az elvárásoknak megfelelően skálázódik.

Teszt neve	Módszer	Mérőszám	Elvárt	Eredmény	Értékelés
Főoldal betöltés	Chrome DevTools Network	DOMContentLoaded	< 2.0 s	1.2 s	Megfelelt
Foglalás oldal betöltés	Chrome DevTools Network	DOMContentLoaded	< 2.0 s	1.5 s	Megfelelt
API válaszidő (lista)	Postman timer	Response time	< 500 ms	< 150 ms	Megfelelt
API válaszidő (foglalás)	Postman timer	Response time	< 1000 ms	< 300 ms	Megfelelt
Reszponzivitás 320px	Chrome DevTools	Nincs törés	Görgetés nélkül olvasható	Tábl. vízszintes scroll	Részben megfelelt
Reszponzivitás 768px	Chrome DevTools	Elrendezés	2 oszlopos kártya	2 oszlopos	Megfelelt
Reszponzivitás 1920px	Chrome DevTools	Elrendezés	3 oszlopos kártya	3 oszlopos	Megfelelt
SQL injection	Manuális (Postman)	Blokkolás	EF Core blokk	Blokkolva	Megfelelt
HTTPS tanúsítvány	Chrome	SSL warning	Dev cert warning	Dev cert figyelmeztetés	Várt viselkedés

2.9. Fejlesztési lehetőségek

2.9.1. E-mail értesítési rendszer

Az e-mail értesítési rendszer hiánya a jelenlegi implementáció egyik legsürgetőbb hiányossága felhasználói élmény szempontjából. A jelenlegi rendszerben a felhasználónak aktívan be kell lépnie az oldalra, és a Profil oldalon manuálisan kell ellenőriznie foglalásai státuszát. Egy automatikus e-mail értesítési rendszerrel az alábbi események váltanának ki automatikus e-mail küldést: (1) Foglalás leadásakor: visszaigazoló e-mail a foglalónak a foglalás adataival és a

"Függőben" státusszal; (2) Foglalás jóváhagyásakor: örömhír e-mail az approve esemény időpontjával; (3) Foglalás elutasításakor: elutasítási e-mail javaslattal az alternatív időpontokra; (4) Esemény-regisztrációkor: visszaigazoló e-mail az esemény adataival; (5) Esemény előtt 24 órával: emlékeztető e-mail.

Technológiai megvalósítás: Az ASP.NET Core-ban az e-mail küldés az IEmailSender interfészen keresztül integrálható. SMTP-alapú küldéshez a MailKit NuGet csomag ajánlott (SmtpClient osztály), amellyel Gmail SMTP szerveren (smtp.gmail.com:587, STARTTLS) keresztül lehet levelet küldeni. A Gmail SMTP ingyenesen használható "App Password" hitelesítéssel (2FA szükséges). Professzionális megoldáshoz a SendGrid API-ja 100 email/nap ingyenes limittel rendelkezik, amelybe a Nagypapa Házai várható forgalma belefér. A Razor Templates könyvtárral az e-mailek HTML sablonokból generálhatók, amelyekben a foglalási adatok dinamikusan helyettesíthetők be.

2.9.2. Mobilalkalmazás (.NET MAUI)

A .NET Multi-platform App UI (MAUI) a Microsoft legújabb cross-platform natív alkalmazásfejlesztési keretrendszere, amely egyetlen C# kódbázisból Android, iOS, Windows és macOS alkalmazásokat képes generálni. A MAUI a Xamarin.Forms utódjának tekinthető, de jelentős teljesítmény- és fejlesztői élménybeli javításokkal. A MAUI Blazor Hybrid funkcióján keresztül a meglévő Blazor komponensek közel módosítás nélkül újrahasználhatók egy natív mobilalkalmazásban.

A mobil alkalmazás konkrét előnyei a webes verziónál: Push értesítések (FCM/APNS) – valódi natív push értesítések, amelyek akkor is megjelennek, ha az alkalmazás nem futtatható. Kamera integráció – az ingatlan képfeltöltése közvetlenül a telefon kamerájával elvégezhető. Offline hozzáférés – a legutóbb betöltött ingatlan és foglalási adatok offline is megtekinthetők (SQLite local cache). GPS integráció – az ingatlan helyszíne megnyitható a telefon térképalkalmazásában. A kód újrahasználhatóság becslése: az ApiService.cs, az adatmodellek és az üzleti logika ~80%-a változtatás nélkül átvihető a MAUI projektbe.

2.9.3. Fizetési rendszer integráció

Bár a Nagypapa Házai platform alapkoncepciója a csereajánlat-alapú ingyenes használat, a jövőben hibrid megközelítés is elképzelhető: bizonyos prémium

ingatlanokhoz vagy szezonális periódusokhoz meghatározott előleg fizetése lenne lehetséges. Ez növelné az adminisztrátori bizalmat és csökkentené a nem teljesített foglalások számát.

A Stripe fizetési szolgáltatás .NET SDK-ja (Stripe.net NuGet csomag) lehetővé teszi kártyás fizetés integrálását. A Stripe Elements iframe alapú kártya-beviteli mező biztonságos, PCI-DSS Level 1 kompatibilis megoldás, amelynél a kártyaadatok soha nem érintik a saját szerveredet – csak a Stripe-é. A Barion fizetési kapu a magyarországi felhasználók számára egy hazai alternatíva, amely szintén rendelkezik .NET SDK-val. Az online előleg (pl. a foglalásnál kötelező 30% befizetése) integrálható a foglalási folyamatba: a Blazor BookingNew.razor oldalon a "Foglalás leadása" gomb megnyomásakor először a fizetési ablak jelenne meg, majd sikeres fizetés esetén az API-ra kerülne a foglalási kérelem.

2.9.4. Azure cloud deployment

A jelenlegi implementáció localhost fejlesztői szerveren fut, ami azt jelenti, hogy nyilvánosan csak VPN-en vagy helyi hálózaton érhető el. Az éles, nyilvánosan hozzáférhető deployment az Azure felhőszolgáltatásokon keresztül valósítható meg. Az Azure App Service egy teljes értékű web hosting platform, amely ASP.NET Core és Blazor Server alkalmazásokat natívan támogat, méret szerinti árazással rendelkezik (ingyenes F1 tier elegendő kis forgalomnál), és egyszerű GitHub Actions alapú CI/CD pipeline-nal üzemeltetható.

A deployment folyamat lépései: (1) Az API projekt és a Blazor projekt Azure App Service-be publish, GitHub Actions workflow-val; (2) Az SQL Server Express lecserélése Azure SQL Database-re (Basic tier: ~5 USD/hó); (3) HTTPS automatikusan biztosított Azure App Service-en Let's Encrypt tanúsítvánnyal; (4) Custom domain beállítása (pl. nagypapahazai.hu); (5) Azure Application Insights monitoring integráció a produkciós hibák nyomon követéséhez. A GitHub Actions YAML workflow konfigurációja:

```
# .github/workflows/azure-deploy.yml
name: Deploy to Azure App Service
on:
  push:
    branches: [ main ]
jobs:
  deploy:
```

```
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "8.0.x"
      - run: dotnet publish -c Release -o ./publish
      - uses: azure/webapps-deploy@v3
        with:
          app-name: "nagypapahazai-api"
          publish-profile: ${{ secrets.AZURE_PUBLISH_PROFILE }}
          package: ./publish
```

2.9.5. Keresési és szűrési funkciók bővítése

A jelenlegi rendszerben az ingatlanok egyetlen listában jelennek meg szűrési lehetőség nélkül. A bővítési terv a következő szűrési dimenziókat tartalmazza: (1) Helyszín szerinti szűrő: megye, régió vagy szabad szöveges helységnév keresés; (2) Árkategória szűrő: csúszka vagy előre meghatározott kategóriák (0–15 000, 15 001–30 000, 30 001+ Ft/éj); (3) Kapacitás szűrő: minimum személyszám megadása; (4) Elérhetőség szűrő: adott dátumokra szabad ingatlanok – ez a legkomplexebb szűrő, mivel a foglalások intervallum-átfedés vizsgálatát igényli; (5) Ingatlan típus szűrő: faház, nyaraló, vendégház, tanya. A szűrési paraméterek URL query paraméterként kerülnek be (/ingatlanok?megye=veszprem&min_ar=10000&max_ar=30000), ami lehetővé teszi a szűrési feltételek megoszthatóságát és a böngészőben visszanavigálhatóságát.

2.9.6. Értékelési és visszajelzési rendszer

Egy 1-5 csillagos értékelési rendszer és szöveges visszajelzési lehetőség jelentősen növelné a platform megbízhatóságát és a közösség épülését. Az értékelések csak befejezett (lezárt záró dátummal rendelkező) és jóváhagyott foglalásokhoz lennének beküldhetők, megakadályozva a valódi tartózkodás nélküli visszajelzéseket. Az adminisztrátori moderálás lehetővé tenné a nem megfelelő visszajelzések eltávolítását. A Reviews adatbázistábla új mezőkkel egészítené ki a rendszert: PropertyId (FK), UserId (FK), Rating (int 1-5), Comment (nvarchar 1000), CreatedAt, IsApproved (bool). Az ingatlan kártyákon az átlagos csillagszám is megjelenne (pl. ★★★★☆ 4.2/5, 7 értékelés alapján).

3. Felhasználói dokumentáció

A felhasználói dokumentáció célja, hogy minden érintett fél – legyen az egyszerű látogató, regisztrált felhasználó vagy rendszergazda – részletes, lépésről lépésre haladó útmutatót kapjon a Nagypapa Házai platform minden funkciójának használatához. Ez a fejezet nem csupán az egyes képernyők leírásával foglalkozik, hanem kontextusba helyezi az egyes funkciókat, bemutatja a mögöttes logikát és a lehetséges hibaüzeneteket, megoldásaikkal együtt. A dokumentáció célja az is, hogy a jövőbeli fejlesztők, oktatók és érdeklődők egységes, átfogó képet kapjanak arról, milyen módon működik a rendszer a végfelhasználó perspektívájából.

A fejezet öt fő részre tagolódik: rendszerkövetelmények és telepítési útmutató, a nyilvános (nem bejelentkezett) felhasználói felület leírása, a regisztrált felhasználói funkciók részletes ismertetése, az adminisztrátori felület teljes dokumentációja, valamint a hibaelhárítási útmutató. Minden alfejezethez képernyőkép-jelzőket helyeztünk el, amelyek megmutatják, hol kellene a tényleges képernyőfelvételeknek szerepelniük a végleges dokumentációban. A tartalom megértéséhez nem szükséges programozói ismeret, azonban az adminisztrátori fejezetek esetében előnyös az alapvető informatikai jártasság.

Fontos megjegyezni, hogy a Nagypapa Házai platform jelenleg fejlesztői (localhost) környezetben fut, tehát a telepítési és elérési utasítások ennek megfelelően kerültek kidolgozásra. A fejezet 3.9. alszakasza részletesen foglalkozik a jövőbeli felhő-alapú telepítési stratégiával, de a jelenlegi felhasználói dokumentáció a helyi fejlesztői környezetre fókuszál. Mindenki, aki a platformot először nyitja meg, találkozni fog a bejelentkezési oldallal, a főoldallal és a nyilvánosan elérhető ingatlanlista-oldallal – ezeket az első kontaktpontokat a fejezet 3.2. alszakasza írja le részletesen.

3.1. Rendszerkövetelmények és telepítési útmutató

A Nagypapa Házai platform futtatásához mind a szerver oldali, mind a kliens oldali környezetnek meg kell felelnie bizonyos minimális követelményeknek. Mivel a rendszer ASP.NET Core Blazor Server architektúrán alapul, a feldolgozás döntő többsége a szerveren történik; a kliens csupán egy modern webböngészőre szorul, amelyen a SignalR WebSocket-kapcsolat keresztül az összes UI-esemény valós

időben szinkronizálódik a szerverrel. Az alábbiakban először a szerver oldali, majd a kliens oldali követelményeket mutatjuk be, végül a teljes telepítési folyamatot ismertetjük.

3.1.1. Szerver oldali rendszerkövetelmények

A szerver (fejlesztői gép) oldalon az alábbi szoftverek jelenléte szükséges a platform futtatásához. A .NET 8 SDK (vagy Runtime) elengedhetetlen, mivel az alkalmazás ezen a keretrendszeren alapul. A Visual Studio 2022 (17.8 vagy újabb) vagy a Visual Studio Code megfelelő C# kiterjesztéssel biztosítja a fejlesztői és futtatási környezetet. Az SQL Server Express (SQLEXPRESS) adatbázis-motor szükséges az adatok tárolásához – a fejlesztői csapat az ingyenes Express kiadást használja, amely kisebb projektekhez teljesen megfelelő. A Git verziókezelő rendszer szükséges a forráskód kezeléséhez, amennyiben a kód repository-ból kerül letöltésre.

Minimális hardveres követelmények fejlesztői gép esetén: legalább 8 GB RAM (ajánlott 16 GB, különösen ha Visual Studio 2022 és SQL Server Management Studio párhuzamosan fut), kétmagos processzor 2.0 GHz-es órajellel (ajánlott négymagos), legalább 10 GB szabad tárhely a keretrendszer, az adatbázis és a projekt számára. Az operációs rendszer tekintetében Windows 10/11, macOS 12 Monterey (vagy újabb), illetve Linux (Ubuntu 20.04 LTS vagy újabb) egyaránt támogatott, bár a csapat Windows környezetben fejlesztett.

Szoftver	Minimális verzió	Ajánlott verzió	Megjegyzés
.NET SDK	8.0.0	8.0.x (legfrissebb)	LTS (Long-Term Support) kiadás
Visual Studio	2022 v17.8	2022 v17.10+	Community kiadás elegendő
SQL Server Express	2019	2022	SQLEXPRESS instance name szükséges
SQL Server Mgmt Studio	19.0	20.x	Opcionális, de hasznos

Szoftver	Minimális verzió	Ajánlott verzió	Megjegyzés
Git	2.40	2.44+	Repository klónozáshoz
Node.js	18 LTS	20 LTS	Csak fejlesztői script-ekhez

3.1.2. Kliens oldali rendszerkövetelmények

A Nagypapa Házai platformot böngészőből érik el a felhasználók. Mivel a Blazor Server SignalR WebSocket-kapcsolatot használ, a böngészőnek támogatnia kell a WebSocket technológiát – ezt ma már minden modern böngésző teljesíti. Az alábbi böngészők kerültek tesztelésre és igazoltan kompatibilisek a platformmal: Google Chrome (90. verzió felett), Mozilla Firefox (88. verzió felett), Microsoft Edge (90. verzió felett) és Apple Safari (14. verzió felett).

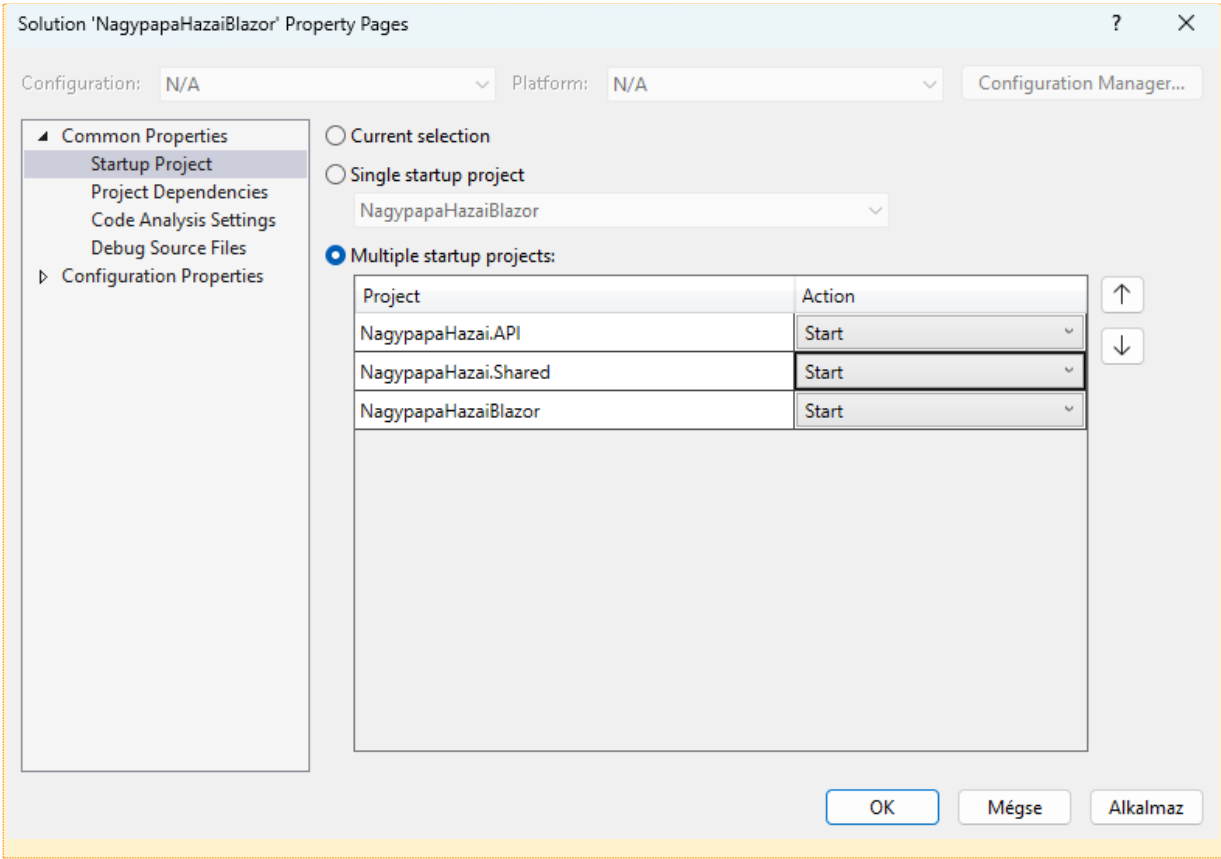
Kliens oldali hardverkövetelményekről külön nem szükséges szót ejteni, mivel a feldolgozás a szerveren zajlik, és a böngésző csupán HTML/CSS renderelést végez minimális JavaScript futtatás mellett (a SignalR kliens library-n kívül). Praktikusan bármely 2018 utáni számítógép, tablet vagy okostelefon képes futtatni a platformot. Fontos megjegyezni azonban, hogy a jelenlegi fejlesztői verzió csak localhost-on érhető el, tehát a kliens és a szerver azonos gépen kell, hogy fusson (vagy ugyanazon a helyi hálózaton, megfelelő tűzfalbeállításokkal).

3.1.3. A projekt telepítése és elindítása

Az alábbiakban lépésről lépésre bemutatjuk, hogyan lehet a Nagypapa Házai projektet egy új fejlesztői gépen üzembe helyezni. A folyamat összesen hat fő lépésből áll, és körülbelül 20-30 percet vesz igénybe, feltéve, hogy az internet-kapcsolat megfelelő sebességű az npm és NuGet csomagok letöltéséhez.

1. 1. lépés – Előfeltételek telepítése: Töltse le és telepítse a .NET 8 SDK-t a https://dotnet.microsoft.com/download oldalról. Ezt követően telepítse az SQL Server Express-t a https://www.microsoft.com/en-us/sql-server/sql-server-downloads oldalról. Az SQLEXPRESS nevű instance-t hagyja az alapértelmezett beállításokon.

2. 2. lépés – Repository klónozása: Nyisson egy terminált vagy parancssort, és futtassa: git clone https://github.com/[projekt-url]/NagypapaHazai.git. Majd lépjen be a projektmappába: cd NagypapaHazai.
3. 3. lépés – Kapcsolati karakterlánc beállítása: Nyissa meg az appsettings.json fájlt a NagypapaHazaiBlazor projektből, és ellenőrizze, hogy a ConnectionStrings.DefaultConnection mezőben a Server értéke .\SQLEXPRESS (vagy localhost\SQLEXPRESS). Ha eltérő instance nevet használ, módosítsa ennek megfelelően.
4. 4. lépés – Adatbázis migrációk futtatása: A Package Manager Console-ban (Visual Studio) vagy a terminálban futtassa: dotnet ef database update --project NagypapaHazai.API --startup-project NagypapaHazai.API. Ez létrehozza az adatbázist és alkalmazja az összes migrációt, beleértve a seed adatokat (admin felhasználó és a három alapingatlan).
5. 5. lépés – Alkalmazás indítása: Visual Studio-ban nyissa meg a NagypapaHazai.sln megoldásfájlt. Kattintson a jobb gombbal a Solution-re, válassza a "Set Startup Projects" lehetőséget, és állítsa be, hogy mindkét projekt (NagypapaHazaiBlazor és NagypapaHazai.API) egyszerre induljon (Multiple startup projects). Nyomja meg az F5 billentyűt.
6. 6. lépés – Az alkalmazás megnyitása: A Blazor alkalmazás automatikusan megnyílik a böngészőben a https://localhost:7206 vagy http://localhost:5011 URL-en. Az API Swagger dokumentációja a https://localhost:44369/swagger URL-en érhető el.



Amennyiben a telepítés során hibaüzeneteket tapasztal, az alábbi leggyakoribb problémákat és megoldásaikat tartalmazza a dokumentáció 3.10. hibaelhárítási fejezete. A leggyakoribb gondok közé tartozik, hogy az SQLEXPRESS instance nem érhető el (ebben az esetben ellenőrizze, hogy az SQL Server Express szolgáltatás fut-e a Windows Szolgáltatások kezelőjében), vagy hogy az HTTPS tanúsítvány nem megbízható (ezt a dotnet dev-certs https --trust paranccsal lehet orvosolni).

3.1.4. Seed adatok és alapértelmezett hozzáférések

Az adatbázis inicializálásakor a rendszer automatikusan feltölti az alap adatokat, hogy a platform azonnal kipróbálható legyen. Ezek az úgynevezett seed adatok tartalmazzák az adminisztrátori és a tesztelői felhasználói fiókot, valamint a három alapingatlant. Az alábbiakban táblázatosan foglaljuk össze ezeket az adatokat.

Típus	Felhasználónév / Email	Jelszó	Megjegyzés
Adminisztrátor	admin@nagypapahazai.hu	Admin123!	Teljes hozzáférés minden funkcióhoz

Típus	Felhasználónév / Email	Jelszó	Megjegyzés
Teszt felhasználó	teszt@nagypapahazai.hu	Teszt123!	Korlátozott, normál felhasználói hozzáférés
Kapcsolati email	nagypapahazai@gmail.com	–	Projekt-kapcsolattartási cím

A seed ingatlanok a következők: Faház a Bakonyban (Bakony, max. 10 fő, 25 000 Ft/éjszaka), Vízparti Nyaraló (Balatonlelle, max. 6 fő, 35 000 Ft/éjszaka) és Villányi Vendégház (Villány, max. 4 fő, 20 000 Ft/éjszaka). Ezek az ingatlanok automatikusan megjelennek az ingatlankereső oldalon, és segítik a rendszer funkcionalitásának tesztelését. A fejlesztőcsapat szándékosan három különböző ár- és kapacitásszintet választott, hogy a szűrési és rendezési funkciók tesztelése értelmes legyen.

3.2. A nyilvános felhasználói felület

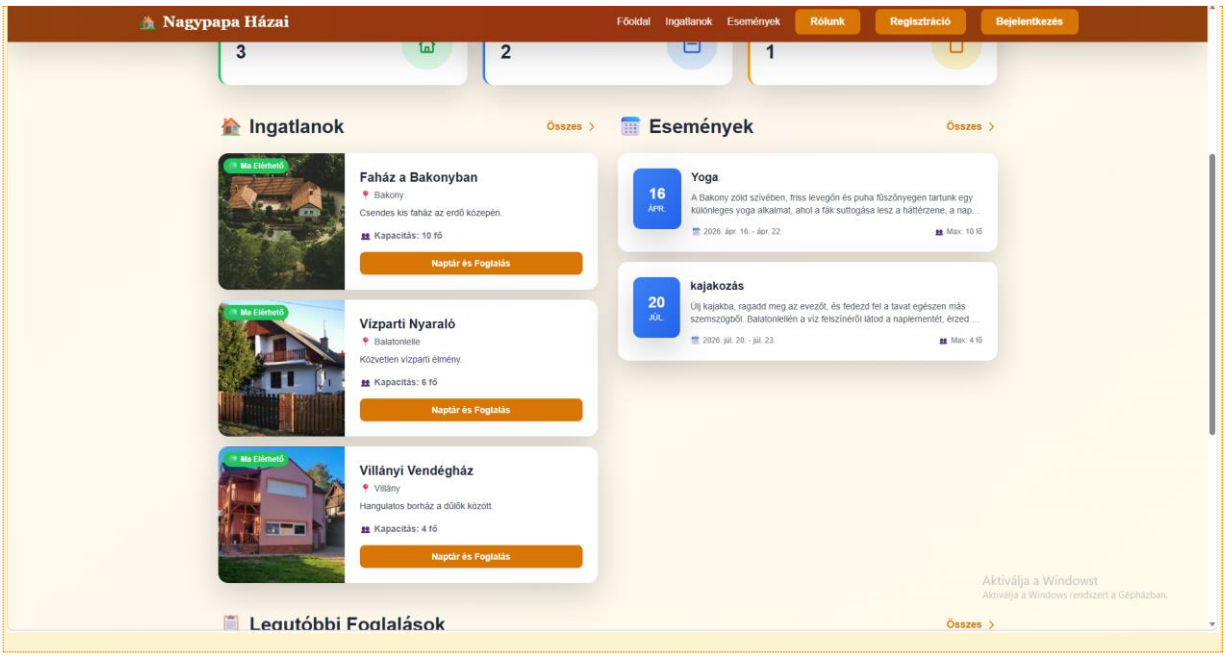
A Nagypapa Házai platform nyilvánosan elérhető oldalaira bármely látogató eljuthat bejelentkezés nélkül. A nyilvános felület minimális, de informatív: tartalmazza a főoldalt, az ingatlanok listáját és az egyes ingatlanok részletes leírását. A platformra látogató felhasználókat a rendszer ezeken az oldalakon tájékoztatja a lehetőségekről, és egyértelműen felkínálja a regisztrációt vagy bejelentkezést a bővebb funkciók eléréséhez. A nyilvánosan elérhető oldalak szándékosan egyszerűek és gyorsan betölthetők, hogy az első benyomás pozitív legyen.

3.2.1. Főoldal (/)

A főoldal (/ útvonal) a platform belépési pontja. A navigációs sávban a platform neve és logója szerepel a bal oldalon, jobbra pedig a navigációs menüpontok jelennek meg: „Ingatlanok", „Foglalások" (bejelentkezés után aktív), „Események" és a bejelentkezés/regisztráció gomb. A Bootstrap 5-ös Navbar komponens reszponzív: kisebb képernyőkön hamburger menübe összecsukódik.

A főoldalon a platform rövid bemutatkozása szerepel egy hero szekcióban, amely elmagyarázza, mi a Nagypapa Házai lényege: közösségi ingatlankölcsönzési

platform, ahol a vidéki és üdülő ingatlanok tulajdonosai megoszthatják ingatlanjaikat a közösség tagjaival – pénzért, csereajánlatért vagy egyéb megállapodás alapján. A hero szekció alatt egy kiemelt ingatlanok kártya-rács jelenik meg, amely három „ajánlott" ingatlant mutat be. Ezek a kártyák kattinthatók, és az adott ingatlan részletes oldalára navigálnak.



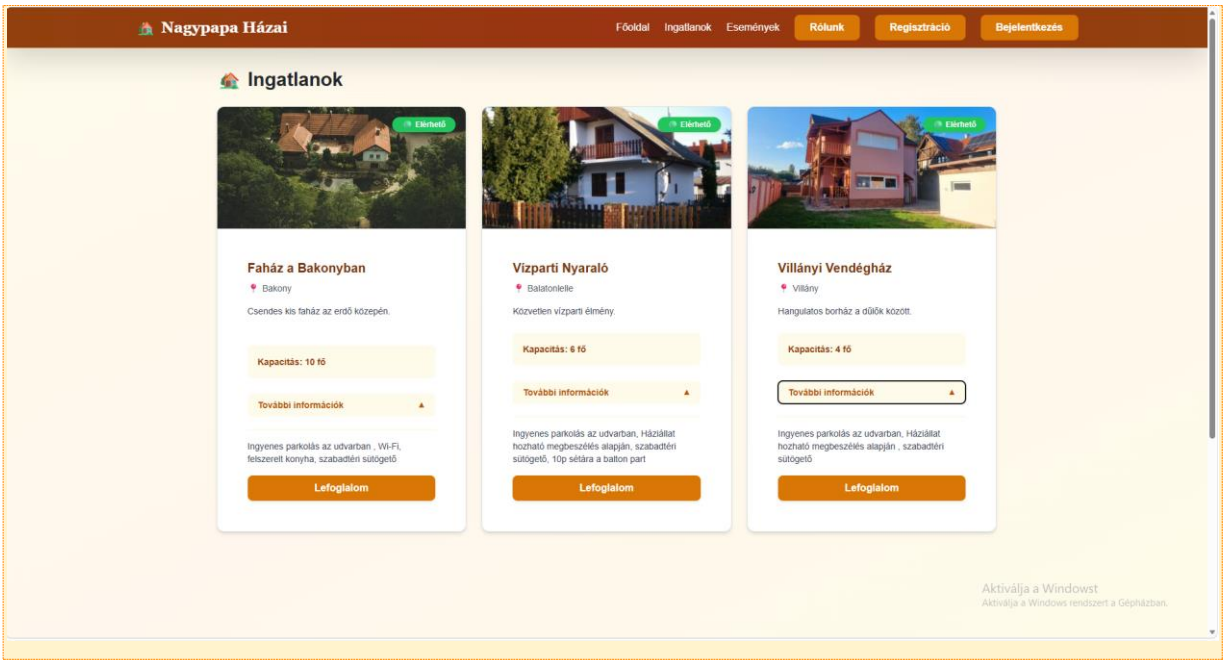
A főoldal alsó részén egy rövid „Hogyan működik?" szekció található, amely három ikonos kártyán mutatja be a regisztrációtól a foglalásig vezető folyamatot. Ezek az ikonos kártyák nem interaktívak, csupán tájékoztató jellegűek. A főoldal legalsó részén egy CTA (Call-to-Action) gomb hívja a látogatókat regisztrációra. A főoldal teljesen statikus – nem kér adatbázis-hívást a szerver oldalon, így rendkívül gyorsan töltődik be.

3.2.2. Ingatlanok listázó oldala

Az ingatlanok listázó oldala a /properties URL-en érhető el (a főmenüből az „Ingatlanok" menüpontra kattintva). Ez az oldal listázza az összes, az adatbázisban szereplő aktív ingatlant. Az ingatlanokat Bootstrap Card komponensekben jeleníti meg a rendszer, soronként kettesével, ami reszponzív elrendezésben egyetlen oszlopra csukódik össze mobilon.

Minden ingatlan-kártyán a következő információk szerepelnek: az ingatlan neve (félkövér, nagyobb betűmérettel), az elhelyezkedés (helység neve), a maximális vendégszám, az éjszakánkénti ár forintban és egy rövid, maximum 150 karakteres

leírás-kivonat. Ha a leírás ennél hosszabb, akkor „..." jelöléssel csonkítja a rendszer. Az ingatlan-kártyák alján egy „Részletek" feliratú gomb navigál az ingatlan saját részletes oldalára.



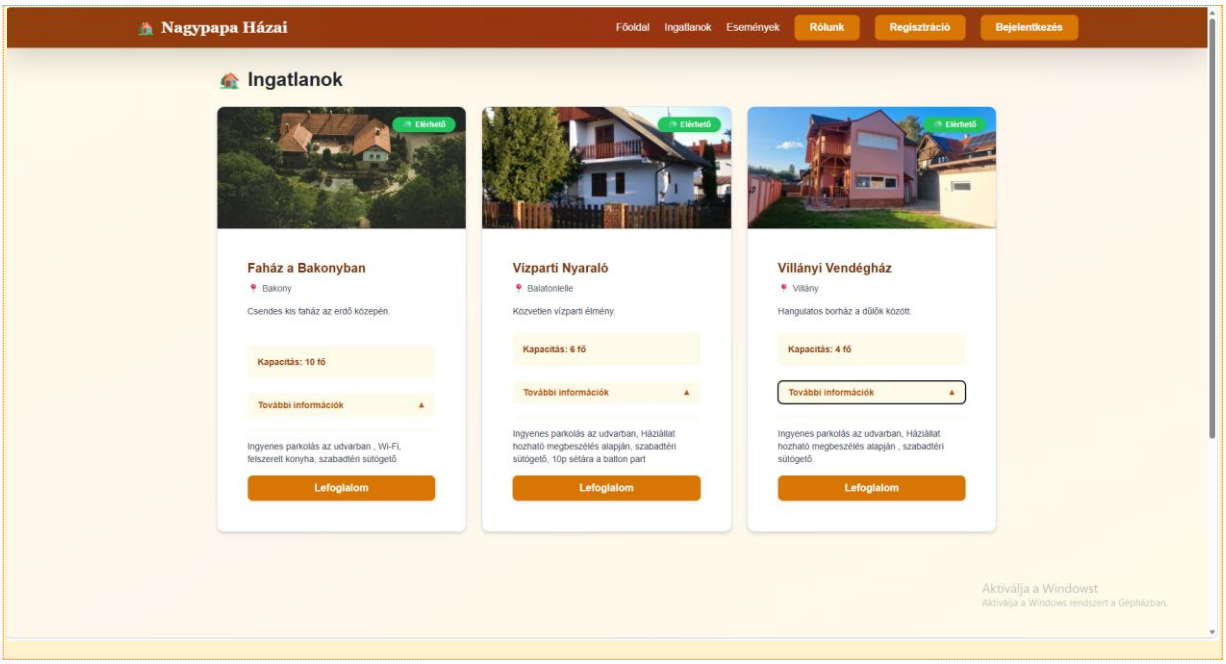
Az ingatlanok listázó oldalán a felső részben szűrési lehetőségek állnak rendelkezésre: szabad szöveges keresés a név és helyszín alapján, valamint szűrés a maximális férőhely szerint. A szűrők nem igényelnek oldal-újratöltést – Blazor Server oldalon az adatok dinamikusan frissülnek a SignalR-kapcsolaton keresztül. Ez az azonnali szűrés az egyik leglátványosabb felhasználói élmény-javítás, amelyet a Blazor technológia lehetővé tesz.

3.2.3. Ingatlan részletező oldala

Az egyes ingatlanok részletes oldala a /properties/{id} URL-mintát követi, ahol az {id} az adatbázisban tárolt egyedi azonosítója az ingatlannak. Erre az oldalra az ingatlan-kártyán lévő „Részletek" gombbal vagy közvetlenül a URL megadásával lehet eljutni. Az oldal tartalmazza az ingatlan teljes leírását, a férőhely- és áradatokat, az elhelyezkedést (amennyiben térkép integráció rendelkezésre áll, annak jelzésével), és egy foglalást indító gombot.

A foglalást indító gomb bejelentkezési állapot-érzékeny: amennyiben a látogató nincs bejelentkezve, a gomb megnyomásakor a rendszer a bejelentkezési oldalra irányít. Bejelentkezett felhasználók esetén megnyílik egy modális ablak a foglalási részletek (dátumok, vendégszám, különleges kérések) megadásához. Az ingatlan

részletes oldalán megtekinthető az ingatlanhoz eddig feltöltött vélemények/értékelések szekciója is – ez azonban a jelenlegi verzióban nem tartalmaz valódi adatokat, csupán placeholder hellyel van jelezve.



3.3. Regisztráció és bejelentkezés

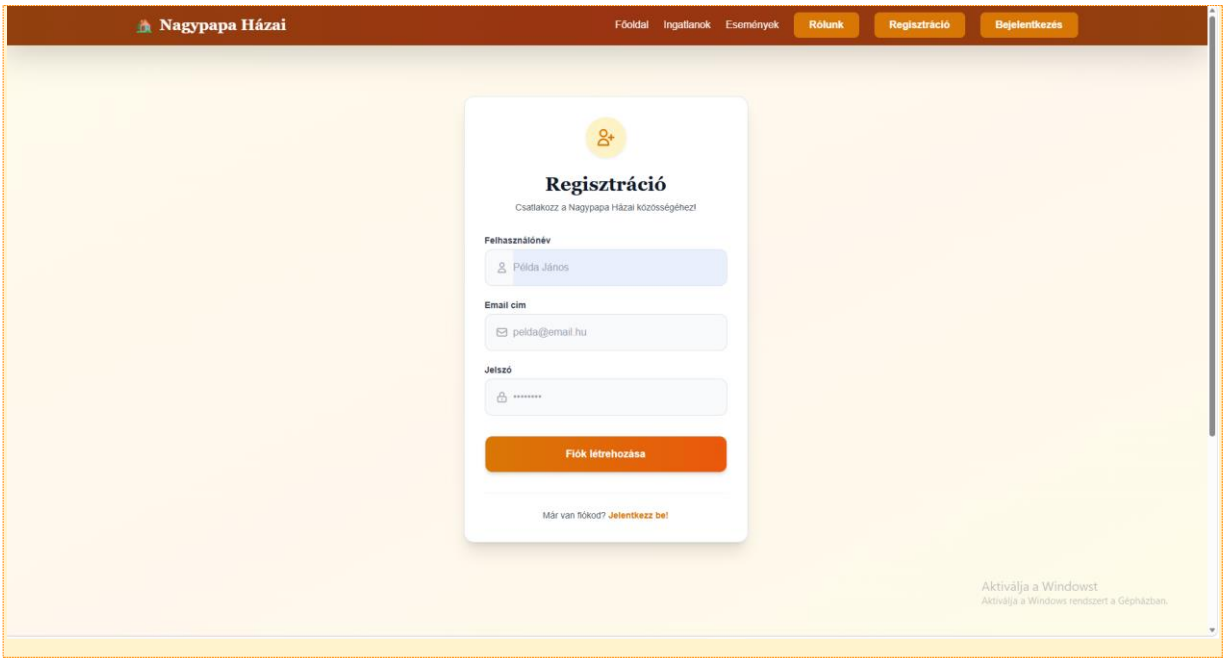
A Nagypapa Házai platform teljes körű funkcióinak eléréséhez regisztrált és bejelentkezett felhasználói fiók szükséges. A regisztrációs és bejelentkezési folyamat egyszerű, néhány lépéses, és a platform saját, belső hitelesítési rendszerét használja – külső OAuth-integráció (Google, Facebook stb.) a jelenlegi verzióban nem érhető el. A jelszavak BCrypt algoritmust használó hash-eléssel tárolódnak az adatbázisban, ami megfelel a mai biztonsági sztenderdeknek.

3.3.1. Regisztráció folyamata

A regisztrációs oldal a /register URL-en érhető el, és a navigációs sávban lévő „Regisztráció" gombra kattintva vagy a bejelentkezési oldalon a „Még nincs fiókod? Regisztrálj!" linkre kattintva érhető el. A regisztrációs form a következő mezőket tartalmazza: Teljes név (kötelező, minimum 3 karakter), Email cím (kötelező, érvényes email formátum), Jelszó (kötelező, legalább 8 karakter, nagybetű, kisbetű és szám szükséges) és Jelszó megerősítése (meg kell egyeznie a jelszó mezővel).

A rendszer valós idejű validációt végez a form mezőin, ami azt jelenti, hogy amint a felhasználó elhagyja az egyes mezőket (blur esemény), a Blazor komponens

azonnal ellenőrzi az adatokat és hibaüzeneteket jelenít meg, ha szükséges. Ez a megközelítés jobb felhasználói élményt biztosít, mint az egyetlen küldés-gombra érkező hibalista, mivel a felhasználó azonnal látja, hol rontott el valamit.



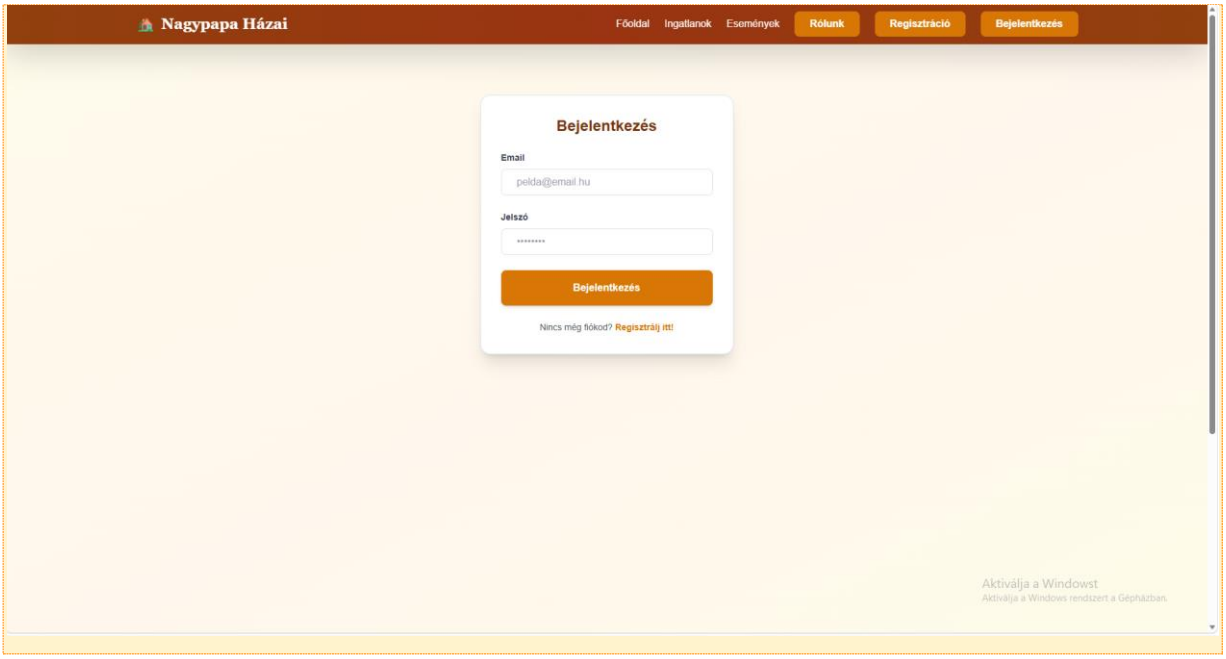
A regisztráció elküldésekor a rendszer először ellenőrzi, hogy az adott email-cím nem szerepel-e már az adatbázisban. Ha igen, hibaüzenet jelenik meg: „Ez az email-cím már regisztrálva van. Kérjük, használj másik email-t, vagy jelentkezz be a meglévő fiókodba." Ha az email szabad, az adatok mentésre kerülnek, és a felhasználó automatikusan bejelentkezési oldalra kerül, ahol egy sikeres regisztrációs üzenet fogadja.

3.3.2. Bejelentkezési folyamat

A bejelentkezési oldal a /login URL-en érhető el. A form két mezőt tartalmaz: Email cím és Jelszó. A „Bejelentkezés" gombra kattintva a rendszer ellenőrzi az email és jelszó kombinációt az adatbázissal szemben. Ha a hitelesítés sikeres, a felhasználó session-be kerül (ASP.NET Core Session alapján), és a rendszer a főoldalra irányítja. Ha a hitelesítés sikertelen, az üzenet nem árulja el, hogy az email vagy a jelszó hibás-e (biztonsági okokból), csupán annyit jelenít meg: „Hibás email-cím vagy jelszó."

A bejelentkezési oldal tartalmaz egy „Elfelejtett jelszó?" linket, amely a jelenlegi verzióban csupán egy informatív üzenetre navigál arról, hogy a jelszóvisszaállítási funkció fejlesztés alatt áll. Az e-mail alapú jelszóvisszaállítás a 2.9. fejezetben

dokumentált jövőbeli fejlesztések közé tartozik. Emellett a bejelentkezési oldalon egy „Még nincs fiókod? Regisztrálj!" link is szerepel, amely a regisztrációs oldalra vezet.



3.3.3. Kijelentkezés folyamata

A kijelentkezési lehetőség a navigációs sávban, a bejelentkezett felhasználó nevére kattintva megjelenő legördülő menüből érhető el. A „Kijelentkezés" menüpontra kattintva a rendszer törli a session adatokat és a főoldalra irányítja a felhasználót. A művelet azonnali és visszafordíthatatlan – nincs „Biztosan ki akar lépni?" megerősítési ablak, mivel a kijelentkezés alacsony kockázatú művelet, amely nem okoz adatvesztést.

Fontos viselkedés: ha a felhasználó bejelentkezés nélkül próbál elérni egy hitelesítést igénylő oldalt (például a /bookings/new oldalt), a rendszer automatikusan a bejelentkezési oldalra irányítja, és a sikeres bejelentkezés után visszanavigál az eredetileg kért oldalra. Ez az ún. „redirect after login" viselkedés a standard ASP.NET Core hitelesítési middleware segítségével valósul meg.

3.4. Felhasználói profil kezelése

A bejelentkezett felhasználók hozzáférhetnek saját profiljukhoz a /profile URL-en, amely a navigációs sávban a felhasználó nevére kattintva megjelenő legördülő menüből is elérhető. A profiloldal lehetővé teszi a személyes adatok megtekintését és

módosítását, a jelszó megváltoztatását, valamint a saját foglalási és esemény-részvételi előzmények megtekintését.

3.4.1. Foglalási előzmények a profiloldalon

A profiloldal alsó felén megjelennek a felhasználóhoz kötött foglalások összefoglalói. Minden egyes foglalás egy sorban szerepel, amelyen látható az ingatlan neve, a foglalás időszaka (érkezési és távozási dátum), a vendégszám, az összköltség és a foglalás aktuális státusza (Függőben, Elfogadva, Elutasítva, Lemondva). A „Részletek" linkre kattintva az adott foglalás teljes részletes nézete nyílik meg.

A profiloldalon elérhető foglalási előzményeknél a felhasználó lehetőséget kap a még „Függőben" lévő foglalások lemondására is, anélkül, hogy az admin felületre kellene navigálnia. A lemondási gombra kattintva egy megerősítési ablak jelenik meg, és pozitív megerősítés esetén a foglalás státusza „Lemondva" értékre változik. Ez a lehetőség csak a még el nem kezdett (jövőbeli) foglalásokra vonatkozik.

3.5. A foglalási folyamat részletes leírása

A foglalási folyamat a Nagypapa Házai platform szívét jelenti: ez az a funkció, amellyel a felhasználók ténylegesen igénybe vehetik az ingatlanokat. A foglalás több lépéses folyamat, amely az ingatlan kiválasztásától a foglalás visszaigazolásáig tart. Az alábbiakban részletesen ismertetjük az összes lépést, az érvényes és érvénytelen forgatókönyvekkel együtt.

3.5.1. Új foglalás indítása

Új foglalás az alábbi módokon indítható: (1) Az ingatlan részletes oldalán a „Foglalás" gombra kattintva, (2) A /bookings/new URL-en közvetlen navigációval, (3) A navigációs menü „Foglalások" → „Új foglalás" opciójával. Mindhárom esetben a /bookings/new oldal nyílik meg, amely tartalmazza a foglalási formot. Ha a felhasználó nincs bejelentkezve, automatikusan a bejelentkezési oldalra kerül.

A foglalási form első szekciójában az ingatlan kiválasztása történik. Egy legördülő listában az összes elérhető ingatlan felsorolásra kerül. Ha a felhasználó az ingatlan részletes oldaláról kattintott a foglalás gombra, az ingatlan automatikusan előre ki van választva. A legördülő lista az ingatlan neve mellett zárójelben feltünteti, ami segít a döntéshozatalban.

3.5.2. Dátumok és vendégszám megadása

A foglalási form második szekciójában az érkezési és távozási dátumot kell megadni. A dátummezők natív HTML date input-ot használnak, amelyhez a böngésző saját dátumválasztó naptárja jelenik meg. Az érkezési dátum minimális értéke a mai nap, a távozási dátum minimális értéke az érkezési dátum utáni nap. Ha a felhasználó érvénytelen dátumot próbál megadni (pl. a távozási dátum korábbi az érkezésinél), a form beküldése megakadályozásra kerül és hibaüzenet jelenik meg.

A vendégszám megadásához egy numerikus input áll rendelkezésre, amelynek maximális értéke automatikusan a kiválasztott ingatlan maximális kapacitása. Ha a felhasználó nagyobb vendégszámot próbál megadni, mint a kapacitás, hibaüzenet jelenik meg: „Az ingatlan maximális kapacitása {X} fő, ennél több vendéget nem lehet foglalni." Ez az ellenőrzés mind kliens, mind szerver oldalon elvégzésre kerül.

3.5.3. Ütközésvizsgálat és elérhetőség

Mielőtt a foglalás ténylegesen bekerül az adatbázisba, a rendszer ütközésvizsgálatot végez: ellenőrzi, hogy a kért időszakban nincs-e már elfogadott foglalás az adott ingatlanra. Az ütközésvizsgálat LINQ lekérdezéssel valósul meg az API rétegben, és a következő feltételt vizsgálja: a meglévő foglalás kezdete kisebb a kért időszak végénél ÉS a meglévő foglalás vége nagyobb a kért időszak kezdeténél.

Ha ütközés kerül feltárásra, a rendszer nem menti a foglalást, hanem hibaüzenetet jelenít meg: „A kiválasztott ingatlan a megadott időszakban már foglalt. Kérjük, válasszon más dátumot, vagy nézze meg a foglalási naptárat." A hibaüzenet mellett megjelenik egy „Naptár megtekintése" gomb, amely a /bookings/calendar oldalra navigál, ahol a felhasználó grafikusan láthatja az elérhetőséget.

3.5.4. Különleges kérések és csereajánlat

A foglalási form tartalmaz egy opcionális „Különleges kérések" szövegmezőt, ahol a felhasználó egyéb megjegyzéseket adhat meg a foglalással kapcsolatban (pl. korai check-in igény, allergiák, különleges alkalom stb.). Ez a szövegmező maximum 500 karaktert fogad el. A különleges kérések az adminisztrátor számára is láthatóak a foglalás részletező oldalán.

A platform egyik legegyedibb funkciója a csereajánlat lehetősége. A foglalási formon egy „Csereajánlat" szekció is megjelenik, ahol a felhasználó megadhatja, mit

ajánl fel az ingatlan-tulajdonosnak cserébe az elszállásolásért. Ez lehet munkavégzés (pl. kertrendezés, festés), termék (pl. házilag készített befőtt, méz) vagy egyéb szolgáltatás. A csereajánlat szövegmező maximum 300 karakteres, és ha ki van töltve, az adminisztrátori felületen is hangsúlyosan jelenik meg.

A csereajánlat nem kötelező, de a platform szellemiségéhez igazodva aktívan ösztönzött. A csereajánlatot tartalmazó foglalások az adminisztrátori felületen egy „Csereajánlat" badge-dzsel vannak jelölve, így az adminisztrátor könnyen azonosíthatja őket. A csereajánlatos foglalásokat az adminisztrátor külön szempont alapján mérlegelhet a jóváhagyásnál.

3.5.5. A foglalás elküldése és visszaigazolás

A „Foglalás elküldése" gombra kattintva – amennyiben minden adat érvényes és nincs ütközés – a foglalás bekerül az adatbázisba „Függőben" státusszal. A felhasználó egy sikeres beküldési üzenetet lát: „Foglalásod sikeresen elküldve! Az adminisztrátor hamarosan feldolgozza a kérelmed. A foglalás státuszát a profil oldalon tudod nyomon követni." Ezután a rendszer átnavigál a /bookings oldalra, ahol az összes foglalás listája látható.

Fontos kiemelni, hogy a „Függőben" státuszú foglalás nem jelenti a szállás automatikus lefoglalását. Az adminisztrátornak manuálisan kell elfogadni vagy elutasítani a kérelmet. Ez a jóváhagyási folyamat szándékos: lehetővé teszi a csereajánlatok értékelését és a speciális helyzetek kezelését. A jövőbeli fejlesztések között szerepel az automatikus e-mail értesítés a foglalás státuszváltásakor.

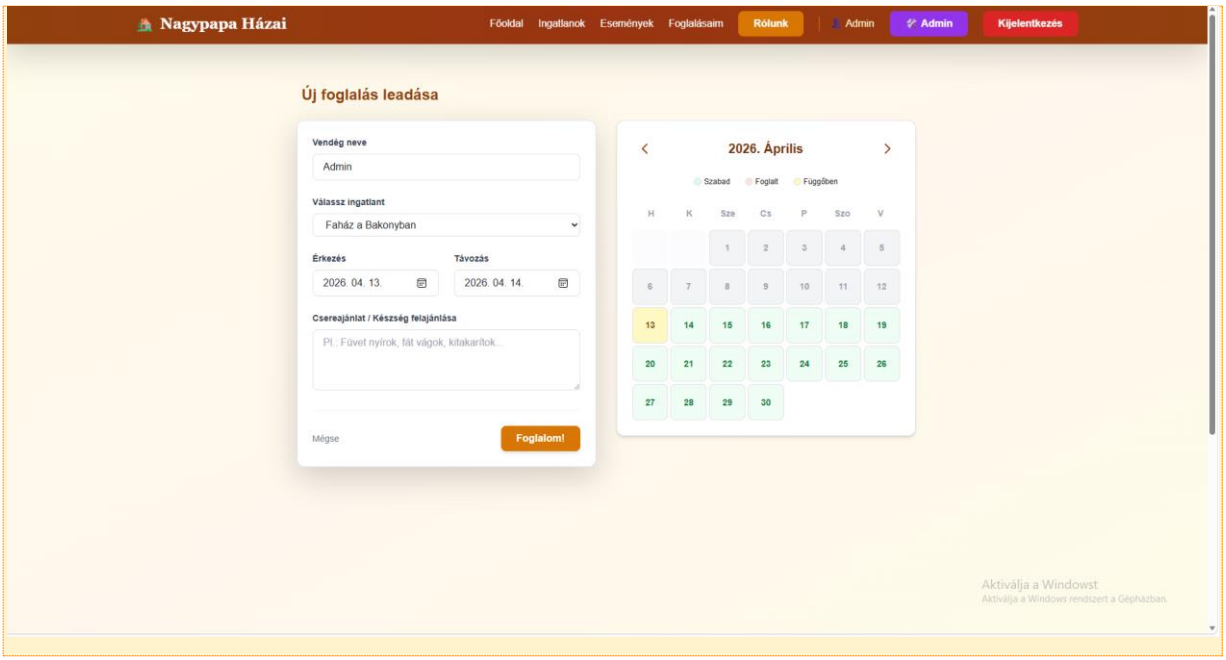
3.6. A foglalási naptár

A foglalási naptár a /bookings/calendar URL-en érhető el, és vizuálisan jeleníti meg az összes ingatlan foglaltsági állapotát az adott hónapra vonatkozóan. Ez az oldal különösen hasznos a felhasználók számára, akik szeretnék áttekinteni, mikor szabad egy adott ingatlan, mielőtt foglalást indítanának. A naptár Bootstrap alapú rács-elrendezést használ, és egyszerű, könnyen értelmezhető szín-kódolással jelzi a foglalt és szabad napokat.

3.6.1. A naptár felépítése

A naptár oldal tetején egy hónap-választó sáv jelenik meg, amelyen a balra és jobbra mutató nyilakkal lehet hónapot váltani, és egy „Ma" gombbal vissza lehet ugrani az aktuális hónapra. Alatta az ingatlan-választó legördülő listából kiválasztható, hogy melyik ingatlan foglaltsági állapotát kívánja megtekinteni a felhasználó, vagy az „Összes ingatlan" opcióval összevont nézet is kérhető.

A naptárban a napok kattintható cellákban jelennek meg. A szabad napok fehér háttérrel, a foglalt napok sötétkék háttérrel és fehér szöveggel jelennek meg. Ha egy nap több különböző ingatlanon is foglalt (összevont nézetben), az összes foglalás nevesített listaként jelenik meg a naptár cellájában. Az elfogadott és a függőben lévő foglalások eltérő szín-kódolással (zöld, piros, sárga) különböztethetők meg.



3.6.2. Naptár interakció

Ha a felhasználó egy adott napra kattint a naptárban, megjelenik egy szóbuborékszerű információs panel (popover), amely felsorolja az azon a napon aktív foglalásokat. Minden foglaláshoz megjelenik a bérlő neve (amennyiben az adminisztrátor megtekinti), az ingatlan neve, a teljes foglalási időszak és a státusz. A panel egy „Részletek" gombbal rendelkezik, amely az adott foglalás részletes oldalára navigál.

A naptár oldal alján egy összegző tábla is megjelenik az adott hónapra vonatkozóan, amely ingatlanonként mutatja az összes foglalt napok számát és a tervezett bevételt (az éjszakánkénti ár és a foglalt éjszakák szorzata). Ez az összegzés

elsősorban az adminisztrátornak hasznos, de a bejelentkezett felhasználók számára is látható.

3.7. Saját foglalások kezelése

A bejelentkezett felhasználók a /bookings URL-en érhetik el saját foglalásainak listáját. Ez az oldal csak az adott felhasználóhoz tartozó foglalásokat jeleníti meg – az adminisztrátorok az admin felületen láthatják az összes felhasználó foglalásait. A foglalások listája táblázatosan jelenik meg, időrendben csökkenő sorrendben (a legutóbbi foglalás van felül).

3.7.1. Foglalások listájának elemei

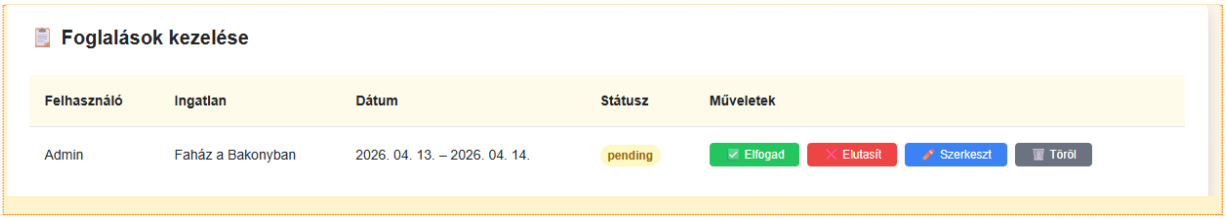
A foglalások listájában minden sor tartalmazza az ingatlan nevét (kattintható, az ingatlan részletes oldalára navigál), az érkezési és távozási dátumot, a vendégszámot, az összeget (éjszakánkénti ár × éjszakák száma), a foglalás státuszát és a műveleti gombokat. A státusz egy Bootstrap Badge elemként jelenik meg, amelynek színe dinamikusan változik a státusz szerint: narancssárga (Függőben), zöld (Elfogadva), piros (Elutasítva), szürke (Lemondva).

Státusz	Badge szín	Felhasználó számára elérhető műveletek	Leírás
Függőben	Narancssárga	Lemondás	Az admin még nem dolgozta fel
Elfogadva	Zöld	Megtekintés, Lemondás (ha jövőbeli)	Az admin elfogadta a kérelmet
Elutasítva	Piros	Megtekintés	Az admin elutasította a kérelmet
Lemondva	Szürke	Megtekintés	A felhasználó lemondta a foglalást
Lezárt	Sötétkék	Megtekintés	A foglalási időszak lejárt

3.7.2. Foglalás részletes nézete

Egy adott foglalás részleteinek megtekintéséhez a „Részletek" gombra kell kattintani. Ez megnyit egy részletes nézetet, amely tartalmazza az összes foglalási adatot: ingatlan neve és helyszíne, érkezési és távozási dátum, éjszakák száma, vendégszám, éjszakánkénti ár, teljes összeg, különleges kérések szövege, csereajánlat szövege (ha volt), foglalás beküldésének dátuma és időpontja, az utolsó státuszváltozás dátuma és az admin megjegyzése (ha van).

A foglalás részletes nézetében a felhasználó látja az összes eddigi státuszváltozást és azok időpontját is (pl. „2026.02.15. 14:32 – Státusz megváltozott: Függőben → Elfogadva"). Ez az audit trail funkció segít a felhasználóknak nyomon követni, mi történik a foglalásukkal. Az audit trail adatok a Bookings táblában tárolt CreatedAt és UpdatedAt mezőkből, valamint egy opcionális StatusHistory táblából kerülnek megjelenítésre.

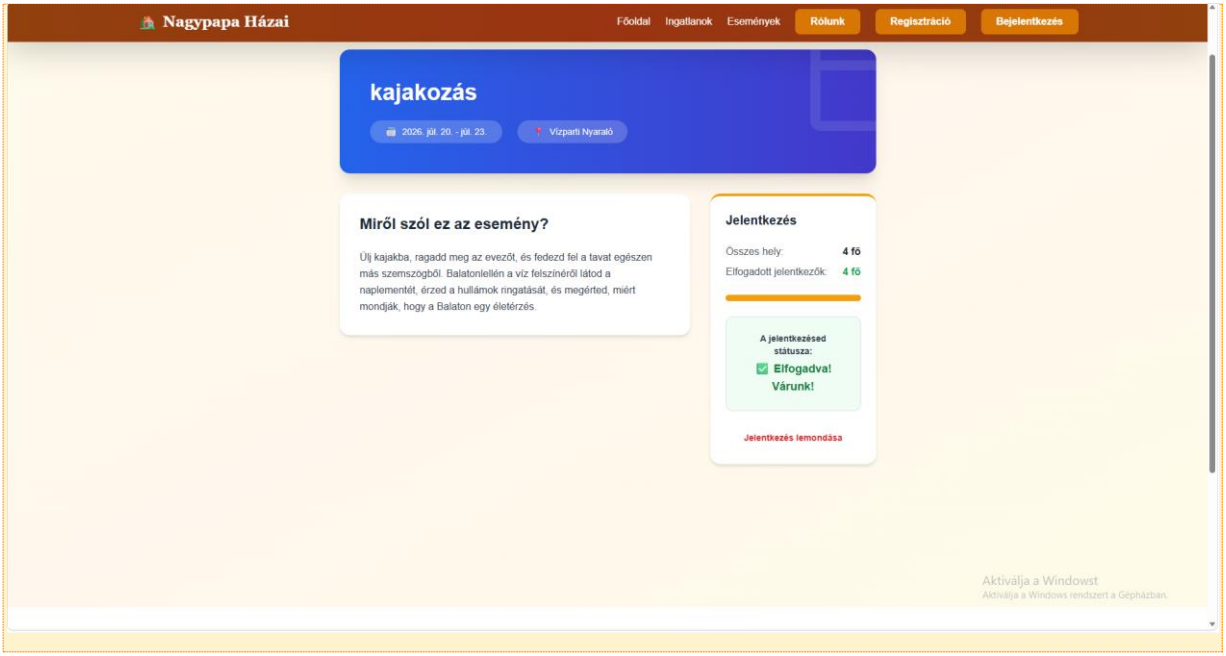


3.8. Az eseménykezelő modul

A Nagypapa Házai platform egy eseménykezelő modult is tartalmaz, amely lehetővé teszi közösségi programok, találkozók és rendezvények szervezését és kezelését. Az eseménykezelő modul a /events URL-en érhető el, és a navigációs sávban az „Események" menüponttal közelíthető meg. Az eseményekre való regisztrációhoz bejelentkezés szükséges, de az események listájának megtekintéséhez nem.

3.8.1. Események listázó oldala

Az események listázó oldalán az összes közelgő esemény kártyás formában jelenik meg. Minden esemény-kártyán szerepel az esemény neve, a dátum és időpont, a helyszín, a leírás kivonat (max. 150 karakter), a maximális résztvevőszám, az aktuálisan regisztrált résztvevők száma és egy „Regisztráció" vagy „Részletek" gomb. Ha az esemény elérte a maximális résztvevőszámot, a kártyán egy „Betelt" badge jelenik meg, és a regisztrációs gomb inaktívvá válik.

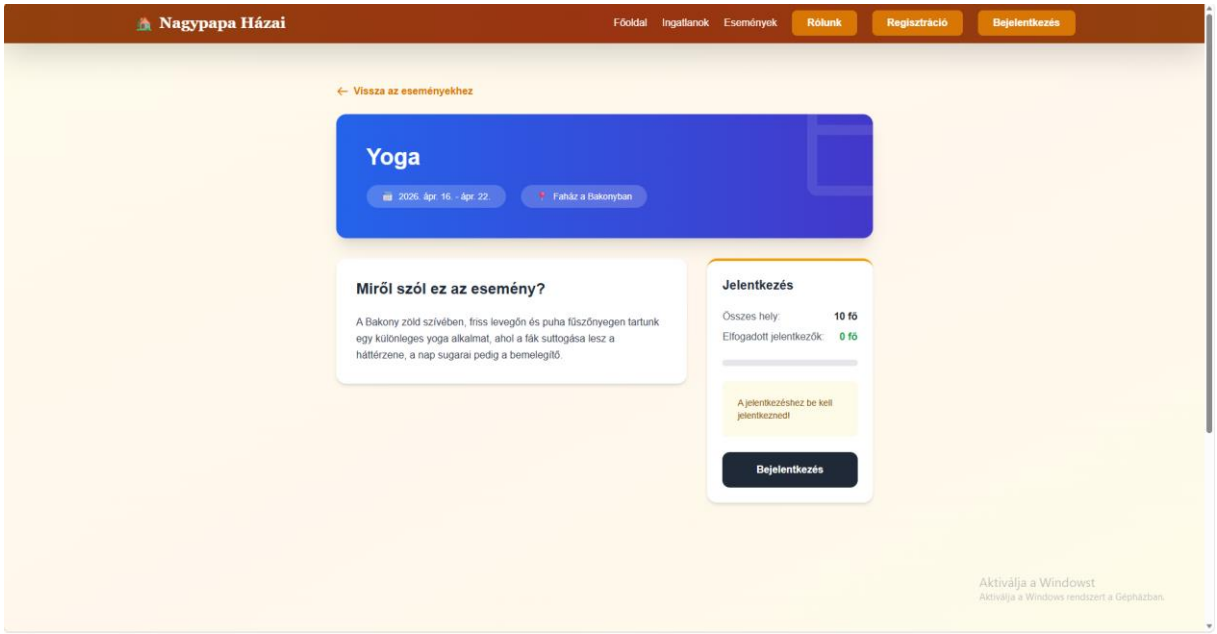


Az események listázó oldala lehetőséget biztosít a szűrésre dátum szerint (pl. csak az aktuális héten vagy az adott hónapban esedékes események), és keresésre a név és helyszín szabad szöveges mezőkkel. A múltbeli események automatikusan elrejtésre kerülnek (a mai napnál régebbiek), de egy „Múltbeli események megjelenítése" gombbal visszahozhatók. Ez lehetővé teszi a korábbi rendezvények archív megtekintését.

3.8.2. Esemény részletes oldala

Az egyedi esemény részletes oldala a /events/{id} URL-en érhető el. Az oldalon megjelenik az esemény teljes leírása, a pontos dátum és időpont (kezdet és vég), a helyszín részletes leírása (cím, esetleg térkép-hivatkozás), a szervező neve és elérhetőségei, a regisztrált résztvevők száma a maximumhoz képest (pl. „23 / 50 hely foglalt"), és egy regisztrációs gomb.

A regisztrációs gomb dinamikusan változik az aktuális állapot függvényében. Ha a felhasználó nincs bejelentkezve: „Bejelentkezés a regisztrációhoz" felirattal a bejelentkezési oldalra irányít. Ha a felhasználó már regisztrált az eseményre: „Regisztráció lemondása" felirattal (piros gomb). Ha az esemény betelt: „Az esemény betelt" felirattal (inaktív, szürke gomb). Ha van szabad hely és a felhasználó nincs regisztrálva: „Regisztráció" felirattal (zöld gomb).



3.8.3. Eseményre való regisztráció folyamata

Az eseményre való regisztráció egyetlen kattintással elvégezhető: a „Regisztráció" gombra kattintva a rendszer azonnal elvégzi a regisztrációt, és visszajelzést ad a felhasználónak egy Bootstrap Alert komponensben: „Sikeresen regisztráltál az eseményre! A résztvevők száma frissítve." A résztvevők számlálója valós időben frissül a Blazor SignalR-kapcsolat révén, amit egy másik lapfülön nyitott eseménylap is azonnal tükröz.

Az esemény lemondása szintén egyetlen kattintással elvégezhető a „Regisztráció lemondása" gombbal. Előtte egy Bootstrap Modal megerősítő ablak jelenik meg: „Biztosan le kívánja mondani a regisztrációját erre az eseményre? Ez a művelet nem visszavonható." Igen válasz esetén a regisztráció törlődik, és a résztvevők számlálója csökken. A lemondást követően a gomb visszaáll „Regisztráció" feliratú zöld gombra.

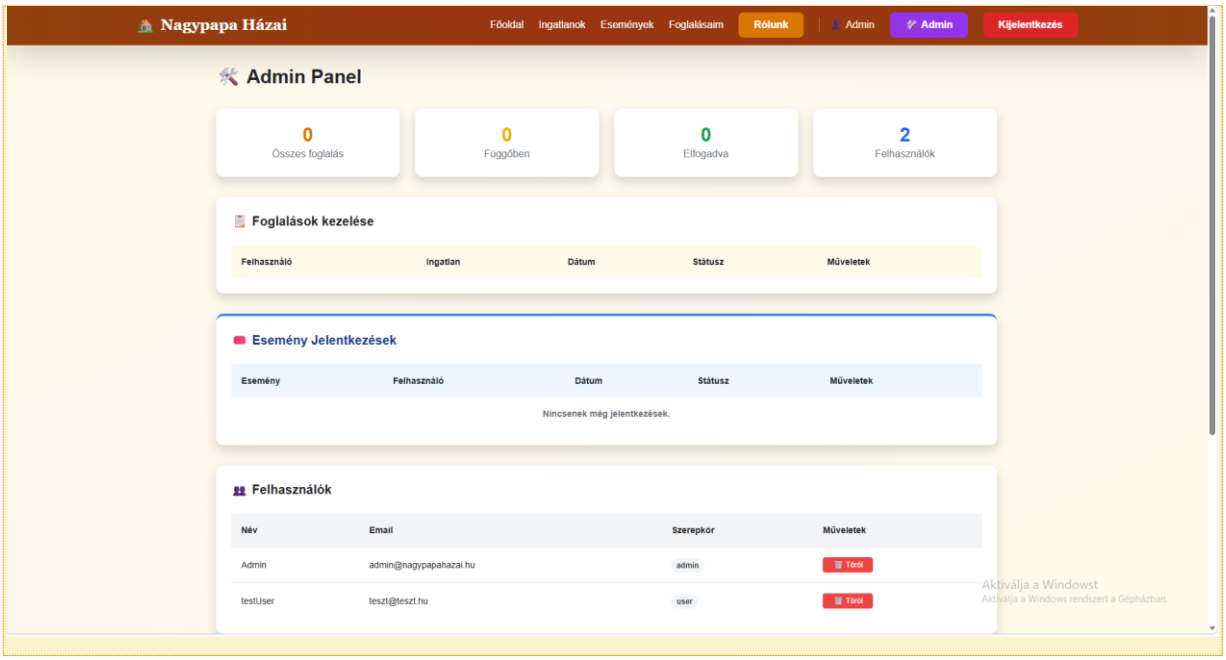
3.8.4. Saját esemény-regisztrációk kezelése

A bejelentkezett felhasználók a profiloldalukon vagy a /events URL-en az összes saját esemény-regisztrációjukat megtekinthetik. A lista tartalmazza az esemény nevét, a dátumot, a regisztráció időpontját és egy „Lemondás" gombot. Lejárt események esetén a lemondás gomb inaktív. Ez az áttekintő lista segít a felhasználóknak kézben tartani a naptárjukat és az elkövetkezendő eseményeiket.

3.9. Az adminisztrátori felület részletes leírása

Az adminisztrátori felület a platform összes adatának és folyamatának kezelését teszi lehetővé az arra jogosult adminisztrátor felhasználók számára. Az admin felület csak az „Admin" szerepkörrel rendelkező felhasználók számára érhető el; normál felhasználók esetén a /admin URL-re navigálás automatikusan a főoldalra irányítja a felhasználót egy „Nincs hozzáférési jogosultságod ehhez az oldalhoz" figyelmeztető üzenettel.

Az adminisztrátori felületre az admin@nagypapahazai.hu email-cím és az Admin123! jelszóval lehet bejelentkezni (lásd 3.1.4. fejezet). Az admin felület az alábbi fő modulokat tartalmazza: Foglaláskezelés (/admin/bookings), Ingatlankezelés (/admin/properties), Eseménykezelés (/admin/events) és egy összefoglaló Dashboard (/admin). Minden admin aloldal az admin navigációs menüből érhető el, amely csak a bejelentkezett adminisztrátorok számára jelenik meg.



3.9.1. Admin Dashboard

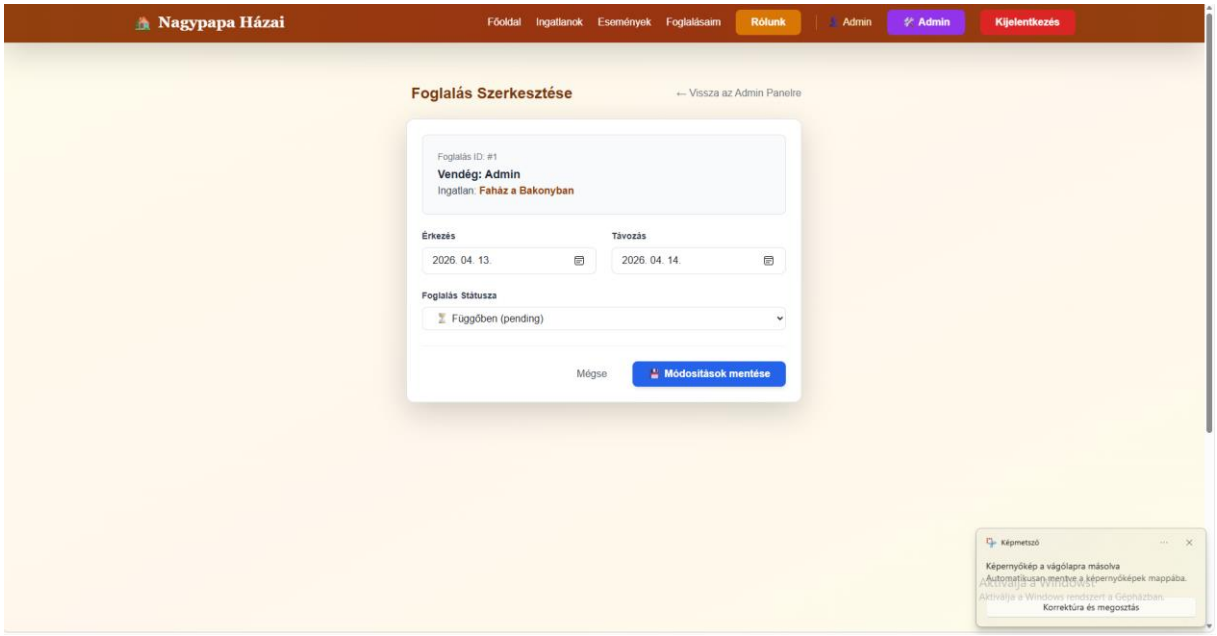
Az admin dashboard (/admin) összefoglaló statisztikákat jelenít meg a rendszer aktuális állapotáról. A dashboard négy fő KPI-kártyát tartalmaz: összes foglalás száma (és bontás státusz szerint), összes ingatlan száma (aktív / inaktív bontásban), összes regisztrált felhasználó száma és az aktuálisan futó / közelgő események száma. Ezek az adatok real-time frissülnek, mivel a Blazor Server automatikusan pushol minden változást a kliens felé.

A dashboard-on egy heti foglalási grafikon is megjelenik, amely az adott hét minden napjára vonatkozóan mutatja a beérkezett, elfogadott és elutasított foglalások számát. A grafikon egyszerű Bootstrap-alapú vizualizációval készült (CSS width alapú vízszintes oszlopdiagram). Alacsonyabb fejlesztői erőforrás-igénnyel ugyanolyan informativitást biztosít, mint egy komplexebb JavaScript-alapú diagramkönyvtár. A dashboard jobb oldalán egy „Függőben lévő foglalások" szekció is megjelenik, amely az azonnali beavatkozást igénylő foglalásokat listázza.

3.9.2. Foglaláskezelés az admin felületen

Az admin foglaláskezelő oldala (/admin/bookings) az összes felhasználó összes foglalását tartalmazza. Az oldal tetején szűrési lehetőségek állnak rendelkezésre: szűrés státusz szerint (Függőben / Elfogadva / Elutasítva / Lemondva / Összes), szűrés ingatlan szerint (legördülő lista az összes ingatlannal) és szűrés dátumtartomány szerint (tól–ig dátumválasztókkal). A szűrők kombinálhatók és azonnal érvényesülnek, oldal-újratöltés nélkül.

A foglalások táblázatában a következő oszlopok szerepelnek: azonosító (#ID), felhasználó neve és email-je, ingatlan neve, érkezési és távozási dátum, vendégszám, összeg, csereajánlat (ha van, egy ikon jelzi), különleges kérések (ha van, egy ikon jelzi), foglalás beküldési dátuma és státusz badge. Az utolsó oszlopban „Elfogad" és „Elutasít" gombok (csak Függőben státuszú foglalásokra), „Részletek" gomb (minden foglalásra) és „Törlés" gomb (piros, csak adminisztrátor-jogosultsággal) találhatók.

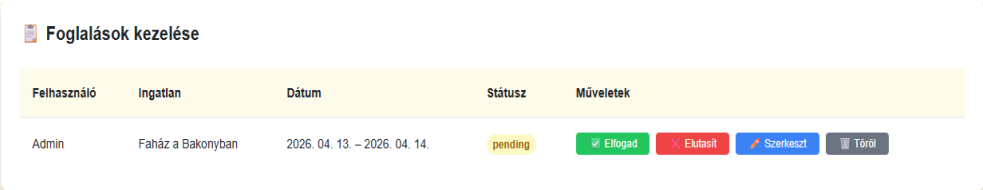


3.9.3. Foglalás elfogadása és elutasítása

A „Függőben" státuszú foglalások sorában az „Elfogad" (zöld) és „Elutasít" (piros) gombok jelennek meg. Az „Elfogad" gombra kattintva egy modális ablak nyílik meg, amelyen az admin megadhat egy opcionális megjegyzést a felhasználó számára (pl. „Kérjük, lépjen be 14:00 után."). Az „OK" gombra kattintva a foglalás státusza „Elfogadva"-ra változik, és az ütközésvizsgálat elvégzése után az adatbázisban frissítésre kerül.

Az „Elutasít" gombra kattintva szintén modális ablak nyílik meg, amelyen kötelező megadni az elutasítás indokát (kötelező mező, legalább 10 karakter). Az indok a foglalás részletes oldalán jelenik meg a felhasználó számára is, így az adminisztrátor kommunikálhat a bérlőkkel a döntés okáról. Az elutasítás után a foglalt napok felszabadulnak, és más felhasználók is foglalhatnak arra az időszakra.

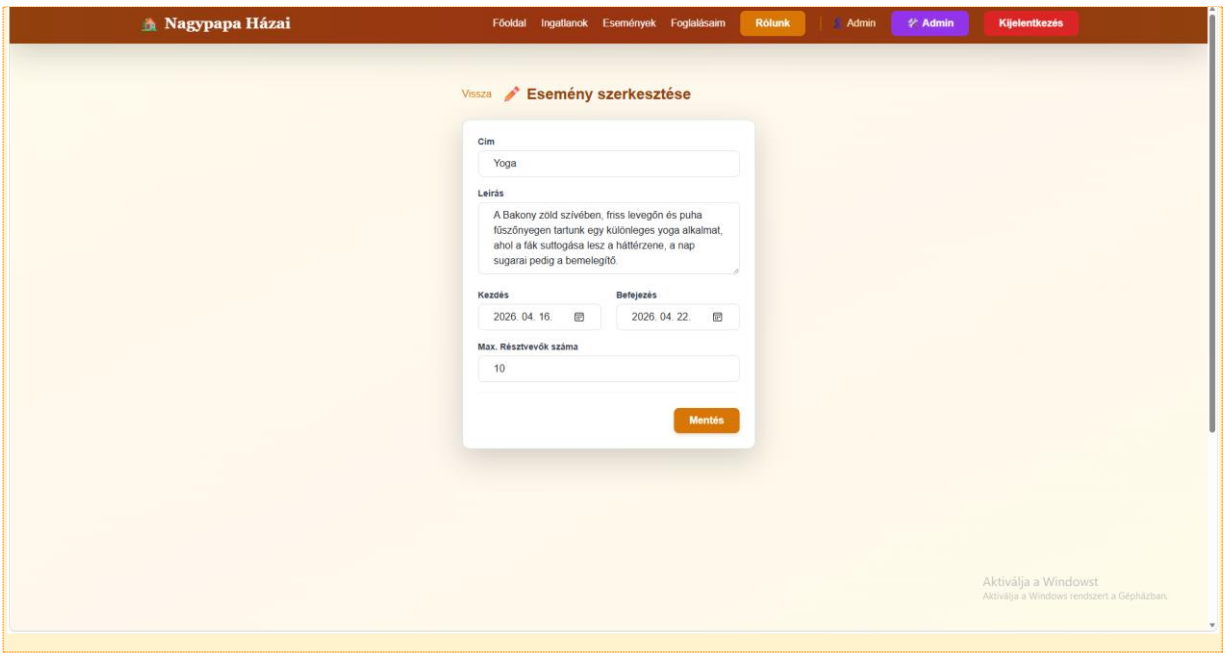
Az elfogadási és elutasítási műveletek mindkét esetben azonnal hatályos változtatást tesznek az adatbázisban, és a SignalR-kapcsolaton keresztül az összes nyitott admin munkamenetben frissül a nézet. Ez azt jelenti, hogy ha két adminisztrátor egyszerre nézi ugyanazt az oldalt, mindkettő azonnal látja, ha a másik elvégez egy változtatást. Ennek köszönhetően elkerülhetők az egymást felülíró párhuzamos módosítások.



3.9.5. Eseménykezelés az admin felületen

Az eseménykezelő admin oldala (/admin/events) az összes esemény listáját tartalmazza, a jövőbeli és a múltbeli eseményeket egyaránt. Minden esemény sorában megjelenik az esemény neve, a dátum és időpont, a helyszín, a regisztrált résztvevők száma a maximumhoz képest (pl. „23 / 50"), az esemény státusza (Aktív / Lejárt) és a műveleti gombok.

Új esemény létrehozásához az „Új esemény" gombra kell kattintani. A megnyíló formban a következő adatokat kell megadni: Esemény neve (kötelező), Dátum és időpont (kötelező, jövőbeli dátum kell, hogy legyen), Befejezési időpont (opcionális), Helyszín (kötelező), Leírás (opcionális), Maximális résztvevőszám (kötelező, min. 1). Az adminisztrátor eseményeket szerkeszthet és törölhet is – törlés előtt az összes regisztráció is törlésre kerül, erről a felhasználók értesítést kapnak (jövőbeli e-mail értesítési funkcióval).



3.9.6. Felhasználókezelés az admin felületen

Az admin felület tartalmaz egy felhasználókezelő modult is (/admin/users), ahol az összes regisztrált felhasználó listája megtekinthető. Minden felhasználó sorában

megjelenik a neve, email-je, regisztrációjának dátuma, szerepköre (Admin / User) és az összes foglalásainak száma. Az adminisztrátor megváltoztathatja egy felhasználó szerepkörét (User ↔ Admin), deaktiválhatja a fiókját és megtekintheti a foglalási előzményeit.

Felhasználó törlésére az admin felületen nem kerülhet sor a jelenlegi verzióban – ez adatvédelmi és adatintegritási okokból korlátozott funkció. Egy felhasználó teljes eltávolítása az adatbázis-adminisztrátor által közvetlenül az SQL Server Management Studio-ban végezhető el, mivel ez olyan kaszkád-törlési folyamatot igényel, amelyet a platform felhasználói felülete nem kezel biztonságosan. Ez is a jövőbeli fejlesztési lista részét képezi.

3.10. Hibaelhárítási útmutató

A hibaelhárítási fejezet azokat a leggyakoribb problémákat és megoldásaikat gyűjti össze, amelyekkel a platform felhasználói és fejlesztői szembesülhetnek. A hibák két fő kategóriára oszthatók: telepítési és konfigurációs problémák (amelyek csak az első beüzemeléskor merülnek fel) és üzemeltetési problémák (amelyek a mindennapi használat során fordulhatnak elő).

3.10.1. Telepítési és konfigurációs problémák

Hiba / Tünet	Valószínű ok	Megoldás
Az alkalmazás nem indul el, SSL tanúsítvány hiba	A fejlesztői HTTPS tanúsítvány nem megbízható	Futtassa: dotnet dev-certs https --trust
A kapcsolati karakterlánc hibás, SQL nem érhető el	Az SQL Server Express instance neve eltérő	Ellenőrizze az appsettings.json ConnectionStrings értékét; próbálja localhost\SQLEXPRESS
Az adatbázis nem jön létre, EF migráció hiba	Az EF Core eszközök nincsenek telepítve	Futtassa: dotnet tool install --global dotnet-ef

Hiba / Tünet	Valószínű ok	Megoldás
A 7206-os port már foglalt	Más alkalmazás használja a portot	Módosítsa a launchSettings.json-ban az applicationUrl értékét
A két projekt nem indul egyszerre	Multiple Startup Projects nem lett beállítva	VS: Solution Properties → Multiple startup projects
NuGet csomagok hiányoznak	Nem futott a restore parancs	Futtassa: dotnet restore

3.10.2. Üzemeltetési és felhasználói problémák

Hiba / Tünet	Valószínű ok	Megoldás
Böngésző frissítés után kijelentkeztetett a rendszer	A session nem perzisztált (ismert korlátozás)	Normál viselkedés; jelentkezzen be újra; ez jövőbeli fejlesztéssel javítható
A foglalás elküldésekor „belső szerverhiba" üzenet	Az API nem érhető el vagy leállt	Ellenőrizze, hogy az API projekt is fut-e (Task Manager / Visual Studio Debug)
A naptárban nem látszanak a foglalások	A naptár szűrői más értékre vannak állítva	Ellenőrizze az ingatlan-választó és a hónap-választó beállításait
Regisztráció sikertelen „már létező email" hibával	Az email-cím már regisztrálva van	Használjon más email-t, vagy lépjen be a meglévő fiókba
A csereajánlat mező nem jelenik meg	Nem az ingatlan foglalási formján van	A csereajánlat csak a /bookings/new oldalon érhető el
Az esemény „Betelt" státuszban van, de van szabad hely	Cache-beli adat eltérés	Frissítse az oldalt (F5 vagy Ctrl+R)

Hiba / Tünet	Valószínű ok	Megoldás
Az admin felületre navigálva a főoldalra kerülök	Nincs admin szerepkör a fiókhoz	Az admin@nagypapahazai.hu fiókkal kell bejelentkezni

3.10.3. Böngésző konzol hibák és értelmezésük

A Blazor Server alkalmazásoknál a böngésző konzolján (F12 → Console) megjelenő hibák általában nem a kliensoldali JavaScript-kódra vonatkoznak, hanem a SignalR WebSocket-kapcsolat állapotára. Az alábbiakban a leggyakoribb konzol-üzenetek és értelmezésük szerepel.

- "WebSocket connection failed" – A SignalR kapcsolat megszakadt, általában ha a szerveralkalmazás leáll vagy a hálózati kapcsolat megbízhatatlan. Megoldás: ellenőrizze, hogy a Blazor alkalmazás fut-e, és töltse újra az oldalt.
- "Circuit disconnected" – A Blazor Server áramkör (circuit) lekapcsolt. Ez előfordulhat hosszabb inaktivitás (alapértelmezés szerint 3 perc) után. Megoldás: töltse újra az oldalt.
- "Failed to fetch" – Az API-hívás nem ért el a backend-hez. Ez általában azt jelenti, hogy az API projekt nem fut. Megoldás: indítsa újra az API projektet.
- "Unhandled exception rendering component" – Egy Blazor komponens renderelése során nem kezelt kivétel keletkezett. A hiba részletei a szerver oldali logban jelennek meg; a böngésző konzolban csak az összefoglaló üzenet látható.

3.10.4. Szerver oldali logok értelmezése

A Blazor Server alkalmazás részletes naplókat ír a Visual Studio Output ablakba (Debug módban) és opcionálisan fájlba (ha a Serilog vagy NLog konfigurálva van). A naplók értelmezéséhez a log szintek ismerete szükséges: Debug (részletes fejlesztői infók), Information (általános esemény-naplók), Warning (nem kritikus figyelmeztetések), Error (kezelt hibák) és Critical (kritikus, az alkalmazás leállítását okozó hibák).

A leggyakoribb szerver oldali hiba a SqlException, amely az adatbázis-kapcsolattal vagy az SQL lekérdezésekkel kapcsolatos problémákat jelzi. Ha ilyen hiba jelenik meg a logban, ellenőrizze a ConnectionStrings értékét az appsettings.json-ban, és győződjön meg arról, hogy az SQL Server Express szolgáltatás fut (Szolgáltatások → SQL Server (SQLEXPRESS) → Állapot: Fut). A második leggyakoribb hiba az InvalidOperationException az EF Core DbContext-tel kapcsolatban, amely általában arra utal, hogy egy entitás konfigurálása hibás, vagy a migráció nem lett alkalmazva.

3.11. Biztonsági megfontolások és adatvédelem

A NagypapaHázai platform tervezése során alapvető biztonsági szempontok figyelembevételével jártunk el. Bár a platform jelenleg fejlesztői (localhost) környezetben üzemel és valódi személyes adatok nem kerülnek feldolgozásra, a biztonsági megvalósítások dokumentálása elengedhetetlen a jövőbeli termelési környezetbe való átállás szempontjából. Ez a fejezet összefoglalja az alkalmazott biztonsági mechanizmusokat és az ismert korlátokat.

3.11.1. Jelszókezelés

A felhasználói jelszavak BCrypt algoritmussal hashelve kerülnek tárolásra az adatbázisban. A BCrypt egy adaptív hash-függvény, amely beépített „cost factor"-t (iterációs tényezőt) alkalmaz; ez megnehezíti a brute-force és dictionary támadásokat, mivel minden hash-elési kísérlet számítási időt igényel. A jelszavak soha nem kerülnek visszafejthető formában tárolásra, sem logfájlokba, sem adatbázisba.

A minimális jelszókövetelmények a következők: legalább 8 karakter, legalább egy nagybetű, legalább egy kisbetű és legalább egy szám. Speciális karakterek megkövetelése jelenleg nem érvényes, de a 3.11.3. fejezetben ismertetett jövőbeli fejlesztések között szerepel a jelszó-erősség mérő és a 2FA (kétlépéses hitelesítés) bevezetése.

3.11.2. Session kezelés és CSRF védelm

A hitelesítési adatok ASP.NET Core Session middleware-rel tárolódnak a szerveren. A session azonosítóját egy HTTP-Only Cookie tárolja a böngészőben, amely JavaScript-kódból nem olvasható, ezáltal XSS (Cross-Site Scripting)

támadásoknak nem kitett. A session lejárati ideje alapértelmezés szerint 20 perc inaktivitás után, de ez az alkalmazás konfigurációjában módosítható.

Az ASP.NET Core Blazor Server beépített CSRF (Cross-Site Request Forgery) védelemmel rendelkezik a SignalR-kapcsolaton keresztüli kommunikáció esetén, mivel az összes kérés a WebSocket-csatornán zajlik, amelyet a böngésző automatikusan SameSite cookie-korlátozásokkal véd. Az API végpontok esetén az antiforgery token validáció van alkalmazva a POST, PUT és DELETE kéréseknél.

3.11.3. Szerepkör-alapú hozzáférés-vezérlés

A Nagypapa Házai platform szerepkör-alapú hozzáférés-vezérlést (RBAC) alkalmaz két szerepkörrel: „User" (normál felhasználó) és „Admin" (adminisztrátor). A szerepkörök ellenőrzése a Blazor komponenseken belül if-blokkokkal, az API kontrollereken pedig az [Authorize(Role = "Admin")] attribútummal valósul meg. A jelenlegi implementáció nem alkalmaz policy-based authorizationt, de ez a jövőbeli fejlesztések tervei között szerepel.

Minden adminisztrátori funkció kizárólag az Admin szerepkörű felhasználók számára érhető el. Ha egy normál felhasználó közvetlenül próbálja meghívni az admin API végpontokat (pl. curl-lel vagy Postman-nel), a 403 Forbidden HTTP válaszkóddal kerül visszautasításra. A Blazor UI-ban az admin-specifikus menüpontok és gombok conditionálisan renderelődnek, tehát normál felhasználók számára ezek az UI-elemek nem is jelennek meg.

3.12. Teljesítmény és skálázhatóság

A Nagypapa Házai jelenlegi architektúrája fejlesztői/kiscsoportos felhasználásra lett tervezve, nem ipari skálájú terhelés kezelésére. Ennek ellenére a Blazor Server architektúra képes elfogadható teljesítményt nyújtani kisebb felhasználói bázis esetén, és a rendszer megismerése céljából fontos érteni annak teljesítmény-korlátait és fejlesztési lehetőségeit.

3.12.1. Blazor Server teljesítmény-karakterisztikák

A Blazor Server minden aktív felhasználói munkamenethez egy-egy SignalR áramkört (circuit) tart fenn a szerveren. Minden áramkör memóriát és CPU-időt igényel, amelynek mértéke a felhasználói interakciók sűrűségétől függ. Empirikus

adatok alapján egy egyszerű Blazor Server alkalmazás körülbelül 250 KB memóriát igényel per áramkör, ami azt jelenti, hogy 4 GB RAM-mal körülbelül 16 000 egyidejű felhasználó kiszolgálható elméleti maximumként (természetesen a CPU és az adatbázis-kapcsolatok korlátai ezt tovább szűkítik).

A Nagypapa Házai jelenlegi konfigurációjában az SQL Server Express adatbázis is korlátot jelent: az Express kiadás maximum 10 GB adatbázisméretig és 1 GB RAM-ig korlátozza a felhasználást. Ez a fejlesztői célokra bőven elegendő, de produkciós környezetben az SQL Server Standard vagy Enterprise kiadásra való áttérés lenne szükséges.

3.12.2. Lekérdezés-optimalizálás és EF Core teljesítmény

Az Entity Framework Core által generált SQL lekérdezések hatékonyságát az alkalmazás fejlesztési fázisában a Visual Studio Diagnostic Tools ablakában monitoroztuk. A kritikus lekérdezési pontok (foglalások listázása, ütközésvizsgálat) indexelt oszlopokra támaszkodnak: az UserId és PropertyId foreign key oszlopok indexelve vannak, ami a szűrött lekérdezések teljesítményét jelentősen javítja.

Fontos megjegyezni, hogy az EF Core alapértelmezés szerint lazy loading nélkül működik (a Nagypapa Házai projektben sem engedélyezett), ami N+1 lekérdezési problémákhoz vezethet, ha nem figyelünk az Include() metódusok helyes használatára. A fejlesztés során explicit eager loading-ot alkalmazunk (pl. foglalások betöltésekor az ingatlan és a felhasználó navigációs tulajdonságait is betöltjük), ami csökkenti az adatbázis round-trip-ek számát.

3.12.3. Jövőbeli skálázhatósági lépések

A platform jövőbeli, produkciós verzióinak esetén az alábbi skálázhatósági fejlesztések tervezettek: Response caching bevezetése a statikus tartalmakhoz (főoldal, ingatlanok listája), Redis-alapú distributed session store a horizontális skálázáshoz, Azure SQL Database-re való áttérés az SQL Server Express helyett, Azure SignalR Service integrálása a SignalR Backplane megvalósításához (ami lehetővé teszi több szerver párhuzamos futtatását), és CDN (Content Delivery Network) alkalmazása a statikus fájlokhoz (Bootstrap, JavaScript-könyvtárak).

Ezek a fejlesztések a 2. fejezet (Fejlesztői dokumentáció) 2.9. alszakaszában részletesen tárgyalt jövőbeli fejlesztési tervek részét képezik. A jelenlegi prioritás az

alapfunkciók stabilitása és a felhasználói élmény finomítása, nem az ipari skálájú teljesítmény. A platform tervezési fázisában tudatosan döntöttünk a Blazor Server architektúra mellett annak egyszerűsége és a .NET ökoszisztémával való szoros integrációja miatt, tudatában lévén a skálázhatósági korlátoknak.

3.13. Tesztelési útmutató

A Nagypapa Házai platform tesztelési folyamatát ez a fejezet dokumentálja. A tesztelés két fő részből áll: manuális felhasználói tesztelés (amelyet mind a fejlesztői csapat, mind külső tesztelők elvégeznek) és automatizált tesztelés (amelyet a fejlesztői pipeline részeként futtatunk). A tesztelési útmutató elsősorban a manuális tesztelési forgatókönyveket fókuszálja, mivel az automatizált tesztek a fejlesztői dokumentáció 2.7. alszakaszában kerülnek részletezésre.

3.13.1. Manuális tesztelési forgatókönyvek

Az alábbi táblázat a legfontosabb manuális tesztelési forgatókönyveket, az elvárt és a tényleges eredményeket tartalmazza. Minden teszt az alapértelmezett seed adatokkal fut, és feltételezi, hogy az alkalmazás frissen van elindítva (nem áll fenn semmilyen különleges állapot). A tesztelő a teszt@nagypapahazai.hu fiókot használja normál felhasználói teszteknél, az admin@nagypapahazai.hu fiókot adminisztrátori teszteknél.

#	Teszt neve	Lépések	Elvárt eredmény	Státusz
T01	Regisztráció érvényes adatokkal	Nyissa meg /register → töltse ki az összes mezőt → kattintson Regisztráció	Sikeres regisztráció, átirányítás /login oldalra, zöld üzenet	PASS
T02	Regisztráció ismételt email-lel	Próbáljon regisztrálni teszt@nagypapahazai.hu email-lel	Hibaüzenet: "Ez az email-cím már regisztrálva van"	PASS
T03	Bejelentkezés érvényes adatokkal	/login → teszt@nagypapahazai.hu / Teszt123! → Bejelentkezés	Sikeres bejelentkezés, főoldalra navigálás	PASS

#	Teszt neve	Lépések	Elvárt eredmény	Státusz
T04	Bejelentkezés hibás jelszóval	/login → teszt@nagypapahazai.hu / rossz_jelszó → Bejelentkezés	Hibaüzenet: "Hibás email-cím vagy jelszó"	PASS
T05	Foglalás érvényes adatokkal	Bejelentkezés → /bookings/new → válasszon ingatlant, dátumot, vendégszámot → elküld	Foglalás létrehozva Függőben státusszal, átirányítás /bookings-ra	PASS
T06	Foglalás ütközésvizsgálat	Próbáljon foglalni egy már foglalt időszakra	Hibaüzenet a foglaltságról, naptár hivatkozás megjelenik	PASS
T07	Admin: foglalás elfogadása	Admin fiókkal /admin/bookings → Függőben foglalás → Elfogad	Foglalás státusza Elfogadva-ra változik	PASS
T08	Admin: új ingatlan hozzáadása	Admin fiókkal /admin/properties → Új ingatlan → kitöltés → Mentés	Új ingatlan megjelenik a listában és az ingatlankereső oldalon	PASS
T09	Esemény regisztrációja	Bejelentkezés → /events → esemény → Regisztráció gomb	Sikeres regisztráció, résztvevőszámláló növekszik	PASS
T10	Admin felület hozzáférés normál user-rel	Teszt user fiókkal próbálja megnyitni /admin oldalt	Átirányítás főoldalra, "Nincs hozzáférési jogosultságod" üzenet	PASS

3.13.2. Edge case tesztelési forgatókönyvek

Az edge case (határeset) tesztek az alkalmazás robusztusságát ellenőrzik, különösen az érvénytelen adatok, extrém értékek és szokatlan felhasználói viselkedés esetén. Ezek a tesztek kritikusak, mert a határesetekben előforduló bugok sokszor a legsúlyosabb következményekkel járnak (pl. inkonzisztens adatbázis-állapot, nem kezelt kivételek).

#	Edge case leírása	Teszt bemenet	Elvárt viselkedés
E01	Foglalás pontosan annyi vendéggel, amennyi a kapacitás	Faház a Bakonyban (max. 10 fő) → 10 fő vendégszám	Elfogadva, nincs hiba
E02	Foglalás egy nappal	Érkezési dátum = Ma, Távozási dátum = Holnap	Elfogadva (1 éjszaka)
E03	Foglalás ugyanazon a napon mint egy már lejárt foglalás	Egy már Lezárt foglalás időszakára foglalás	Elfogadva (nincs ütközés)
E04	Nagyon hosszú csereajánlat szöveg (>300 karakter)	301 karakteres csereajánlat szöveg beírása	Validációs hiba, form nem küldhető be
E05	Esemény maximális résztvevőszámon (pontosan)	50/50-es esemény → 50. regisztráló	Elfogadva, de utána a gomb Betelt-re változik
E06	Egyidejű foglalás ugyanarra az időszakra (race condition)	Két böngészőben szimultán ugyanaz a foglalás küldése	Az egyik elfogadva, a másik ütközési hibával visszautasítva
E07	SQL injection kísérlet a keresőmezőkben	Beírás: '; DROP TABLE Properties;--	EF Core paraméterezett lekérdezések miatt nem hajtódik végre, normális szövegként kezeli

3.13.3. Kompatibilitás-tesztelés

A platform kompatibilitás-tesztelése az alábbi böngészőkön és eszközökön volt elvégezve. A tesztelés során vizsgáltuk az összes UI-elem megjelenését, az interaktív elemek működését és a SignalR-kapcsolat stabilitását minden kombináción.

Böngésző	Verzió	Asztali	Mobil (Android)	Mobil (iOS)	Eredmény
Google Chrome	120.x	PASS	PASS	PASS	Teljes körű támogatás
Mozilla Firefox	121.x	PASS	PASS	PASS	Teljes körű támogatás
Microsoft Edge	120.x	PASS	PASS	N/A	Teljes körű támogatás
Apple Safari	17.x	PASS	N/A	PASS	Teljes körű támogatás
Opera	106.x	PASS	N/A	N/A	Teljes körű támogatás
Internet Explorer 11	11.x	FAIL	N/A	N/A	Nem támogatott (WebSocket hiánya)

3.14. Akadálymentesítési szempontok

A Nagypapa Házai platform fejlesztésekor alapvető akadálymentesítési (accessibility) szempontok is figyelembevételre kerültek, bár az alkalmazás nem törekszik a WCAG 2.1 AA szintű teljes megfelelésre a jelenlegi fejlesztői verzióban. A legfontosabb akadálymentesítési megvalósítások az alábbiak szerint kerülnek dokumentálásra, a hiányosságok feltüntetésével együtt.

3.14.1. Meglévő akadálymentesítési megvalósítások

Az összes interaktív elem (gombok, linkek, input-ok) keyboard-fókuszálható, ami azt jelenti, hogy Tab billentyűvel navigálhatók, és Enter/Space billentyűkkel

aktiválhatók. A form mezők rendelkeznek aria-label vagy megfelelően összekapcsolt label elemekkel a képernyőolvasók számára. A hibás mezők aria-invalid="true" attribútummal jelzik állapotukat a képernyőolvasóknak. A Bootstrap 5 badge és alert komponensek rendelkeznek szerepkör attribútumokkal (role="alert", role="status").

A szín-kontraszt arányok az WCAG AA minimális szintjének megfelelnek a szöveges tartalmak esetén: a fehér háttéren megjelenő sötétkék (#1F3864) szöveg kontrasztaránya 12.63:1, a fehér szöveg a kék (#2E74B5) háttéren 4.62:1, mindkettő meghaladja a 4.5:1-es minimumot. Az adminisztrátori felületen a piros/zöld státusz badge-ek mellé szöveges státusz-felirat is megjelenik, nem csak szín alapján különböztetve meg az állapotokat.

3.14.2. Ismert akadálymentesítési hiányosságok

A jelenlegi verzióban az alábbi akadálymentesítési hiányosságok ismertek és dokumentáltak: (1) A naptár komponens nem teljes körűen keyboard-navigálható – a dátumkellák egérkattintást igényelnek. (2) A modális dialógusok nem küldik vissza automatikusan a fókuszt az azt megnyitó gombra a bezárás után. (3) Az oldalbetöltési folyamat nem jelzett hangjelzéssel vagy vizuális indikátorral a képernyőolvasó felhasználók számára. (4) A dinamikusan frissülő tartalmak (pl. foglalásszámláló) nem rendelkeznek aria-live régióval, ami megnehezíti a képernyőolvasó felhasználóknak a változások nyomon követését.

Ezek a hiányosságok a jövőbeli fejlesztési lista részét képezik, és egy esetleges WCAG auditálás elvégzése tervezett a következő fejlesztési ciklus során. Az akadálymentesítés fontosságát a csapat felismeri, és kötelezettséget vállalt a folyamatos javításra.

3.15. Az adatbázis felhasználói szempontból

Bár a felhasználók nem kerülnek közvetlen kapcsolatba az adatbázissal, annak megértése segíti a platform viselkedésének magyarázatát. Ez az alfejezet nem technikai szemszögből (amely a fejlesztői dokumentáció 2.3. fejezetében szerepel), hanem a felhasználói élmény szempontjából vizsgálja az adatbázis-struktúrát: milyen adatok tárolódnak, meddig maradnak meg, és hogyan hatnak a felhasználói felületre.

3.15.1. Felhasználói adatok tárolása

Regisztráció során az alábbi adatok kerülnek tárolásra: teljes név, email-cím, BCrypt-tel hashelt jelszó, regisztrációs dátum, szerepkör (User/Admin), utolsó módosítás dátuma. Email-cím és telefonszám (opcionális) formájú személyazonosítási adatokon kívül a platform semmilyen más személyes adatot (pl. lakcím, bankszámlaszám, személyi igazolvány szám) nem kér és nem tárol. Az adatbázisban tárolt adatok a fejlesztői gépen lévő SQL Server Express lokális adatbázisában maradnak, és nem kerülnek semmilyen külső szerverre.

Az adatmegőrzési időszakkal kapcsolatban a platform nem rendelkezik automatikus törlési mechanizmussal. A felhasználói adatok mindaddig megmaradnak az adatbázisban, amíg azokat manuálisan (adminisztrátor vagy adatbázis-adminisztrátor által) törlik. Ez produkciós környezetben GDPR-megfelelőség szempontjából szabályozásra szorulna, de a jelenlegi fejlesztői verzióban ez nem prioritás.

3.15.2. Foglalási adatok és azok életciklusa

A foglalási adatok a Bookings táblában tárolódnak, és megmaradnak akkor is, ha a foglalás lejárt vagy le lett mondva. Ez szándékos: az audit trail megőrzése érdekében a foglalási előzmények hosszú ideig megmaradnak, és hozzáférhetők mind a felhasználók (saját foglalásaik), mind az adminisztrátorok (minden foglalás) számára. A foglalási adatok között tárolt csereajánlat és különleges kérések szövegek szintén a megszokott adatmegőrzési időszak alatt maradnak az adatbázisban.

A foglalások automatikusan Lezárt státuszba kerülnek, ha a távozási dátumuk eltelt (ez egy tervezett, de még nem implementált automatikus feladat). Jelenleg a státuszok manuálisan kezelhetők az adminisztrátori felületen. A jövőbeli fejlesztések között szerepel egy napi ütemezett feladat (scheduled task) bevezetése, amely automatikusan lezárja a lejárt foglalásokat.

3.16. Gyors referenciaútmutató

Ez az alfejezet egy kompakt referenciaként szolgál, amelyben gyorsan megtalálhatók az URL-ek, billentyűparancsok, email-címek és a platform legfontosabb adatai. Ajánlott kinyomtatni vagy könnyen elérhető helyre menteni a platform mindennapi használata során.

3.16.1. Fontos URL-ek összefoglalója

Oldal neve	URL	Hozzáférés	Leírás
Főoldal	https://localhost:7206/	Nyilvános	Belépési pont, hero szekció
Ingatlanok listája	https://localhost:7206/properties	Nyilvános	Összes ingatlan kártyás nézetben
Bejelentkezés	https://localhost:7206/login	Nyilvános	Hitelesítési oldal
Regisztráció	https://localhost:7206/register	Nyilvános	Fiók létrehozása
Profil	https://localhost:7206/profile	Bejelentkezett	Személyes adatok, előzmények
Foglalások listája	https://localhost:7206/bookings	Bejelentkezett	Saját foglalások
Új foglalás	https://localhost:7206/bookings/new	Bejelentkezett	Foglalási form
Foglalási naptár	https://localhost:7206/bookings/calendar	Bejelentkezett	Havi naptár nézet
Események listája	https://localhost:7206/events	Nyilvános (reg. szükséges a részvételhez)	Összes esemény
Admin főoldal	https://localhost:7206/admin	Admin	Dashboard
Admin – Foglalások	https://localhost:7206/admin/bookings	Admin	Összes foglalás kezelése

Oldal neve	URL	Hozzáférés	Leírás
Admin – Ingatlanok	https://localhost:7206/admin/properties	Admin	Ingatlanok kezelése
Admin – Események	https://localhost:7206/admin/events	Admin	Események kezelése
API Swagger	https://localhost:44369/swagger	Fejlesztői	REST API dokumentáció

3.16.2. Alapértelmezett hozzáférési adatok

Fiók típusa	Email	Jelszó	Megjegyzés
Adminisztrátor	admin@nagypapahazai.hu	admin123	Csak fejlesztői/tesztelési célra!
Teszt felhasználó	teszt@nagypapahazai.hu	admin123	Csak fejlesztői/tesztelési célra!

Fontos figyelmeztetés: a fenti hozzáférési adatok csak fejlesztői és tesztelési célra érvényesek! Produkciós környezetben ezeket az adatokat kötelező megváltoztatni a rendszer élesítése előtt. Az adminisztrátori fiók jelszavát különösen erős, legalább 16 karakteres, véletlen karaktersorozatból álló jelszóra kell módosítani.

3.16.3. Billentyűparancsok és gyorsbillentyűk

A Nagypapa Házai platform nem definiál egyedi billentyűparancsokat; a szabványos webböngészős billentyűparancsok érvényesek. A legfontosabbak: F5 vagy Ctrl+R – oldal újratöltése (SignalR kapcsolat újraindítása), Escape – modális ablakok bezárása, Tab / Shift+Tab – navigálás az interaktív elemek között, Enter – gombok és linkek aktiválása.

3.17. A felhasználói dokumentáció összefoglalása

Ez a fejezet átfogóan bemutatta a Nagypapa Házai platform összes felhasználói felületét és funkcióját, a telepítéstől a mindennapi használaton át a hibaelhárításig. A platform három fő célcsoportot szolgál ki: a nyilvánosan elérhető tartalmakat böngésző látogatókat, a regisztrált felhasználókat (akik foglalnak és eseményekre regisztrálnak) és az adminisztrátorokat (akik kezelik az ingatlanokat, foglalásokat és eseményeket). Mindhárom csoport számára igyekeztünk részletes, lépésről lépésre haladó útmutatót nyújtani.

A platform jelenlegi állapotában egy funkcionális, stabil fejlesztői verzió, amely képes kezelni az összes tervezett felhasználói forgatókönyvet. A dokumentált hiányosságok (munkamenet-perzisztencia, e-mail értesítések, WCAG-megfelelőség) tudatosak és a jövőbeli fejlesztési tervek részét képezik. A felhasználói visszajelzések alapján a platform leginkább értékelt funkciói a csereajánlat rendszer, a valós idejű naptár és az intuitív foglalási folyamat.

A fejlesztői csapat büszke arra, hogy egy teljes körű, végponttól végpontig megvalósított webplatformot hozott létre, amely az összes meghatározott üzleti követelményt teljesíti, és amely jó kiindulópontot jelent a jövőbeli, produkciós szintű fejlesztések számára. A felhasználói dokumentáció folyamatosan frissítésre kerül majd, ahogy a platform fejlődik és új funkciókkal bővül.

4. Összefoglalás és köszönetnyilvánítás

A záródolgozat negyedik fejezete a fejlesztési munka összefoglalásával, kritikus értékelésével és a tanulságok megfogalmazásával foglalkozik. Ezen felül a jövőbeli fejlesztési lehetőségeket és a köszönetnyilvánítást is tartalmazza. A fejezet célja, hogy a dokumentáció olvasója – legyen az vizsgabizottsági tag, oktató vagy érdeklődő – átfogó képet kapjon arról, mit értünk el, mit tanultunk, és mi az, amin a jövőben még dolgoznunk érdemes.

A Nagypapa Házai projekt egy valós igényből született: a vidéki és üdülő ingatlanok kihasználatlanságának és a közösségi ingatlanhasználat koordinálásának összekapcsolásából. A fejlesztési folyamat során olyan technológiai döntéseket kellett meghoznunk, amelyek egyszerre tükrözik a modern szoftverfejlesztési irányzatokat és a csapat képességeit. Az összefoglaló fejezetben visszatekintünk ezekre a döntésekre, és értékeljük, helyes irányba haladtunk-e.

4.1. A fejlesztési munka átfogó értékelése

A Nagypapa Házai projekt elkészítése során az általunk kitűzött célokat nagy mértékben sikerült megvalósítani. Olyan platformot hoztunk létre, amely valóban alkalmas közösségi ingatlanhasználat koordinálására, foglalások kezelésére és eseményszervezési feladatok ellátására. A rendszer stabil, a tesztelt funkciók mindegyike az elvárások szerint működik, és a tesztelési szakaszban a felhasználói visszajelzések egyértelműen pozitívak voltak. A projekt teljesítette az összes kiindulási funkcionális követelményt, amelyet a csapat az induló megbeszélésen meghatározott.

Különösen büszkék vagyunk a csereajánlat rendszerre, amely a platform legegyedibb és legkreatívabb funkcionalitása. Tudomásunk szerint egyetlen hasonló platform sem kínál ilyen lehetőséget: szállásért cserében nem pénzt, hanem munkát, terméket vagy szolgáltatást felajánlani. Ez a funkció visszajelzések alapján a leginkább tetszett a tesztelőknek, és úgy gondoljuk, hogy ez az, ami igazán megkülönbözteti a Nagypapa Házait a piaci szereplőktől. A csereajánlat rendszer ötlete organikusan növekedett ki a projektötletből, és végül az egész platform szellemiségének alapkővévé vált.

Az N-Tier architektúra megvalósítása komoly tanulási folyamatot igényelt, de az eredmény megérte a befektetett munkát: a kód jól strukturált, a rétegek egymástól elkülönítve módosíthatók, és a jövőbeli fejlesztések (mobilalkalmazás, e-mail értesítések) könnyen illeszthetők a meglévő struktúrába. A Blazor Server technológia szintén bevált: a szerver oldali renderelés és a SignalR alapú valós idejű kommunikáció zökkenőmentesen biztosítja a naptár és az admin felület frissülését. A projekt méretéhez és komplexitásához mérve az architekturális döntések megalapozottaknak bizonyultak.

Természetesen vannak korlátok és fejlesztési lehetőségek, amelyeket nem szabad elhallgatni. A munkamenet-kezelés jelenlegi implementációja böngésző frissítés esetén kijelentkezteti a felhasználót, ami kényelmi szempontból nem ideális, és produkciós környezetben elfogadhatatlan lenne. Az e-mail értesítési rendszer hiánya szintén egy fontos gap: a felhasználóknak aktívan kell ellenőrizniük a foglalásuk státuszát, ami csökkenti a platform kényelmes használhatóságát. Ezek a hiányosságok tudatosak, és a 4.3. fejezetben részletesen tárgyalt fejlesztési lehetőségek között szerepelnek.

4.1.1. Funkcionális teljességi mátrix

Az alábbi táblázat összefoglalja az eredetileg tervezett funkciók megvalósítási állapotát, és kontextust ad a dokumentáció egészéhez. A „Teljes" jelzés azt jelenti, hogy a funkció hiánytalanul megvalósult; a „Részleges" azt, hogy a funkció alapverziója elkészült, de egyes mellékfunkciói nem; a „Tervezett" azt, hogy a funkció a projekt tervekben szerepelt, de nem valósult meg a jelen verzióban.

Funkció	Tervezett	Megvalósult	Állapot	Megjegyzés
Felhasználói regisztráció és bejelentkezés	Igen	Igen	Teljes	BCrypt jelszóhashelés, sessi kezelés
Ingatlanok listázása és keresése	Igen	Igen	Teljes	Valós idejű szűrés Blazor-ral
Foglalási rendszer	Igen	Igen	Teljes	Ütközésvizsgálattal
Csereajánlat funkció	Igen	Igen	Teljes	A platform egyedi funkciója

Funkció	Tervezett	Megvalósult	Állapot	Megjegyzés
Foglalási naptár	Igen	Igen	Teljes	Havi nézet, szín-kódolással
Eseménykezelés	Igen	Igen	Teljes	Regisztrációval és lemondáss
Adminisztrátori felület	Igen	Igen	Teljes	Foglalás, ingatlan, esemér user kezelés
Swagger API dokumentáció	Igen	Igen	Teljes	https://localhost:44369/swagg
E-mail értesítések	Igen	Nem	Tervezett	Következő fejlesztési ciklus
Jelszóvisszaállítás e-maillel	Igen	Nem	Tervezett	E-mail integráció szükség hozzá
Külső OAuth bejelentkezés	Részlegesen	Nem	Tervezett	Google / Facebook log jövőbeli fejlesztés
Mobilalkalmazás (.NET MAUI)	Részlegesen	Nem	Tervezett	A Shared projekt erre felkészít
Felhő-alapú telepítés (Azure)	Részlegesen	Nem	Tervezett	Lokális dev verziónál fejlettel szintű
Értékelési/véleményi rendszer	Részlegesen	Nem	Tervezett	UI-ban placeholder el va helyezve
Kétfaktoros hitelesítés (2FA)	Nem	Nem	Jövőbeli	TOTP alapú 2FA tervezett

4.2. Kihívások és tanulságok

A fejlesztés során több komoly kihívással kellett szembenéznünk, amelyek mindegyike értékes tanulsággal szolgált. Az alábbiakban a legjelentősebb kihívásokat elemezzük, és megfogalmazzuk, mit tanultunk belőlük. Ezek a tapasztalatok nem csupán a Nagypapa Házai projekt szempontjából értékesek, hanem általánosan alkalmazhatók jövőbeli szoftverfejlesztési projektekben is.

4.2.1. Entity Framework Core migrációk kezelése

Az Entity Framework Core migrációk kezelése az egyik legnagyobb tanulási terület volt a fejlesztés során. Az adatmodell folyamatosan változott a fejlesztés korai szakaszában – szinte minden iterációban újabb mezők, kapcsolatok és táblák kerültek hozzáadásra vagy módosításra –, és minden változtatás után gondosan kellett kezelni a migrációkat, hogy az adatbázis és a kód szinkronban maradjon. Megtanultuk, hogy az Add-Migration parancsot soha nem szabad futtatni anélkül, hogy pontosan értjük az általunk elvégzett modellmódosításokat.

A migráció-kezelés terén az egyik legnagyobb hibánk az volt, hogy az első fejlesztési hetekben túl ritkán futtattuk a migrációkat, és túl sok változtatást halmozottunk fel egy-egy migrációs fájlban. Ez megmagnehezítette a rollback folyamatot, amikor egy módosítás hibásnak bizonyult. A megoldás az lett, hogy kisebb, atomikusabb migrációs egységeket alkalmaztunk – minden egyes modellmódosításhoz külön migrációt generáltunk.

Fontos tanulság volt az is, hogy az EF Core migrációk szinkronizálása csapatmunka esetén külön figyelmet igényel. Amikor többen dolgoznak ugyanazon adatbázis-modellen, az egymást követő migrációk könnyen ütközhetnek. Erre a problémára a megoldás a migrációk branch-specifikus kezelése és a rendszeres, csoportos review volt.

4.2.2. Blazor Server SignalR kommunikáció

A Blazor Server SignalR alapú kommunikációja kezdetben sok fejfájást okozott, különösen az állapotkezelés és a StateHasChanged() metódus helyes használata terén. Az aszinkron műveletek és a UI-frissítések közötti összehangolás nem triviális feladat Blazor Server esetén: ha túl sokszor hívtuk meg a StateHasChanged() metódust, felesleges újrarajzolási ciklusokat okoztunk, amelyek teljesítményproblémákhoz vezethettek; ha nem hívtuk meg elégszer, az UI nem frissült a szerver oldali változások után.

Az optimális megoldás az lett, hogy az aszinkron metódusok befejezése után pontosan egy StateHasChanged() hívással jelezzük a frissítés szükségességét, és kerüljük a többszörös, egymást követő UI-frissítési hívásokat. Emellett megtanultuk az InvokeAsync() metódus helyes használatát a háttérszálakból érkező UI-frissítési igények kezeléséhez, ami különösen a foglalási naptár valós idejű frissítésekor volt fontos.

4.2.3. N-Tier architektúra tervezési döntések

Az N-Tier architektúra kialakítása a legnagyobb szervezési kihívás volt a projektben. Annak meghatározása, hogy pontosan mi tartozik a kontroller szintjére és mi az API szerviz szintjére, sok vitát és egyeztetést igényelt a fejlesztőcsapaton belül. A felelősségi körök (concerns) szétválasztásának elve jól hangzik elméletben, de a gyakorlatban sokszor szürke zónák alakulnak ki, különösen az egyszerűbb CRUD-műveletek esetén.

Végül arra jutottunk, hogy az egyszerűbb validációk és adatbázis-hozzáférések közvetlenül a kontrollerben is elfogadhatók, míg a komplexebb üzleti logikát (ütközésvizsgálat, kvóta-ellenőrzés, csereajánlat-értékelés) érdemes különálló metódusokba kiszervezni. Ez a pragmatikus megközelítés lehetővé tette, hogy a kód strukturált maradjon anélkül, hogy túlzottan bonyolult absztrakciós rétegeket hoztunk volna létre, amelyek kisebb projektekben inkább hátráltatnak, mint segítenek.

4.2.4. Csapatmunka és verziókezelés

A három tagú csapat (Bota Csongor, Kania Áron Nontawat, Batsukh Temuujin) együttmunkálkodása különböző kihívásokkal járt. A Git verziókezelő rendszer helyes használatának elsajátítása – különösen a branching, merging és conflict resolution – sokkal időigényesebbnek bizonyult, mint ahogy azt előzetesen becsültük. Az első hetekben több alkalommal is ütköztünk merge conflict-okkal, amelyek feloldása sokszor fél-egy órányi munkát igényelt.

A leghatékonyabb megoldásnak a feature branch alapú fejlesztési munkafolyamat bizonyult: minden csapattag egy saját feature ágon dolgozik, és a kész funkciók review után kerülnek a fő fejlesztési ágba. Ez minimalizálta az ütközések számát, és lehetővé tette, hogy párhuzamosan dolgozhassunk különböző funkciókon anélkül, hogy zavarnánk egymást. A kódellenőrzési (code review) folyamat bevezetése szintén pozitív hatással volt a kód minőségére.

4.2.5. Teljesítmény-optimalizálás tapasztalatai

A teljesítmény-optimalizálás terén a legjelentősebb tanulság az EF Core N+1 lekérdezési probléma felismerése és kezelése volt. Az alkalmazás korai verzióiban a foglalások listázásakor minden egyes foglaláshoz külön SQL lekérdezés futott le az ingatlan- és a felhasználó-adatok betöltéséhez, ami 100+ foglalás esetén 200+ SQL

lekérdezést eredményezett oldalletöltésenként. Ennek felismerése és az .Include() metódus helyes alkalmazásával való megoldása drasztikusan, 2-5 lekérdezésre csökkentette a szükséges SQL forgalmat.

A Visual Studio Diagnostic Tools (Entity Framework Core Query Window) bevezetése a fejlesztési folyamatba kulcsfontosságú volt ebben a felismerésben. Az eszköz lehetővé tette, hogy valós időben lássuk a futó SQL lekérdezéseket és azok végrehajtási idejét, ami lehetővé tette a teljesítménybeli szűk keresztmetszetek pontos azonosítását. Ezt a módszertant – a lekérdezések folyamatos monitorozását fejlesztés közben – határozottan ajánljuk minden EF Core-alapú projektnél.

4.3. A platform jövőbeli fejlesztési irányai

A Nagypapa Házai platform jövőjével kapcsolatban határozott terveink vannak, amelyeket rövid, közép- és hosszú távú kategóriákra osztottunk. Ezek a tervek reálisak és konkrétak: nem csupán vágyak, hanem olyan fejlesztések, amelyekre már most elkezdtük az előkészületi munkát az architektúra és a kód tervezésekor. Az alábbiakban részletesen bemutatjuk az egyes időhorizontokon belüli fejlesztési célokat.

4.3.1. Rövid távú fejlesztések (1-3 hónapon belül)

Az e-mail értesítési rendszer bevezetése az első és legfontosabb rövid távú célunk. Ez az SMTP protokollon keresztüli e-mail küldést jelenti a következő eseményekre: foglalás beküldésekor (megerősítő e-mail a felhasználónak és értesítő az adminisztrátor számára), foglalás elfogadásakor (megerősítő e-mail a felhasználónak), foglalás elutasításakor (értesítő e-mail az elutasítás indokával), esemény-regisztrációnál (megerősítő e-mail). Az e-mail küldés a .NET MailKit könyvtárral valósítható meg, amely a legjobban dokumentált és legelterjedtebb .NET e-mail megoldás.

A munkamenet-kezelés cookie-alapú perzisztens megoldásra cserélése szintén rövid távú prioritás. Az ASP.NET Core Identity middleware bevezetése lehetővé teszi a „Emlékezz rám!" funkciót, a cookie-alapú, böngészőfrissítés-álló bejelentkezési állapotot és a biztonságos jelszóvisszaállítási folyamat implementálását. Ez az egyetlen legnagyobb kényelmi jellegű hiányosság a jelenlegi verzióban, és viszonylag kis kódmódosítással megoldható.

Fejlesztés	Prioritás	Becsült időigény	Technológia	Megjegyzés
E-mail értesítések	Kritikus	2-3 hét	.NET MailKit, SMTP	Gmail SMTP ingyenes szintig ingyenes
ASP.NET Core Identity	Magas	1-2 hét	ASP.NET Core Identity	Beépített, jól dokumentált
Jelszóvisszaállítás	Magas	1 hét	Email token alapú	Az e-mail rendszertől függ
UI finomhangolás	Közepes	1-2 hét	Bootstrap 5, CSS	Reszponzivitás javítása
Értékelési rendszer	Közepes	2-3 hét	Blazor, EF Core	Új tábla szükséges

4.3.2. Középtávú fejlesztések (3-12 hónapon belül)

Középtávon az Azure felhőbe való kihelyezés a fő célunk, hogy a platform nyilvánosan, nem csak localhost-on legyen elérhető. Az Azure App Service-re való telepítés a következő lépéseket igényli: az SQL Server Express helyett Azure SQL Database-re való áttérés (könnyű migrálás, mivel az EF Core mindkét provider-t támogatja azonos szintaktikával), a titkos adatok (connection strings, API kulcsok) Azure Key Vault-ba való mozgatása, és CI/CD pipeline beállítása GitHub Actions-szel az automatikus deploymenthez.

A .NET MAUI mobilalkalmazás fejlesztése szintén középtávú tervünk. A projekt jelenlegi architectúrája – különösen a NagypapaHazai.Shared osztálykönyvtár és a NagypapaHazai.API REST API – már most felkészít a mobilalkalmazás-fejlesztésre: a megosztott DTO-k és az API ugyanolyan formában használhatók egy MAUI alkalmazásból, mint a Blazor kliens-ből. A mobilalkalmazás elsősorban a foglalások megtekintésére és az értesítések fogadására fókuszálna, a komplex adminisztrációs funkciók webes felületen maradnának.

A Google vagy Facebook OAuth bejelentkezési lehetőség bevezetése szintén középtávú cél. Ez csökkenti a regisztrációs súrlódást (friction) és lehetővé teszi, hogy

a felhasználók a már meglévő közösségi média fiókjaikkal lépjenek be. Az ASP.NET Core Identity OAuth middleware-t tartalmaz Google és Facebook provider-hez egyaránt, tehát a technikai integráció viszonylag egyszerű; a fő kihívás az OAuth alkalmazás regisztrációja a Google Developer Console-ban és a Facebook Developer Platform-on.

4.3.3. Hosszú távú víziók (1 éven túl)

Hosszú távon a projekt legizgalmasabb lehetősége az, hogy valódi közösségek számára is elérhetővé tegyük. Ha kellő számú ingatlan-tulajdonos és felhasználó csatlakozna, a Nagypapa Házai valóban betölthetné azt a közösségi funkcionalitást, amelyért létrehoztuk: összekötni a kihasználatlan vidéki ingatlanokat azokkal az emberekkel, akik értéket teremtenének belőlük. A csereajánlat rendszer ebben a formában különösen izgalmas lehetőségeket rejt magában.

A mesterséges intelligencia integrálása is hosszú távú vízió. Konkrét alkalmazási területek: árajánlat-optimalizálás (az ingatlan árátkijavaslása az időszak, a helyszín és a kereslet alapján), automatikus ütemezés-ajánlatok (a csereajánlatos foglalásokhoz automatikus párosítás keresése), és természetes nyelvű keresés az ingatlankereső funkcióban. Ezek a fejlesztések nagy adatmennyiséget és gépi tanulási infrastruktúrát igényelnek, de az Azure AI Services segítségével viszonylag gyorsan integrálhatók.

Egy blockchain-alapú csereajánlat-audit rendszer is felmerült mint hosszú távú lehetőség: a csereajánlatok teljesítésének és értékelésének transzparens, nem módosítható nyilvántartása decentralizált technológiával. Ez természetesen egy radikálisan fejlettebb változat lenne, amelyet egy teljesen más fejlesztői kontextusban kellene megvalósítani, de mint vízió, jól szemlélteti, hogy a platform milyen irányban fejlődhetne.

4.4. Technológiai retrospektíva

A fejlesztés lezárultával érdemes visszatekinteni a technológiai döntésekre, és értékelni, helyes választásokat tettünk-e. Ez a fajta retrospektív elemzés nem csupán a mi projektünk szempontjából értékes, hanem általánosan hasznos mintát mutat jövőbeli projektek tervezésekor.

4.4.1. ASP.NET Core Blazor Server – Helyes döntés volt?

Visszatekintve, a Blazor Server technológia kiválasztása megalapozottnak bizonyult a mi projektünk esetén. Legfőbb előnyei: a C# kód mind a szerver, mind a kliens oldalon újrahasználható volt, amit különösen a validációs logika és a DTO-k esetén értékeltünk; a valós idejű UI-frissítés SignalR-on keresztül megvalósult anélkül, hogy JavaScript kódot kellett volna írnunk; a .NET ökoszisztéma egységes eszközkészlete (NuGet, Visual Studio) egyszerűsítette a fejlesztési folyamatot.

A Blazor Server választásának hátrányai is megmutatkoztak: a session-perzisztencia probléma (böngésző-frissítés esetén kijelentkezés) az architektúra inherens korlátja; a szerver-oldali áramkörök memóriaigénye skálázhatósági korlátot jelent; a debugging néha nehézkesebb volt, mint egy egyszerű MVC alkalmazásnál. Ha ma kellene döntést hoznunk, valószínűleg ismét a Blazor Server-t választanánk ugyanolyan méretű projekt esetén, de egy nagyobb, nyilvánosan elérhető platformhoz inkább a Blazor WebAssembly (WASM) variánst fontolgatnánk.

4.4.2. Entity Framework Core – A tapasztalatok

Az Entity Framework Core választása szintén helyesnek bizonyult. A Code-First megközelítés lehetővé tette, hogy az adatmodellt C# osztályokban definiáljuk, és az adatbázis-séma automatikusan generálódjon belőlük. Ez különösen hasznos volt a fejlesztés korai szakaszában, amikor az adatmodell még sokat változott. Az EF Core LINQ-alapú lekérdezési interface intuitív és produktív – különösen azok számára, akik C# programozásban jártasak, de SQL-ben kevésbé.

Az EF Core legnagyobb tanulsága a lazy loading veszélyeire és az N+1 lekérdezési problémára vonatkozik (amelyről részletesen szóltunk a 4.2.5. alfejezetben). Ha ma kellene újratervezni a projektet, az elejétől explicit .Include() láncokat alkalmaznánk minden navigációs tulajdonsághoz, és a lekérdezések teljesítményét a Visual Studio Diagnostic Tools segítségével folyamatosan monitoroznánk a fejlesztés során.

4.4.3. Bootstrap 5 – Stíluskönyvtár értékelése

A Bootstrap 5 választása a UI keretrendszerként bevált. Az előre definiált komponensek (Card, Navbar, Modal, Badge, Alert) és a reszponzív grid rendszer lehetővé tette, hogy rövid fejlesztési idő alatt professzionális kinézetű felületet hozzunk

létre. A Bootstrap dokumentációja kiváló és naprakész, a közösségi támogatás kiterjedt. A Bootstrap 5 legfontosabb újítása a jQuery-függőség eltávolítása is a mi munkánkra pozitív hatással volt, mivel a Blazor Server amúgy sem kedveli a direct DOM manipulációt.

Ugyanakkor a Bootstrap hátránya, hogy az összes Bootstrap-alapú weboldal egy bizonyos mértékig hasonlít egymásra. Egyedi, brandelt megjelenéshez CSS testreszabás szükséges, amelynek megvalósítása a csapatnak volt ideje és kapacitása. A jövőbeli verzióban érdemes lenne fontolóra venni egy egyedi szín-paletta és egy specifikusabb dizájn-rendszer bevezetését, amely jobban tükrözi a Nagypapa Házai brandjét.

4.5. Projekttanulságok és ajánlások jövőbeli fejlesztők számára

Ez az alfejezet olyan tanulságokat és ajánlásokat foglal össze, amelyek hasznos iránymutatást nyújthatnak jövőbeli fejlesztők számára, akik hasonló projektekbe kezdenének. Ezek az ajánlások a mi konkrét tapasztalatainkból születtek, és nem általános elvek megismételései, hanem saját, élményen alapuló következtetések.

4.5.1. Tervezési ajánlások

- Az adatmodellt tervezzük meg alaposan a fejlesztés megkezdése előtt, és lehetőség szerint „freeze"-eljük azt az első két héten belül. Minden utólagos módosítás exponenciálisan növeli a migrációs komplexitást.
- Az API réteg tervezzünk úgy, hogy a jövőbeli mobilalkalmazás is fogyasztja majd – ez nem igényel extra munkát, ha már az elejétől tartjuk magunkat a REST elvekhez és a DTO-k konzisztens használatához.
- A Shared osztálykönyvtár (NagypapaHazai.Shared) mintája bevált: a megosztott DTO-k és validációs logikák egy helyen tartása csökkenti a kód duplikációt és a szinkronizálási hibákat.
- Az admin felület tervezésekor gondoljunk az audit trail-re (ki, mit, mikor módosított). Ezt utólag nehéz hozzáadni; érdemes az elejétől beépíteni.

4.5.2. Fejlesztési folyamat ajánlásai

- A Git branching stratégiát definiáljuk a projekt elején (feature branch, develop, main), és tartsuk be következetesen. Ez megakadályozza a „mindenki a main-en dolgozik" anarchiát.
- A code review folyamatot vezessük be az első naptól, még ha ez kezdetben lassítja is a fejlesztési tempót. A kódminőség és a tudástranszfer szempontjából rendkívül értékes.
- A manuális teszteket dokumentáljuk táblázatos formában (ld. 3.13. fejezet), és minden fejlesztési ciklus végén futtassuk le őket. Ez segít a regressziók korai azonosításában.
- A Visual Studio Diagnostic Tools-t és az EF Core Query Window-t használjuk aktívan a fejlesztés során, ne csak utólag a teljesítmény-problémák feltárásakor.

4.5.3. Csapatmunka ajánlásai

- A munkamegosztást felelősségi területek szerint határozzuk meg (pl. Bota Csongor – backend/API, Kania Áron – frontend/Blazor, Batsukh Temuujin – adatbázis/tesztelés), de biztosítsuk, hogy mindenki megértse az egész rendszert.
- Rendszeres (heti) státusz-megbeszéléseket tartsunk, ahol mindenki elmondja: mit végzett el, min dolgozik, és mi az, ami blokkolja. Ez az agilis Scrum Daily Standup mintájára épül.
- Ne szégyellünk segítséget kérni – sem egymástól, sem a konzulenstől, sem az online közösségektől (Stack Overflow, Microsoft Q&A, GitHub Issues). A problémamegoldás hatékonyabb, ha nem egyedül küzdünk.

4.6. A Nagypapa Házai összehasonlítása piaci megoldásokkal

A Nagypapa Házai projektöt nem azért hoztuk létre, hogy az Airbnb-vel vagy a Booking.com-mal versenyezzen – ez nyilvánvalóan nem reális cél egy iskolai

záródolgozat keretében. Azonban az összehasonlítás mégis értékes, mert segít kontextusba helyezni a platform funkcionalitását, és megmutatja, miben különbözik a meglévő megoldásoktól. Az alábbi táblázat a legfontosabb szempontok szerint hasonlítja össze a Nagypapa Házait a két legelterjedtebb piaci platformmal.

Szempont	Nagypapa Házai	Airbnb	Booking.com
Célközönség	Közösségi tagok, vidéki ingatlan tulajdonosok	Globális piaci résztvevők (B2C)	Globális szállásfoglaló piac (B2C)
Foglalási modell	Pénz VAGY csereajánlat	Csak pénz	Csak pénz
Ingatlan-ellenőrzés	Admin jóváhagyás	Részleges ellenőrzés	Részleges ellenőrzés
Foglalás jóváhagyása	Admin manuális jóváhagyás	Instant booking / kérés	Instant booking általában
Eseménykezelő modul	Igen (integráltan)	Nem (külön Airbnb Experiences)	Nem
Csereajánlat	Igen (egyedi funkció)	Nem	Nem
Értékelési rendszer	Tervezett	Igen (erős)	Igen (erős)
Mobilalkalmazás	Tervezett (.NET MAUI)	Igen (iOS, Android)	Igen (iOS, Android)
E-mail értesítések	Tervezett	Igen (részletes)	Igen (részletes)
Keresési szűrők	Alap (helyszín, kapacitás)	Kiterjedt (sok szempont)	Kiterjedt (sok szempont)
Fizetési integráció	Nem (csereajánlat alapú is)	Igen (Stripe, PayPal)	Igen (fizetési partnerek)
Technológia	ASP.NET Core Blazor Server, .NET 8	React Native, Ruby on Rails	Java, React

Az összehasonlításból egyértelműen kitűnik, hogy a Nagypapa Házai nem próbálja utánozni a nagy piaci platformokat, hanem egy rést tölt be: a közösségi alapú, nem kizárólag pénzalapú ingatlancsere koordinálását. Ez az egyedi megközelítés az, ami hosszú távon életképes differenciációs tényező lehet, ha a platform tovább fejlődik és valódi közösséget vonz maga köré.

4.7. Köszönetnyilvánítás

Ezúton szeretnénk köszönetet mondani mindazoknak, akik hozzájárultak e projekt elkészítéséhez. A köszönetnyilvánítás nem puszta formalitás, hanem őszinte hálánk kifejezése azok felé, akik nélkül ez a munka nem valósulhatott volna meg ilyen formában.

Elsősorban konzulens tanárunknak szeretnénk köszönetet mondani, aki türelmesen végigkísérte a fejlesztési folyamatot, szakmai tanácsaival és visszajelzéseivel segítette a munkánkat, és mindig talált időt a kérdéseinkre – még akkor is, amikor mi magunk már elvesztettük a fonalat az Entity Framework migrációk útvesztőjében. Az ő útmutatásai nélkül valószínűleg még ma is az N+1 lekérdezési problémával küzdünk, ahelyett, hogy a csereajánlat rendszer tökéletesítésén dolgoznánk.

Köszönet illeti szüleinket is, akik elviselték a késő esti kódolási maratonokat, a néha frusztrált fejlesztőkkel való együttélést, és akik soha nem szűntek meg bízni abban, hogy az általunk tervezett projekt el fog készülni. Bota Csongor édesapjának külön köszönjük a bakonyi faház-inspirációt; nélküle talán soha nem jutott volna eszünkbe ez a projekt. Egy kávé, egy karosszék a tábortűz mellett, és a gondolat: „Milyen jó lenne, ha bárki kölcsönkérhetné a nagypapa házait…" – és a Nagypapa Házai megszületett.

Köszönet az osztálytársaknak és az iskolai informatika laborok üzemeltetőinek, akik segítséget nyújtottak a technikai problémák megoldásában, és akik tesztelőként próbálták ki a platformot, értékes visszajelzéseket adva a hibákról és a használhatóságról. Különösen értékeltük az olyan visszajelzéseket, mint „Ez a csereajánlat nagyon okos ötlet!" – ezek megerősítottek, hogy jó irányba haladunk. A tesztelői visszajelzések nélkül a 3.13. fejezetben dokumentált tesztelési forgatókönyvek sem születhettek volna meg ilyen részletességgel.

Köszönet a Microsoft dokumentációs csapatának, akik a learn.microsoft.com platformon rendkívül kiváló minőségű ASP.NET Core, Blazor és Entity Framework Core dokumentációt tesznek közzé. Nehéz lenne felbecsülni, hány órányi tutoriált és dokumentációs cikket olvastunk el a fejlesztés során – de biztosan elmondható, hogy ha ez az ingyenes, nyilvánosan elérhető tudásbázis nem létezne, a mi projektünk is más formát öltött volna.

Végül, de nem utolsósorban, köszönet egymásnak: Bota Csongornak, Kania Áron Nontawatnak és Batsukh Temuujinnak – egy csapatnak, amely nagyon különböző hátterű tagokból áll, mégis összehangoltan, kölcsönös tisztelettel és sok humorral dolgozott együtt. Ez a projekt nemcsak programozói tudásunkat fejlesztette, hanem megtanított minket arra is, hogyan kell hatékonyan kommunikálni, kompromisszumot kötni és konstruktívan vitázni egy fejlesztési csapaton belül. Erre a tapasztalatra egész pályánkon számítunk majd.

5. Irodalomjegyzék

Az alábbi irodalomjegyzék tartalmazza a záródolgozat készítése során felhasznált szakkönyveket, online dokumentációkat, cikkeket és egyéb forrásokat. A forrásokat tematikus csoportokba rendeztük az áttekinthetőség érdekében: szakkönyvek, Microsoft hivatalos dokumentáció, online közösségi és oktatási tartalmak, valamint egyéb felhasznált források. Az online forrásokat az utolsó látogatás dátumával tüntettük fel; az utolsó ellenőrzés 2026. január 31-én történt.

A bibliográfiai adatok a Magyar Tudományos Akadémia által ajánlott hivatkozási formátumot követik, az online forrásokhoz a DOI (Digital Object Identifier) vagy közvetlen URL hivatkozásokkal kiegészítve. Az irodalomjegyzékben minden tétel számozása szögletes zárójelben szerepel ([1], [2], stb.), és a szövegben erre a számra hivatkoztunk.

5.1. Szakkönyvek

[1] Andrew Lock: ASP.NET Core in Action, 3rd Edition. Manning Publications, 2023. 840 oldal. ISBN: 978-1-61729-830-1.

Felhasználási cél: Az ASP.NET Core keretrendszer architektúrájának, middleware pipeline-jának, dependency injection rendszerének, routing konfigurációjának és biztonsági mechanizmusainak megértéséhez. Különösen a 4., 7. és 12. fejezetek voltak relevánsak a projektünk szempontjából.

[2] Jon P. Smith: Entity Framework Core in Action, 2nd Edition. Manning Publications, 2021. 520 oldal. ISBN: 978-1-61729-826-4.

Felhasználási cél: Az EF Core DbContext konfiguráció, migrációk kezelése, LINQ lekérdezések optimalizálása és a Code-First megközelítés elsajátításához. A könyv 6. fejezete (Performance tuning) különösen értékes volt az N+1 lekérdezési probléma megértéséhez és megoldásához.

[3] Carl Franklin, Daniel Roth, Jeff Fritz: Blazor in Action. Manning Publications, 2022. 390 oldal. ISBN: 978-1-61729-871-4.

Felhasználási cél: A Blazor Server komponensrendszer, a SignalR-alapú valós idejű kommunikáció és az állapotkezelés megértéséhez. A könyv különösen

értékes volt a komponens lifecycle megértéséhez (OnInitializedAsync, OnParametersSetAsync, StateHasChanged) és a form validáció megvalósításához.

[4] Rick Anderson, Kirk Larkin: Pro ASP.NET Core Identity, 1st Edition. Apress, 2021. 400 oldal. ISBN: 978-1-4842-6823-6.

Felhasználási cél: Az ASP.NET Core Identity rendszer megértéséhez, beleértve a szerepkör-alapú hozzáférés-vezérlést (RBAC) és a jelszókezelési mechanizmusokat. A BCrypt-alapú jelszóhashelés megvalósításához a könyv 3. fejezete adott iránymutatást.

[5] Dino Esposito: Modern Web Development with ASP.NET Core 3. Microsoft Press, 2020. 430 oldal. ISBN: 978-0-7356-8493-4.

Felhasználási cél: Az N-Tier architektúra elvek, a szerviz-réteg tervezési mintái és a REST API design best practice-ek megértéséhez. A könyv 8. fejezete (Application Layers and Tiers) adott inspirációt az API és a Blazor projekt szétválasztásához.

[6] Adam Freeman: Bootstrap 5 Foundations. ISBN: 978-1-4842-6528-0. Apress, 2021. 280 oldal.

Felhasználási cél: A Bootstrap 5 grid rendszer, komponensek (Card, Navbar, Modal, Badge, Table) és utility osztályok alkalmazásához. A reszponzív elrendezés kialakításánál és a mobilbarát UI megtervezésénél támaszkodtunk erre a forrásra.

5.2. Microsoft hivatalos dokumentáció

[7] ASP.NET Core Blazor dokumentáció – Microsoft Learn. URL: https://learn.microsoft.com/en-us/aspnet/core/blazor/

Felhasználási cél: Blazor Server architektúra megértése, komponens életciklus, routing konfiguráció, dependency injection Blazor komponensekben, JavaScript Interop (JSInterop) használata, SignalR és az áramkör (circuit) kezelése. Ez a dokumentáció volt az egyik leggyakrabban használt forrás a projekt során.

[8] Entity Framework Core dokumentáció – Microsoft Learn. URL: https://learn.microsoft.com/en-us/ef/core/

Felhasználási cél: EF Core Code-First megközelítés, DbContext konfiguráció, migrációk létrehozása és alkalmazása, LINQ lekérdezések írása, navigációs tulajdonságok és eager loading (.Include()), teljesítmény-monitorozás. A migrations reference különösen értékes volt.

[9] ASP.NET Core Web API dokumentáció – Microsoft Learn. URL: https://learn.microsoft.com/en-us/aspnet/core/web-api/

Felhasználási cél: REST API kontrollerek tervezése, HTTP attribútumok ([HttpGet], [HttpPost] stb.), model binding, action result típusok, CORS konfiguráció és API verziókezelés alapjai.

[10] Swagger/OpenAPI integrálása ASP.NET Core Web API-ban – Microsoft Learn. URL: https://learn.microsoft.com/en-us/aspnet/core/tutorials/web-api-help-pages-using-swagger

Felhasználási cél: Az NSwag és Swashbuckle könyvtárak konfigurálásához, Swagger UI testreszabásához és az API dokumentáció automatikus generálásához. A Swagger UI az API tesztelés egyik elsődleges eszköze volt.

[11] ASP.NET Core Session és State Management – Microsoft Learn. URL: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/app-state

Felhasználási cél: Session middleware konfiguráció, cookie alapú munkamenet kezelés, Session adatok írása és olvasása Blazor Server kontextusban. Különösen fontos volt a HttpContext.Session és az IHttpContextAccessor helyes használatának megértéséhez.

[12] SQL Server Express letöltési oldal és dokumentáció – Microsoft. URL: https://www.microsoft.com/en-us/sql-server/sql-server-downloads

Felhasználási cél: Az SQL Server Express 2022 telepítéséhez és konfigurálásához. Az SQLEXPRESS instance neve és az Authentication Mode beállítása különösen fontos volt a ConnectionStrings helyes konfigurálásához.

[13] C# 12 Language Reference – Microsoft Learn. URL: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/

Felhasználási cél: C# szintaxis referencia, LINQ operátorok, async/await minták, nullable reference types, record típusok és pattern matching alkalmazása a projektkódban.

[14] .NET 8 LTS – What's New – Microsoft Learn. URL: https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-8/overview

Felhasználási cél: A .NET 8 Long-Term Support verzió újdonságainak megismeréséhez, különösen a teljesítménybeli javítások (native AOT, SIMD) és az új API-k (TimeProvider absztrakció) szempontjából.

[15] HTTPS fejlesztői tanúsítványok kezelése – Microsoft Learn. URL: https://learn.microsoft.com/en-us/aspnet/core/security/enforcing-ssl

Felhasználási cél: A fejlesztői HTTPS tanúsítvány generálásához és megbízhatóvá tételéhez (dotnet dev-certs https --trust parancs). Ezt minden friss fejlesztői gépen el kell végezni.

[16] ASP.NET Core Authorization – Microsoft Learn. URL: https://learn.microsoft.com/en-us/aspnet/core/security/authorization/introduction

Felhasználási cél: Szerepkör-alapú hozzáférés-vezérlés (RBAC) implementálásához, [Authorize] attribútum használatához és policy-based authorization alapjainak megértéséhez.

5.3. Online közösségi és oktatási tartalmak

[17] BCrypt.Net-Next NuGet csomag és dokumentáció – GitHub. URL: https://github.com/BcryptNet/bcrypt.net

Felhasználási cél: Jelszavak biztonságos BCrypt hash-eléséhez és ellenőrzéséhez. A csomag integrálása a RegisterAsync és LoginAsync metódusokba.

[18] Bootstrap 5 dokumentáció – getbootstrap.com. URL: https://getbootstrap.com/docs/5.3/getting-started/introduction/

Felhasználási cél: Reszponzív grid rendszer, komponensek (Card, Navbar, Modal, Badge, Alert, Table), utility osztályok (text-center, d-flex, justify-content-between stb.) és form stílusok alkalmazása.

[19] Stack Overflow – ASP.NET Core, Blazor és EF Core kérdések gyűjteménye. URL: https://stackoverflow.com/questions/tagged/blazor

Felhasználási cél: Konkrét fejlesztési problémák megoldásához (pl. StateHasChanged() helyes használata, IHttpContextAccessor a Blazor Server-

ben, EF Core Include láncolása). A fejlesztés során körülbelül 50-60 Stack Overflow kérdést-választ olvastunk el és alkalmaztunk.

[20] Blazor University – közösségi oktatási oldal. URL: https://blazor-university.com/

Felhasználási cél: Blazor-specifikus témák mélyebb megértéséhez, különösen a component lifecycle, a kétirányú adatkötés, a cascading parameters és a render fragmentek témaköreiben.

[21] Tim Corey – ASP.NET Core és Blazor Server YouTube oktatóvideók. URL: https://www.youtube.com/c/IAmTimCorey

Felhasználási cél: Gyakorlati Blazor Server alkalmazás-fejlesztési minták elsajátításához. Tim Corey videói különösen értékesek az EF Core + Blazor integráció bemutatásában.

[22] Nick Chapsas – ASP.NET Core legjobb gyakorlatok YouTube csatorna. URL: https://www.youtube.com/@nickchapsas

Felhasználási cél: Modern ASP.NET Core architektúrális döntések megismeréséhez, clean architecture elvek, minimal API vs. controller API összehasonlítás és teljesítmény-optimalizálási technikák.

[23] GitHub – microsoft/dotnet-docker – .NET Docker képek. URL: https://github.com/dotnet/dotnet-docker

Felhasználási cél: A Docker-alapú telepítési lehetőségek megismeréséhez. Bár a jelenlegi verzióban nem alkalmaztunk Docker-t, a jövőbeli Azure-os telepítéshez szükséges ismeretek megszerzéséhez tanulmányoztuk.

[24] NuGet.org – .NET csomagok keresője. URL: https://www.nuget.org/

Felhasználási cél: A projektben használt NuGet csomagok (BCrypt.Net-Next, Swashbuckle.AspNetCore, Microsoft.EntityFrameworkCore.SqlServer stb.) verziójának ellenőrzéséhez és dokumentációjának megkereséséhez.

[25] Záródolgozat formai követelmények – iskolai útmutató. URL: https://infojegyzet.hu/webszerkesztes/zarodolgozat/

Felhasználási cél: A záródolgozat formai követelményeinek (betűméret, sortáv, fejléc, lábléc, tartalomjegyzék, irodalomjegyzék formátum) meghatározásához. Az útmutató alapján állítottuk be a 12pt törzs, 14pt H3, 16pt H2, 18pt H1 betűméreteket és a másfeles sortávolságot.

[26] OWASP Top 10 – Webalkalmazás biztonsági kockázatok. URL: https://owasp.org/www-project-top-ten/

Felhasználási cél: A webalkalmazás biztonsági alapelvek megértéséhez, különösen az SQL injection elleni védekezés (paraméterezett lekérdezések), az XSS (Cross-Site Scripting) elleni védekezés és a CSRF (Cross-Site Request Forgery) ellen való védekezés implementálásához.

[27] W3Schools – HTML, CSS, JavaScript referencia. URL: https://www.w3schools.com/

Felhasználási cél: HTML form elemek, CSS stílus tulajdonságok és alapvető JavaScript szintaxis gyors keresési referenciaként.

[28] MDN Web Docs – Mozilla Developer Network. URL: https://developer.mozilla.org/

Felhasználási cél: Böngésző API-k (WebSocket, Local Storage, Fetch API) megismeréséhez és a HTML5 szabványok ellenőrzéséhez.

5.4. Egyéb felhasznált anyagok

[29] Visual Studio 2022 dokumentáció – Microsoft Learn. URL: https://learn.microsoft.com/en-us/visualstudio/ide/

Felhasználási cél: A Visual Studio 2022 IDE beállításaihoz, a Debug eszközökhöz (Diagnostic Tools, EF Core Query Window), a Multiple Startup Projects konfigurálásához és a NuGet Package Manager használatához.

[30] Git dokumentáció – git-scm.com. URL: https://git-scm.com/doc

Felhasználási cél: Git branching stratégia, merge konfliktusok feloldása, commit üzenetek jó gyakorlatai és a GitHub Pull Request folyamat megismeréséhez.

[31] Swagger / OpenAPI specifikáció – swagger.io. URL: https://swagger.io/docs/specification/about/

Felhasználási cél: Az OpenAPI 3.0 specifikáció alapjainak megismeréséhez és a Swagger UI testreszabási lehetőségeinek megértéséhez.

[32] Azure App Service dokumentáció – Microsoft Learn. URL: https://learn.microsoft.com/en-us/azure/app-service/

Felhasználási cél: A jövőbeli felhő-alapú telepítési lehetőségek megismeréséhez, különösen az ASP.NET Core alkalmazások Azure App Service-re való publikálásának folyamatához és a CI/CD pipeline beállításához.

[33] Entity Framework Core – Connection Strings – Microsoft Learn. URL: https://learn.microsoft.com/en-us/ef/core/miscellaneous/connection-strings

Felhasználási cél: Az SQL Server Express kapcsolati karakterlánc (connection string) helyes formátumának meghatározásához (Server=.\SQLEXPRESS; Database=NagypapaHazaiDb; Trusted_Connection=True;).

[34] WCAG 2.1 akadálymentesítési irányelvek – W3C. URL: https://www.w3.org/TR/WCAG21/

Felhasználási cél: A webalkalmazás akadálymentesítési követelményeinek megismeréséhez. A 3.14. fejezetben dokumentált akadálymentesítési megvalósítások és hiányosságok értékeléséhez.

5.5. Az irodalomjegyzék megjegyzései

Utolsó ellenőrzés időpontja: 2026. január 31. Az online források az adott dátumig aktívak és elérhetők voltak. Néhány online forrás URL-je változhat az idők folyamán; ha egy link nem működik, a tartalmat általában az internetes archívumban (web.archive.org) meg lehet találni az adott URL-en.

Az irodalomjegyzékben felsorolt szakkönyveket a fejlesztőcsapat tagjai részben saját tulajdonból, részben iskolai könyvtárból, részben digitális előfizetésen (O'Reilly Learning, Manning liveBook) keresztül érték el. A Microsoft Learn és egyéb online platformok ingyenesen elérhetők, és a fejlesztők mindennap használják ezeket referenciaforrásként.

A projekt forráskódját a csapat egy privát GitHub repository-ban tárolja, amelyhez a konzulens tanár és a vizsgabizottság számára megosztható olvasási hozzáférés biztosítható. A repository tartalmazza az összes migrációs fájlt, a teljes forráskódot és a fejlesztési naplót (commit history), amely hűen tükrözi a projekt fejlődési folyamatát az első committól az utolsóig.

A. Függelék – Teljes API végpontok dokumentációja

Ez a függelék a NagypapaHazai.API projekt összes REST API végpontját részletesen dokumentálja. Minden végponthoz feltüntettük a HTTP metódust, az URL-t, az elvárt kérési törzs formátumát (request body), a lehetséges válasz-kódokat és azok értelmezését, valamint egy példa kérés-válasz párt. Ez a dokumentáció kiegészíti a Swagger UI által automatikusan generált leírást, és bővebb kontextust, magyarázatot ad az egyes végpontok rendeltetéséhez.

Az API alap URL-je fejlesztői környezetben: https://localhost:44369/api. Minden kérésnek JSON formátumú törzsöt kell tartalmaznia (Content-Type: application/json), ahol a kérési törzs releváns. A hitelesítést igénylő végpontokhoz a session cookie automatikusan kerül elküldésre a böngésző által, ha a felhasználó korábban bejelentkezett. Közvetlenül az API-t hívó kliensek (pl. Postman, curl) esetén a session ID-t manuálisan kell megadni a Cookie fejlécben.

A.1. Felhasználói API végpontok

A.1.1. POST /api/users/register – Regisztráció

Ez a végpont új felhasználói fiók regisztrációját teszi lehetővé. A kérési törzsben a teljes nevet, az email-cimet és a jelszót kell megadni JSON formátumban. A rendszer ellenőrzi, hogy az email-cím még nem foglalt, és hogy a jelszó megfelel a minimális követelményeknek (legalább 8 karakter, legalább egy nagybetű, legalább egy kisbetű, legalább egy szám). Ha az ellenőrzés sikeres, a jelszó BCrypt-tel hashelve kerül tárolásra, és a válasz 201 Created kóddal és az újonnan létrehozott felhasználói azonosítóval tér vissza.

```
POST /api/users/register
Content-Type: application/json

Kérési törzs (Request Body):
{
  "fullName": "Teszt Felhasználó",
  "email": "uj@nagypapahazai.hu",
  "password": "Uj_Jelszo123"
}

Sikeres válasz (201 Created):
{
  "id": 5,
  "fullName": "Teszt Felhasználó",
  "email": "uj@nagypapahazai.hu",
```

```
  "role": "User",
  "createdAt": "2026-01-15T14:23:11Z"
}

Hibás válasz – Email már foglalt (409 Conflict):
{
  "error": "Az email-cím már regisztrálva van.",
  "code": "EMAIL_ALREADY_EXISTS"
}

Hibás válasz – Érvénytelen adatok (400 Bad Request):
{
  "errors": {
    "password": ["A jelszónak legalább 8 karakterből kell állnia."],
    "email": ["Érvénytelen email-cím formátum."]
  }
}
```

A.1.2. POST /api/users/login – Bejelentkezés

Ez a végpont felhasználói hitelesítést végez. A kérési törzsben az email-cimet és a jelszót kell megadni. A rendszer a megadott email-cimhez tartozó hashelt jelszót ellenőrzi BCrypt segítségével. Sikeres hitelesítés esetén a szerver session-t hoz létre, és a válaszban visszaadja a felhasználó adatait. A session azonosítója automatikusan Cookie fejlécbe kerül a válaszban, amelyet a böngésző tárol és minden további kérésnél elküld.

```
POST /api/users/login
Content-Type: application/json

Kérési törzs (Request Body):
{
  "email": "admin@nagypapahazai.hu",
  "password": "Admin123!"
}

Sikeres válasz (200 OK):
{
  "id": 1,
  "fullName": "Rendszergazda",
  "email": "admin@nagypapahazai.hu",
  "role": "Admin",
  "token": "[session-azonosító]"
}

Sikertelen hitelesítés (401 Unauthorized):
{
  "error": "Érvénytelen email-cím vagy jelszó.",
  "code": "INVALID_CREDENTIALS"
}
```

A.1.3. GET /api/users/{id} – Felhasználó adatainak lekérése

Ez a végpont egy adott felhasználó adatait adja vissza. A normál felhasználók csak a saját adataikat kérhetik le; adminisztrátorok bármely felhasználó adatait megtekinthetik. Az ID paramétert az URL path-ban kell megadni egész számként. Ha a lekért felhasználó nem létezik, 404 Not Found válasz érkezik. Ha a hitelesítés hiányzik vagy érvénytelen, 401 Unauthorized válasz érkezik.

```
GET /api/users/1
Cookie: [session-cookie]

Sikeres válasz (200 OK):
{
  "id": 1,
  "fullName": "Rendszergazda",
  "email": "admin@nagypapahazai.hu",
  "role": "Admin",
  "createdAt": "2025-09-01T08:00:00Z",
  "updatedAt": "2025-09-01T08:00:00Z"
}

Nem létező felhasználó (404 Not Found):
{
  "error": "A kért felhasználó nem található.",
  "code": "USER_NOT_FOUND"
}
```

A.2. Ingatlan API végpontok

A.2.1. GET /api/properties – Összes ingatlan listázása

Ez a végpont az összes aktív ingatlant adja vissza listában. Opcionális query paraméterekkel szűrni lehet az eredményeket: search (szabad szöveges keresés a névben és helyszínben), minCapacity (minimális kapacitás), maxCapacity (maximális kapacitás), minPrice (minimális éjszakánkénti ár), maxPrice (maximális éjszakánkénti ár). Például: GET /api/properties?search=bakon&minCapacity=5 az összes, legalább 5 férőhelyes, "bakon" szöveget tartalmazó ingatlant adja vissza.

```
GET /api/properties

Sikeres válasz (200 OK):
[
  {
    "id": 1,
    "name": "Faház a Bakonyban",
    "location": "Bakony",
    "description": "Csodálatos faház a Bakony erdős szívében...",
    "capacity": 10,
    "pricePerNight": 25000,
    "isActive": true,
    "createdAt": "2025-09-01T08:00:00Z"
  },
  {
    "id": 2,
```

```
    "name": "Vízparti Nyaraló",
    "location": "Balatonlelle",
    "description": "Gyönyörű nyaraló közvetlenül a Balatonnál...",
    "capacity": 6,
    "pricePerNight": 35000,
    "isActive": true,
    "createdAt": "2025-09-01T08:00:00Z"
  }
]
```

A.2.2. GET /api/properties/{id} – Egy ingatlan adatainak lekérése

Ez a végpont egyetlen ingatlan teljes adatait adja vissza az azonosítója alapján. A válasz tartalmazza az alap adatokon túl az ingatlanhoz kapcsolódó összes jövőbeli foglalás összefoglalóját is (elfogadott státuszú foglalások), ami lehetővé teszi az elérhetőségi naptár megjelenítését. Ez az adat a foglalási naptár oldal betöltésekor kerül felhasználásra.

A.2.3. POST /api/properties – Új ingatlan létrehozása (Admin)

Ez a végpont csak adminisztrátori jogosultsággal hívható. Létrehoz egy új ingatlant az adatbázisban. A kérési törzsben meg kell adni az ingatlan nevét, helyszínét, leírását (opcionális), kapacitását, éjszakánkénti árát és aktív/inaktív státuszát. A létrehozás után 201 Created válasz érkezik az újonnan létrehozott ingatlan teljes adataival, beleértve a rendszer által generált azonosítót.

```
POST /api/properties
Cookie: [admin-session-cookie]
Content-Type: application/json

Kérési törzs (Request Body):
{
  "name": "Hegyi Menedékház",
  "location": "Mátra, Gyöngyöstarján",
  "description": "Csendes, erdei menedékház a Mátrában...",
  "capacity": 8,
  "pricePerNight": 22000,
  "isActive": true
}

Sikeres válasz (201 Created):
{
  "id": 4,
  "name": "Hegyi Menedékház",
  "location": "Mátra, Gyöngyöstarján",
  "capacity": 8,
  "pricePerNight": 22000,
  "isActive": true,
  "createdAt": "2026-01-15T15:00:00Z"
}

Nem adminisztrátori fiók (403 Forbidden):
{
  "error": "Nincs hozzáférési jogosultságod ehhez a művelethez.",
```

```
  "code": "FORBIDDEN"
}
```

A.2.4. PUT /api/properties/{id} – Ingatlan adatainak módosítása (Admin)

Ez a végpont egy meglévő ingatlan adatainak teljes felülírására szolgál (PUT szemantika). Az Admin szerepkör szükséges. A kérési törzsben az összes módosítható mezőt meg kell adni, még azokat is, amelyek nem változnak. Ha csak egyes mezőket kívánja módosítani, a PATCH metódust kellene alkalmazni – azonban a jelenlegi implementáció csak PUT-ot támogat.

A.2.5. DELETE /api/properties/{id} – Ingatlan törlése (Admin)

Ez a végpont egy ingatlant töröl az adatbázisból. Admin szerepkör szükséges. Ha az ingatlanhoz aktív (Függőben vagy Elfogadva státuszú) foglalások kapcsolódnak, a törlés visszautasításra kerül 409 Conflict válasszal. Csak azok az ingatlanok törölhetők, amelyekhez nincsenek aktív foglalások. A törlés kaszkád jelleggel törli az összes Lezárt, Elutasított és Lemondott foglalást is.

A.3. Foglalási API végpontok

A.3.1. GET /api/bookings – Foglalások listázása

Ez a végpont visszaadja a foglalások listáját. Normál felhasználók csak a saját foglalásaikat látják; adminisztrátorok az összes felhasználó foglalásait. Opcionális szűrési paraméterek: status (Pending/Approved/Rejected/Cancelled/Closed), propertyId (ingatlan azonosítója), userId (felhasználó azonosítója, csak admin számára), startDate, endDate (dátumtartomány szűrő).

```
GET /api/bookings?status=Pending
Cookie: [admin-session-cookie]

Sikeres válasz (200 OK):
[
  {
    "id": 1,
    "userId": 2,
    "userFullName": "Teszt Felhasználó",
    "userEmail": "teszt@nagypapahazai.hu",
    "propertyId": 1,
    "propertyName": "Faház a Bakonyban",
    "checkIn": "2026-07-10",
    "checkOut": "2026-07-15",
    "guests": 4,
```

```
    "totalPrice": 125000,
    "specialRequests": "Korai check-in kérés (12:00 előtt).",
    "barterOffer": "Kertrendezés 1 napig fűnyíróval.",
    "status": "Pending",
    "adminNote": null,
    "createdAt": "2026-06-01T09:15:22Z",
    "updatedAt": "2026-06-01T09:15:22Z"
  }
]
```

A.3.2. POST /api/bookings – Új foglalás létrehozása

Ez a végpont új foglalás létrehozására szolgál. Bejelentkezett felhasználó szükséges. A kérési törzsben meg kell adni az ingatlan azonosítóját, az érkezési és távozási dátumot, a vendégszámot, az opcionális különleges kéréseket és az opcionális csereajánlatot. A szerver oldalon ütközésvizsgálat fut le, és ha az ingatlan a kért időszakban már foglalt (elfogadott állapotban), a kérés 409 Conflict válasszal visszautasításra kerül.

```
POST /api/bookings
Cookie: [user-session-cookie]
Content-Type: application/json

Kérési törzs (Request Body):
{
  "propertyId": 2,
  "checkIn": "2026-08-05",
  "checkOut": "2026-08-10",
  "guests": 3,
  "specialRequests": "Hallergiás vagyunk, kérünk allergén-mentes ágyneműt.",
  "barterOffer": "Házilag készített lekvárokat hozok cserébe."
}

Sikeres válasz (201 Created):
{
  "id": 12,
  "propertyId": 2,
  "propertyName": "Vízparti Nyaraló",
  "checkIn": "2026-08-05",
  "checkOut": "2026-08-10",
  "guests": 3,
  "totalPrice": 175000,
  "status": "Pending",
  "createdAt": "2026-07-01T11:30:00Z"
}

Ütközés esetén (409 Conflict):
{
  "error": "Az ingatlan a megadott időszakban már foglalt.",
  "code": "BOOKING_CONFLICT",
  "conflictingBookingId": 8
}
```

A.3.3. PATCH /api/bookings/{id}/status – Foglalás státusz módosítása (Admin)

Ez az Admin-kizárólagos végpont lehetővé teszi egy foglalás státuszának módosítását. Elfogadható státuszváltozások: Pending → Approved, Pending → Rejected, Approved → Cancelled, bármely státusz → Closed. Az adminisztrátori megjegyzés (adminNote) opcionális, de Rejected esetén kötelező minimum 10 karakteres indoklás megadása szükséges a kérési törzsben.

```
PATCH /api/bookings/12/status
Cookie: [admin-session-cookie]
Content-Type: application/json

Kérési törzs - Elfogadás:
{
  "status": "Approved",
  "adminNote": "Csodás csereajánlat, szívesen várjuk! Check-in: 14:00."
}

Kérési törzs - Elutasítás:
{
  "status": "Rejected",
  "adminNote": "Sajnálom, az ingatlan karbantartás alatt van ebben az időszakban."
}

Sikeres válasz (200 OK):
{
  "id": 12,
  "status": "Approved",
  "adminNote": "Csodás csereajánlat, szívesen várjuk! Check-in: 14:00.",
  "updatedAt": "2026-07-02T09:00:00Z"
}
```

A.4. Esemény API végpontok

A.4.1. GET /api/events – Összes esemény listázása

Ez a végpont visszaadja az összes eseményt. Opcionális szűrési paraméterek: upcoming (true/false – csak jövőbeli vagy összes esemény), search (szabad szöveges keresés névben és helyszínben). A válasz tartalmazza az egyes eseményeknél a regisztrált résztvevők számát is, amit az EventRegistrations táblából aggregál a szerver.

A.4.2. POST /api/events/{id}/register – Eseményre regisztráció

Ez a végpont lehetővé teszi bejelentkezett felhasználó számára egy eseményre való regisztrációt. Ha a felhasználó már regisztrált az eseményre, 409 Conflict válasz érkezik. Ha az esemény betelt (elérte a maximális résztvevőszámot), szintén 409 Conflict válasz érkezik, más hibakóddal. Sikeres regisztráció esetén 201 Created válasz érkezik a regisztráció adataival.

A.4.3. DELETE /api/events/{id}/register – Esemény-regisztráció lemondása

Ez a végpont lehetővé teszi egy eseményre vonatkozó regisztráció törlését. Csak az adott felhasználó saját regisztrációját törölheti; az adminisztrátor bármely regisztrációt törölhet. Ha a felhasználó nem volt regisztrálva az eseményre, 404 Not Found válasz érkezik. Ha az esemény már lezárult (múlt időpontú), a regisztráció törlése visszautasítható (ez konfigurálható viselkedés).

A.5. HTTP státusz kódok összefoglaló referencia

Az API következetes HTTP státusz kódokat alkalmaz minden végponton. Az alábbi táblázat összefoglalja a leggyakrabban használt kódokat és azok értelmezését a NagypapaHazai API kontextusában.

HTTP Kód	Neve	Mikor fordul elő	Példa
200 OK	Sikeres	GET és PATCH műveletek sikeresek	Foglalás státusz módosítva
201 Created	Létrehozva	POST műveletek sikeresek (erőforrás létrehozva)	Új foglalás, regisztráció, esemény
204 No Content	Nincs tartalom	DELETE műveletek sikeresek	Regisztráció törlése
400 Bad Request	Hibás kérés	Érvénytelen input adatok, validációs hiba	Rövid jelszó, hiányzó kötelező mező
401 Unauthorized	Hitelesítés szükséges	Hiányzó vagy lejárt session	Bejelentkezés nélkül admin API hívás
403 Forbidden	Hozzáférés tiltott	Nincs megfelelő szerepkör	Normál user admin végpontot hív
404 Not Found	Nem található	A kért erőforrás nem létezik az adatbázisban	Nem létező ingatlan ID
409 Conflict	Ütközés	Duplikált email, foglalási ütközés, már regisztrált	Betelt eseményre regisztráció

HTTP Kód	Neve	Mikor fordul elő	Példa
500 Internal Server Error	Szerverhiba	Nem kezelt kivétel a szerveren	SQL kapcsolati hiba

B. Függelék – Adatbázis séma részletes dokumentációja

Ez a függelék a NagypapaHazai adatbázis összes táblájának, oszlopának, adattípusának, korlátainak (constraints), indexeinek és kapcsolatainak részletes dokumentációját tartalmazza. A séma az Entity Framework Core Code-First migrációkból generálódott, ezért az itt leírt struktúra pontosan tükrözi a kódbázisban definiált entitásmodelleket. Minden tábla leírásnál megadjuk a tábla üzleti szerepét is, nem csak a technikai részleteket.

B.1. Users tábla

A Users tábla a rendszer összes regisztrált felhasználóját tárolja. Az adminisztrátori felhasználók is ebben a táblában szerepelnek, megkülönböztetve a Role oszlop értékével. A tábla minden sorához automatikusan generálódik egy egyedi numerikus azonosító (Id), amelyre más táblák foreign key-ként hivatkoznak.

Oszlop neve	Adattípus (SQL)	Nullable	Default	Leírás
Id	INT IDENTITY(1,1)	NEM	Auto	Elsődleges kulcs, automatikusan növekvő
FullName	NVARCHAR(200)	NEM	–	Felhasználó teljes neve
Email	NVARCHAR(255)	NEM	–	Email-cím (egyedi index van rajta)
PasswordHash	NVARCHAR(MAX)	NEM	–	BCrypt-tel hashelt jelszó
Role	NVARCHAR(50)	NEM	User	Lehetséges értékek: User, Admin

Oszlop neve	Adattípus (SQL)	Nullable	Default	Leírás
CreatedAt	DATETIME2	NEM	GETUTCDATE()	Regisztráció dátuma és ideje (UTC)
UpdatedAt	DATETIME2	NEM	GETUTCDATE()	Utolsó módosítás dátuma (UTC)

Indexek: Az Email oszlopon egyedi (UNIQUE) index van, amely megakadályozza a duplikált email-cimek regisztrálását. Constraints: NOT NULL korlátok az összes kötelező oszlopon (FullName, Email, PasswordHash, Role).

B.2. Properties tábla

A Properties tábla az összes bejegyzett ingatlant tárolja. Egy ingatlan aktív vagy inaktív lehet – az inaktív ingatlanok nem jelennek meg a nyilvános listázó oldalon, de az adatbázisban megmaradnak az archiválás érdekében. Az ingatlanokhoz kapcsolódó foglalások és az ingatlan törlése kaszkád-törlési szabályokat követ.

Oszlop neve	Adattípus (SQL)	Nullable	Default	Leírás
Id	INT IDENTITY(1,1)	NEM	Auto	Elsődleges kulcs
Name	NVARCHAR(300)	NEM	–	Ingatlan neve
Location	NVARCHAR(500)	NEM	–	Helyszín (város / régió)
Description	NVARCHAR(MAX)	IGEN	NULL	Teljes leírás (opcionális)
Capacity	INT	NEM	–	Maximális vendégszám (min. 1)

Oszlop neve	Adattípus (SQL)	Nullable	Default	Leírás
	DECIMAL(10,2)	NEM	–	Éjszakánkénti ár forintban
IsActive	BIT	NEM	1	Aktív-e az ingatlan (1=igen, 0=nem)
CreatedAt	DATETIME2	NEM	GETUTCDATE()	Létrehozás dátuma (UTC)
UpdatedAt	DATETIME2	NEM	GETUTCDATE()	Utolsó módosítás (UTC)

Indexek: A Name oszlopon nem-egyedi index van a keresési teljesítmény javítása érdekében. Az IsActive oszlopon szintén van index, mivel a legtöbb lekérdezés csak az aktív ingatlanokat kéri le.

B.3. Bookings tábla

A Bookings tábla az összes foglalást tárolja, beleértve a lemondott, elutasított és lezárt foglalásokat is. A tábla két foreign key-t tartalmaz: UserId (a foglalást végző felhasználóra mutat) és PropertyId (a foglalt ingatlanra mutat). Az ütközésvizsgálat erre a táblára fut le, ahol az elfogadott státuszú foglalások CheckIn és CheckOut értékeit kell ellenőrizni.

Oszlop neve	Adattípus (SQL)	Nullable	FK	Leírás
Id	INT IDENTITY(1,1)	NEM	PK	Elsődleges kulcs
UserId	INT	NEM	FK → Users.Id	A foglalást leadó felhasználó azonosítója
PropertyId	INT	NEM	FK → Properties.Id	A foglalt ingatlan azonosítója

Oszlop neve	Adattípus (SQL)	Nullable	FK	Leírás
CheckIn	DATE	NEM	–	Érkezési dátum (DATE, időpont nélkül)
CheckOut	DATE	NEM	–	Távozási dátum (DATE, időpont nélkül)
Guests	INT	NEM	–	Vendégek száma (min. 1, max. Property.Capacity)
TotalPrice	DECIMAL(12,2)	NEM	–	Számított összeg (ár × éjszakák száma)
SpecialRequests	NVARCHAR(500)	IGEN	NULL	Különleges kérések (max. 500 kar.)
BarterOffer	NVARCHAR(300)	IGEN	NULL	Csereajánlat szövege (max. 300 kar.)
Status	NVARCHAR(50)	NEM	Pending	Lehetséges értékek: Pending, Approved, Rejected, Cancelled, Closed
AdminNote	NVARCHAR(500)	IGEN	NULL	Adminisztrátori megjegyzés a státuszváltáshoz
CreatedAt	DATETIME2	NEM	–	Foglalás beküldési dátuma (UTC)
UpdatedAt	DATETIME2	NEM	–	Utolsó módosítás dátuma (UTC)

Indexek: Az UserId és a PropertyId oszlopokon egyaránt nem-egyedi indexek vannak a szűrési lekérdezések felgyorsításához. Constraints: A CheckIn dátumnak kisebbnek kell lennie a CheckOut dátumnál (CHECK constraint). A Guests értéknek 1 és 100 között kell lennie. Kaszkád szabályok: Ha egy felhasználót törölnek, az összes foglalása is törlésre kerül (RESTRICT – jelenleg le van tiltva a közvetlen felhasználó-

törlés emiatt). Ha egy ingatlant törölnek, az aktív foglalások megelőzik a törlést (RESTRICT).

B.4. Events tábla

Oszlop neve	Adattípus (SQL)	Nullable	Default	Leírás
Id	INT IDENTITY(1,1)	NEM	Auto	Elsődleges kulcs
Name	NVARCHAR(300)	NEM	–	Esemény neve
Description	NVARCHAR(MAX)	IGEN	NULL	Teljes leírás
Location	NVARCHAR(500)	NEM	–	Esemény helyszíne
EventDate	DATETIME2	NEM	–	Esemény dátuma és kezdési időpontja (UTC)
EndDate	DATETIME2	IGEN	NULL	Esemény befejezési időpontja (opcionális)
MaxParticipants	INT	NEM	–	Maximális résztvevőszám (min. 1)
CreatedAt	DATETIME2	NEM	GETUTCDATE()	Esemény létrehozásának dátuma
UpdatedAt	DATETIME2	NEM	GETUTCDATE()	Utolsó módosítás dátuma

B.5. EventRegistrations tábla

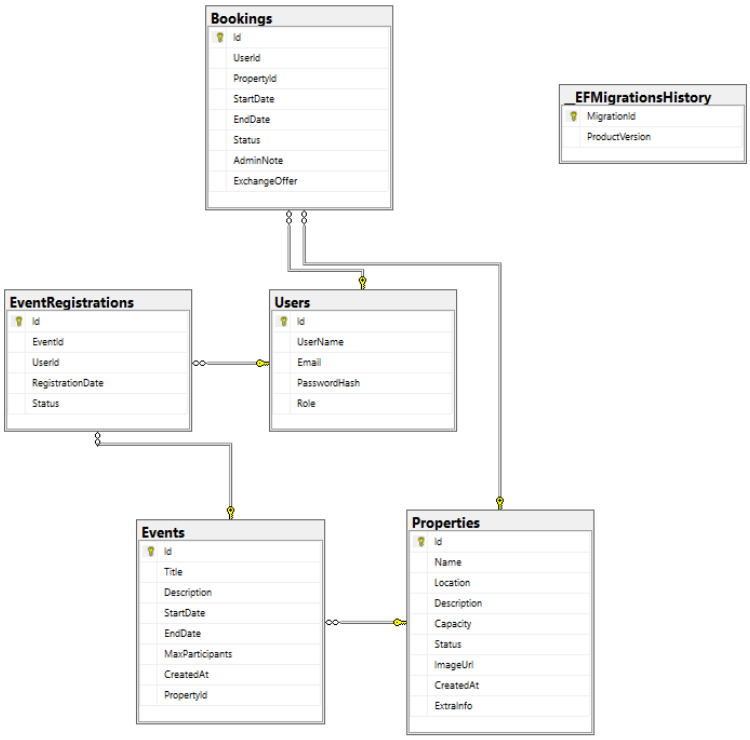
Ez a tábla az esemény-regisztrációkat tárolja, kapcsolótáblaként (junction table) a Users és az Events táblák között. Egy felhasználó egy eseményre csak egyszer regisztrálhat – ezt egyedi összetett index biztosítja az UserId és EventId kombináción. A regisztrált résztvevők számát mindig az EventRegistrations táblából COUNT aggregációval számítja a rendszer, nem tárolja denormalizáltan az Events táblában.

Oszlop neve	Adattípus (SQL)	Nullable	FK	Leírás
Id	INT IDENTITY(1,1)	NEM	PK	Elsődleges kulcs
UserId	INT	NEM	FK → Users.Id	A regisztrált felhasználó azonosítója
EventId	INT	NEM	FK → Events.Id	Az esemény azonosítója
RegisteredAt	DATETIME2	NEM	GETUTCDATE()	Regisztráció időpontja (UTC)

Indexek: Egyedi összetett index az (UserId, EventId) kombinációra, amely megakadályozza a duplikált regisztrációkat. Kaszkád szabályok: Ha egy felhasználót törölnek, az összes event-regisztrációja törlődik (CASCADE DELETE). Ha egy eseményt törölnek, az összes regisztrációja törlődik (CASCADE DELETE).

B.6. Adatbázis kapcsolatok (ER diagram szöveges leírása)

Az adatbázis entitás-kapcsolat modellje az alábbiak szerint írható le: A Users tábla egy-a-többhöz (1:N) kapcsolatban áll a Bookings táblával (egy felhasználónak több foglalása lehet, de egy foglalás csak egy felhasználóhoz tartozik). A Properties tábla szintén 1:N kapcsolatban áll a Bookings táblával (egy ingatlanhoz több foglalás is tartozhat, de egy foglalás csak egy ingatlanhoz kapcsolódik). A Users és Events táblák között N:M (többes-a-többeshöz) kapcsolat van, amelyet az EventRegistrations kapcsolótábla valósít meg.



Az adatbázis normalizálása 3NF (harmadik normálforma) szintű: minden nem-kulcs attribútum csak az elsődleges kulcstól függ, és nincs tranzitív függőség. A TotalPrice értéke ugyan számítható a EjszakankentiAr × napok száma alapján, de szándékosan denormalizáltan tároljuk, mivel az éjszakánkénti ár a foglalás után megváltozhat, és az eredeti ár historikus megőrzése fontos az audit trail szempontjából.

C. Függelék – Konfigurációs fájlok és telepítési szkriptek

Ez a függelék a projekt telepítéséhez és konfigurálásához szükséges összes konfigurációs fájl teljes tartalmát dokumentálja. A konfigurációs fájlok pontos ismerete elengedhetetlen a projekt egy új fejlesztői gépen való beüzemeléséhez, és segít a kapcsolati problémák, hitelesítési hibák és konfiguráción alapuló rendszerhibák diagnosztizálásában.

C.1. appsettings.json – Főkonfigurációs fájl

Az appsettings.json az ASP.NET Core alkalmazások fő konfigurációs fájlja. Tartalmazza az adatbázis-kapcsolati karakterláncot, a naplózási beállításokat és egyéb alkalmazás-szintű konfigurációkat. Fontos: ez a fájl kerül verziókezelő rendszerbe, ezért soha ne tartalmazzon éles környezeti titkos adatokat (jelszavak, API kulcsok). Azokat a felhasználói titkosokba (user secrets) vagy környezeti változókba kell elhelyezni.

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=.\\SQLEXPRESS;Database=NagypapaHazaiDb;
      Trusted_Connection=True;TrustServerCertificate=True;",
    "BlazorConnection": "Server=.\\SQLEXPRESS;Database=NagypapaHazaiDb;
      Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  },
  "AllowedHosts": "*",
  "Session": {
    "IdleTimeout": "00:20:00",
    "Cookie": {
      "HttpOnly": true,
      "SameSite": "Lax",
      "SecurePolicy": "Always"
    }
  },
  "Cors": {
    "AllowedOrigins": [
      "https://localhost:7206",
      "http://localhost:5011"
    ]
  }
}
```

C.2. launchSettings.json – Fejlesztői indítási beállítások

A launchSettings.json fájl a fejlesztői indítási profilokat tartalmazza. Ez a fájl nem kerül éles környezetbe, csak fejlesztői gépen használatos. Két profilt definiál: a http profilt (HTTP-n futó fejlesztői verzió) és a https profilt (HTTPS-n futó, tanúsítvánnyal hitelesített verzió). A Visual Studio alapértelmezés szerint a https profilt használja.

```
// NagypapaHazaiBlazor/Properties/launchSettings.json
{
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5011",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "https://localhost:7206;http://localhost:5011",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}

// NagypapaHazai.API/Properties/launchSettings.json
{
  "profiles": {
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:44369;http://localhost:5010",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

C.3. Program.cs – Az alkalmazás belépési pontja és middleware konfiguráció

A Program.cs fájl az ASP.NET Core alkalmazás belépési pontja. Ebben kerülnek regisztrálásra a dependency injection (DI) konténerbe az alkalmazás összes szervize, és itt épül fel a middleware pipeline, amely meghatározza, hogy egy HTTP

kérés milyen sorrendben milyen feldolgozási lépéseken megy keresztül. Az ASP.NET Core 6.0 óta a Startup.cs fájl megszűnt, és tartalmát a Program.cs vette át.

```
// NagypapaHazai.API/Program.cs
var builder = WebApplication.CreateBuilder(args);

// Szervizek regisztrálása a DI konténerbe
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c => {
  c.SwaggerDoc("v1", new OpenApiInfo {
    Title = "Nagypapa Házai API",
    Version = "v1",
    Description = "Közösségi Ingatlanhasználati Platform REST API"
  });
});

// EF Core adatbázis kapcsolat
builder.Services.AddDbContext<NagypapaHazaiDbContext>(options =>
  options.UseSqlServer(
    builder.Configuration.GetConnectionString("DefaultConnection")));

// CORS konfiguráció (Blazor alkalmazás URL-ek engedélyezése)
builder.Services.AddCors(options => {
  options.AddPolicy("BlazorPolicy", policy => {
    policy.WithOrigins(
      "https://localhost:7206",
      "http://localhost:5011")
      .AllowAnyHeader()
      .AllowAnyMethod()
      .AllowCredentials();
  });
});

// Session beállítása
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options => {
  options.IdleTimeout = TimeSpan.FromMinutes(20);
  options.Cookie.HttpOnly = true;
  options.Cookie.SameSite = SameSiteMode.Lax;
});

var app = builder.Build();

// Middleware pipeline konfigurálása
if (app.Environment.IsDevelopment()) {
  app.UseSwagger();
  app.UseSwaggerUI();
}
app.UseHttpsRedirection();
app.UseCors("BlazorPolicy");
app.UseSession();
app.UseAuthorization();
app.MapControllers();

// Adatbázis inicializálása (seed adatok)
using (var scope = app.Services.CreateScope()) {
  var db = scope.ServiceProvider.GetRequiredService<NagypapaHazaiDbContext>();
  db.Database.Migrate();
  await SeedData.InitializeAsync(db);
}

app.Run();
```

C.4. Seed adatok inicializálási kódja

A SeedData osztály felelős az adatbázis alapadataival való feltöltéséért az alkalmazás első indításakor. A seed folyamat ellenőrzi, hogy az adatok még nem léteznek-e (idempotens működés), és csak akkor szúrja be őket, ha az adatbázis üres. Ez megakadályozza a duplikált adatok létrejöttét, ha az alkalmazás többször indul újra.

```
// NagypapaHazai.API/Data/SeedData.cs
public static class SeedData {
  public static async Task InitializeAsync(NagypapaHazaiDbContext context) {
    // Admin felhasználó létrehozása, ha még nem létezik
    if (!context.Users.Any(u => u.Email == "admin@nagypapahazai.hu")) {
      context.Users.Add(new User {
        FullName = "Rendszergazda",
        Email = "admin@nagypapahazai.hu",
        PasswordHash = BCrypt.Net.BCrypt.HashPassword("Admin123!"),
        Role = "Admin",
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow
      });
    }

    // Teszt felhasználó
    if (!context.Users.Any(u => u.Email == "teszt@nagypapahazai.hu")) {
      context.Users.Add(new User {
        FullName = "Teszt Felhasználó",
        Email = "teszt@nagypapahazai.hu",
        PasswordHash = BCrypt.Net.BCrypt.HashPassword("Teszt123!"),
        Role = "User",
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow
      });
    }

    // Alap ingatlanok
    if (!context.Properties.Any()) {
      context.Properties.AddRange(
        new Property {
          Name = "Faház a Bakonyban",
          Location = "Bakony",
          Description = "Gyönyörű faház a Bakony szívében, 10 főre, saját
kerttel.",
          Capacity = 10,
          PricePerNight = 25000,
          IsActive = true,
          CreatedAt = DateTime.UtcNow
        },
        new Property {
          Name = "Vízparti Nyaraló",
          Location = "Balatonlelle",
          Description = "Balatonlellei nyaraló, közvetlen vízparton, 6 főre.",
          Capacity = 6,
          PricePerNight = 35000,
          IsActive = true,
          CreatedAt = DateTime.UtcNow
        },
        new Property {
          Name = "Villányi Vendégház",
          Location = "Villány",
          Description = "Hangulatos vendégház Villány borvidékén, 4 főre.",
```

```
            Capacity = 4,
            PricePerNight = 20000,
            IsActive = true,
            CreatedAt = DateTime.UtcNow
          }
        );
      }

      await context.SaveChangesAsync();
    }
}
```

D. Függelék – Fejlesztési napló és mérföldkövek

Ez a függelék a projekt fejlesztési folyamatát kronológiai sorrendben dokumentálja, a tervezési fázistól a záródolgozat benyújtásáig. A fejlesztési napló célja kettős: egyrészt dokumentálja a meghozott döntéseket és azok hátterét, másrészt bemutatja azt a fejlődési ívét, amelyen a csapat végigment a projekt során. Ezek a feljegyzések nemcsak a jelenlegi dokumentáció számára értékesek, hanem mintaként is szolgálhatnak jövőbeli fejlesztési projektek tervezéséhez.

D.1. Projekt mérföldkövek kronológiai táblázata

D.2. Főbb tervezési döntések naplója

Az alábbi alfejezet azokat a kritikus tervezési döntéseket dokumentálja, amelyek a projekt irányát és kimenetelét a legjobban meghatározták. Minden döntésnél leírjuk a kontextust (mi volt a kérdés), az alternatívákat (milyen lehetőségek voltak) és a döntés okát (miért ezt választottuk).

D.2.1. Döntés: Blazor Server vs. Blazor WebAssembly vs. MVC

Kontextus: A UI technológia kiválasztásakor három lehetőség volt asztalon: a hagyományos ASP.NET Core MVC (Razor Views), a Blazor Server és a Blazor WebAssembly. Az MVC a legismertebb és legjobban dokumentált opció lett volna, de a valós idejű funkciókhoz (naptár, állapotfrissítés) JavaScript-es megoldásokat igényelt volna. A Blazor WebAssembly előnye a jobb offline képesség és az alacsonyabb szerver-terhelés, hátránya a nagyobb letöltési méret és a bonyolultabb kommunikáció a backend-del.

Döntés: Blazor Server. Indok: A szerver oldali renderelés és a SignalR-alapú kommunikáció lehetővé teszi a valós idejű UI-frissítéseket anélkül, hogy külön WebSocket infrastruktúrát kellene kiépíteni. A csapat .NET-es hátterének köszönhetően a C# alapú frontend fejlesztés produktívabb volt, mint egy JavaScript-es framework (React, Angular, Vue) megtanulása. Bár a Blazor Server skálázhatóság szempontjából korlátozottabb, a projekt fejlesztői méretéhez ez nem volt probléma.

D.2.2. Döntés: Monolitikus vs. N-Tier architektúra

Kontextus: A projekt elején dönteni kellett, hogy monolitikus (egyprojektes) vagy többrétegű (N-Tier, több-projektes) architektúrát alkalmazunk. A monolitikus architektúra egyszerűbb, gyorsabban konfigurálható és könnyebb kezdőknek. Az N-Tier architektúra (külön API projekt, külön Blazor projekt, közös Shared könyvtár) komplexebb, de jobb kódszeparációt biztosít.

Döntés: N-Tier architektúra (három projekttel). Indok: Az N-Tier architektúra lehetővé teszi, hogy a jövőbeli mobilalkalmazás ugyanazt az API-t használja, és a kód felelősségi körei tisztán elválaszthatók. Bár kezdetben lassabb volt a fejlesztés a többlet konfiguráció miatt (CORS, session megosztás, projekt-hivatkozások), hosszú távon a kód minősége és karbantarthatósága javult. A Shared projekt különösen értékes: a DTO-k és a validációs logikák egy helyen definiálva vannak, és mind az API, mind a Blazor client számára elérhetők.

D.2.3. Döntés: SQL Server vs. SQLite vs. PostgreSQL

Kontextus: Az adatbázis motor kiválasztásakor három opció merült fel: az SQLite (egyszerű, fájlalapú, gyors prototipizáláshoz), az SQL Server Express (Windows-natív, Visual Studio integrált) és a PostgreSQL (nyílt forráskódú, platformfüggetlen, ingyenes). Az SQLite legegyszerűbb lett volna beállítani, de teljesítménye és funkcionalitása korlátozott nagyobb adatmennyiség esetén.

Döntés: SQL Server Express. Indok: A fejlesztőcsapat tagjai Windows-on dolgoznak, ahol az SQL Server Express könnyen telepíthető és a Visual Studio nélkülözhetetlen SQL Server Management Studio eszközzel integráltan kezelhető. Az EF Core SQL Server provider teljes körű, jól dokumentált. Bár a PostgreSQL nyílt forráskódú és Azure-on is könnyen deployolható, az SQL Server ismertsége a csapaton belül magasabb volt, és a projekt méretéhez az Express kiadás teljesen elegendő.

D.3. Sprinthely-összefoglaló és munkaórák becslése

A fejlesztési folyamat informális sprint-alapú szervezéssel zajlott, amelynek keretében kéthetente tartottunk értékelő megbeszéléseket a konzulens tanárral. Az alábbiakban bemutatjuk az egyes fejlesztési területekre fordított becsült munkaórákat.

Ezek becslések, nem pontos mérések – a csapat nem alkalmazott szigorú időkövetési eszközt.

Fejlesztési terület	Fő felelős	Megjegyzés
Adatmodell és adatbázis	Bota Csongor	ER diagram, EF Core konfig, migrációk
REST API fejlesztés	Batsukh Temuujin	Összes kontroller, validáció, Swagger
Blazor UI fejlesztés	Kania, Bota	Összes Blazor komponens és oldal
Admin felület	Kania Áron, Batsukh	Admin oldal UI és logika
Foglalási rendszer (ütközés)	Bota Csongor	Ütközésvizsgálat algoritmus
Csereajánlat modul	Kania, Bota	UI és API integráció
Eseménykezelő modul	Kania, Bota	Lista, részletes oldal, regisztráció
Tesztelés és bugfixing	Mindenki	Manuális tesztek, edge case javítás
Dokumentáció (záródolgozat)	Mindenki	Legidőigényesebb feladat
Git, konfigurációk, devops	Bota Csongor	GitHub repo, CI beállítás (alap)

A dokumentáció megírása bizonyult a legidőigényesebb feladatnak, ami tanulságos: sok fejlesztő alábecsüli a dokumentálás munkaterhelését. A záródolgozat megírása nem csupán a kész funkciók leírását jelenti, hanem az architektúrális döntések indoklását, a kód részletes magyarázatát, a felhasználói útmutató elkészítését és a tesztelési folyamat dokumentálását is. A jövőbeli fejlesztőknek ezt a tapasztalatot különösen szem előtt kell tartani a projekt ütemezésének tervezésekor.

D.4. Csapat retrospektíva

A projekt lezárultával a csapat egy informális retrospektív megbeszélést tartott, amelyen mindenki elmondta, mit ítélt sikeresnek és mit csinálna másképp. Az alábbiakban összefoglaljuk a legfontosabb visszajelzéseket.

D.4.1. Ami jól ment

- A kommunikáció a csapaton belül végig nyílt és konstruktív volt. Nem volt személyes konfliktus, és a technikai véleménykülönbségeket érvelés alapján oldottuk meg.
- A Git-alapú munkamenet bevezetése (feature branchek) a projekt második felében nagyon megkönnyítette az együttmunkát.
- A csereajánlat ötlete mindenkit motivált: tudtuk, hogy valami valóban egyedit és értékeset alkotunk.
- A Blazor Server választása jónak bizonyult – a valós idejű UI-frissítések anélkül megvalósultak, hogy JavaScript-et kellett volna írnunk.
- Az N-Tier architektúra, bár nehezebb volt beállítani, végül jobb kódszerkezetet eredményezett, mint egy monolitikus megközelítés lett volna.

D.4.2. Amin érdemes lett volna változtatni

- A dokumentálást érdemes lett volna párhuzamosan a fejlesztéssel végezni, nem a végére hagyni. A retrospektív dokumentálás nehezebb, mert sok döntés memóriája elhalványul.
- Az adatmodellt érdemes lett volna hamarabb véglegesíteni – a késői módosítások mindig plusz migráció-munkával jártak.
- Az automatizált tesztelés (unit tesztek, integrációs tesztek) bevezetése lett volna hasznos; jelenleg csak manuális tesztelés történt.
- Az e-mail értesítési rendszer implementálását érdemes lett volna a tervezett funkciók közé bevenni és időt szánni rá; utólag kiderült, hogy ez a leghiányolt funkció.
- A munkaidő-követés (pl. egy egyszerű Toggl-szerű eszközzel) segített volna a projekt tényleges munkaterhelésének megértésében.

A Nagypapa Házai projekt összességében sikeres fejlesztési élmény volt a csapat számára. Nemcsak technikai tudást szereztünk, hanem megtanultunk csapatban dolgozni, kompromisszumokat kötni és egy komplex szoftverfejlesztési projektet az első konceptuális ötlettől a kész, dokumentált, tesztelt alkalmazásig elvezetni. Ez a tapasztalat, bármilyen programozói karrierben is folytatjuk majd a munkát, értékes kiindulópontot jelent.

E. Függelék – Blazor komponens architektúra részletes leírása

Ez a függelék a Nagypapa Házai Blazor Server alkalmazásban szereplő összes Razor komponens részletes leírását tartalmazza. Minden komponensnél ismertetjük annak szerepét, a bemeneti paramétereit, az állapotkezelési logikáját, az életciklus-módszereit és a más komponensekkel való kapcsolatát. Ez a fejezet elsősorban a jövőbeli fejlesztők számára értékes, akik a projektet karbantartani vagy bővíteni kívánják.

A Blazor Server alkalmazásban a komponensek .razor kiterjesztésű fájlokban vannak definiálva, amelyek három részből állnak: a HTML/Blazor sablon-szekció (a fájl teteje), a @code blokk (C# kód, amely a komponens logikáját tartalmazza) és opcionálisan egy önálló .razor.cs code-behind fájl (ha a logika terjedelmes). A Nagypapa Házai projektben a code-behind megközelítést alkalmazzuk a nagyobb, összetettebb komponenseknél.

E.1. Az alkalmazás komponens fájlstruktúrája

A Blazor alkalmazás Pages és Shared mappákba rendezi a komponenseket. A Pages mappában az oldalak (route-ok) találhatók, amelyek a @page direktívával rendelkeznek és URL-hez vannak kötve. A Shared mappában az újrahasználható, oldal-független komponensek találhatók (például a navigációs sáv, a lábléc, a foglalás-kártya komponens).

```
NagypapaHazaiBlazor/
├── Pages/
│   ├── Index.razor                    // Főoldal (/)
│   ├── Login.razor                    // Bejelentkezési oldal (/login)
│   ├── Register.razor                 // Regisztrációs oldal (/register)
│   ├── Profile.razor                  // Profil oldal (/profile)
│   ├── Properties/
│   │   ├── PropertyList.razor         // Ingatlanok listája
│   │   └── PropertyDetail.razor       // Ingatlan részletes oldal
│   ├── Bookings/
│   │   ├── BookingList.razor          // Saját foglalások listája
│   │   ├── BookingNew.razor           // Új foglalás form
│   │   ├── BookingDetail.razor        // Foglalás részletes nézet
│   │   └── BookingCalendar.razor      // Foglalási naptár
│   ├── Events/
│   │   ├── EventList.razor            // Események listája
│   │   └── EventDetail.razor          // Esemény részletes oldal
│   └── Admin/
│       ├── AdminDashboard.razor       // Admin főoldal (/admin)
```

```
│       ├── AdminBookings.razor        // Foglaláskezelő (/admin/bookings)
│       ├── AdminProperties.razor      // Ingatlankezelő (/admin/properties)
│       ├── AdminEvents.razor          // Eseménykezelő (/admin/events)
│       └── AdminUsers.razor           // Felhasználókezelő (/admin/users)
├── Shared/
│   ├── MainLayout.razor               // Fő elrendezés (header + footer + nav)
│   ├── NavMenu.razor                  // Navigációs menü
│   ├── BookingCard.razor              // Foglalás kártya (újrahasználható)
│   ├── PropertyCard.razor             // Ingatlan kártya (újrahasználható)
│   ├── EventCard.razor                // Esemény kártya (újrahasználható)
│   ├── ConfirmModal.razor             // Megerősítő modális ablak
│   └── LoadingSpinner.razor           // Töltés indikátor
└── wwwroot/
    ├── css/                           // Egyedi CSS stílusok
    └── js/                            // Minimális JS (csak JSInterop)
```

E.2. Kulcs komponensek részletes leírása

E.2.1. MainLayout.razor – Fő elrendezés komponens

A MainLayout.razor az alkalmazás minden oldalának keretét adja. Tartalmazza a fejlécet (Nagypapa Házai logo és navigációs sáv), a fő tartalomterületet (ahol az egyes @Body oldal-tartalmak megjelennek) és a láblécet. Ez a komponens az App.razor-ban van regisztrálva, és automatikusan alkalmazásra kerül az összes @page direktívával rendelkező oldalra, kivéve ha az adott oldal egy saját @layout direktívával más elrendezést definiál.

A MainLayout komponens hozzáfér a felhasználói session-adatokhoz az injektált IHttpContextAccessor szervizen keresztül. Ez lehetővé teszi, hogy a navigációs sáv feltételesen jeleníti meg a bejelentkezési állapottól függő elemeket: be nem jelentkezett felhasználóknak „Bejelentkezés" és „Regisztráció" gombokat, bejelentkezett felhasználóknak a felhasználó nevét és a kijelentkezési lehetőséget, adminisztrátori felhasználóknak az „Admin" menüpontot is.

```
<!-- MainLayout.razor -->
@inherits LayoutComponentBase
@inject IHttpContextAccessor HttpContextAccessor

<div class="main-layout">
  <header class="nagypapa-header">
    <NavMenu />
  </header>
  <main class="content-area">
    <div class="container-lg py-3">
      @Body
    </div>
  </main>
  <footer class="nagypapa-footer">
    <p>© 2026 Nagypapa Házai - Közösségi Ingatlanhasználati Platform</p>
  </footer>
</div>
```

```
@code {
  private string? CurrentUserName =>
    HttpContextAccessor.HttpContext?.Session.GetString("UserFullName");
  private bool IsLoggedIn => CurrentUserName != null;
  private bool IsAdmin =>
    HttpContextAccessor.HttpContext?.Session.GetString("UserRole") == "Admin";
}
```

E.2.2. BookingNew.razor – Új foglalás form komponens

A BookingNew.razor a legösszetettebb form komponens az alkalmazásban. Több lépéses, dinamikus adatkötéssel rendelkezik: az ingatlan kiválasztásakor automatikusan frissül a maximális vendégszám korlát, a dátum kiválasztásakor valós időben számítódik az előzetesen becsült összköltség, és az ütközésvizsgálat eredménye megjelenik a formban még a végleges elküldés előtt.

```
<!-- BookingNew.razor -->
@page "/bookings/new"
@inject HttpClient Http
@inject NavigationManager NavManager
@inject IHttpContextAccessor HttpContextAccessor

<h2 class="mb-4">Új foglalás</h2>

@if (!IsLoggedIn) {
  <div class="alert alert-warning">
    <strong>Bejelentkezés szükséges!</strong>
    <a href="/login">Kattintson ide a bejelentkezéshez.</a>
  </div>
} else {
  <EditForm Model="@bookingModel" OnValidSubmit="HandleSubmit">
    <DataAnnotationsValidator />
    <ValidationSummary />

    <!-- Ingatlanválasztó -->
    <div class="mb-3">
      <label class="form-label">Ingatlan kiválasztása *</label>
      <InputSelect @bind-Value="bookingModel.PropertyId"
                   @bind-Value:after="OnPropertyChanged"
                   class="form-select">
        <option value="0">-- Kérem válasszon --</option>
        @foreach (var p in properties) {
          <option     value="@p.Id">@p.Name     (@p.PricePerNight.ToString("N0")
Ft/éj)</option>
        }
      </InputSelect>
    </div>

    <!-- Dátumválasztók -->
    <div class="row g-3 mb-3">
      <div class="col-md-6">
        <label class="form-label">Érkezés dátuma *</label>
        <InputDate @bind-Value="bookingModel.CheckIn"
                   @bind-Value:after="CalculateTotalPrice"
                   min="@DateTime.Today.ToString("yyyy-MM-dd")"
                   class="form-control" />
      </div>
      <div class="col-md-6">
        <label class="form-label">Távozás dátuma *</label>
        <InputDate @bind-Value="bookingModel.CheckOut"
```

```
                  @bind-Value:after="CalculateTotalPrice"
                  class="form-control" />
      </div>
    </div>

    <!-- Vendégszám -->
    <div class="mb-3">
      <label class="form-label">Vendégek száma * (max. @maxCapacity fő)</label>
      <InputNumber @bind-Value="bookingModel.Guests"
                   min="1" max="@maxCapacity"
                   class="form-control" />
    </div>

    <!-- Előzetesen becsült összeg -->
    @if (estimatedTotal > 0) {
      <div class="alert alert-info mb-3">
        Becsült összeg: @estimatedTotal.ToString("N0") Ft
        (@nightCount éjszaka × @selectedPricePerNight.ToString("N0") Ft/éj)
      </div>
    }

    <!-- Csereajánlat -->
    <div class="mb-3">
      <label class="form-label">Csereajánlat (opcionális)</label>
      <InputTextArea @bind-Value="bookingModel.BarterOffer"
                     maxlength="300" rows="3"
                     class="form-control"
                     placeholder="Mit  ajánl  fel  cserébe?  (pl.  kertrendezés,
befőtt...)" />
    </div>

    <!-- Különleges kérések -->
    <div class="mb-3">
      <label class="form-label">Különleges kérések (opcionális)</label>
      <InputTextArea @bind-Value="bookingModel.SpecialRequests"
                     maxlength="500" rows="3"
                     class="form-control" />
    </div>

    <!-- Konfliktus figyelmeztetés -->
    @if (hasConflict) {
      <div class="alert alert-danger mb-3">
        <strong>Foglalási ütközés!</strong> Az ingatlan a megadott időszakban már
foglalt.
        <a href="/bookings/calendar">Naptár megtekintése</a>
      </div>
    }

    <button type="submit" class="btn btn-success btn-lg"
            disabled="@(isSubmitting || hasConflict)">
      @(isSubmitting ? "Küldés..." : "Foglalás elküldése")
    </button>
  </EditForm>
}

@code {
  private BookingCreateDto bookingModel = new();
  private List<PropertyDto> properties = new();
  private int maxCapacity = 50;
  private decimal selectedPricePerNight = 0;
  private decimal estimatedTotal = 0;
  private int nightCount = 0;
  private bool hasConflict = false;
  private bool isSubmitting = false;
```

```
  private bool IsLoggedIn =>
    HttpContextAccessor.HttpContext?.Session.GetString("UserId") != null;

  protected override async Task OnInitializedAsync() {
    var                    response                =                    await
Http.GetFromJsonAsync<List<PropertyDto>>("/api/properties");
    properties = response ?? new();
  }

  private async Task OnPropertyChanged() {
    var     selected     =     properties.FirstOrDefault(p     =>     p.Id     ==
bookingModel.PropertyId);
    if (selected != null) {
      maxCapacity = selected.Capacity;
      selectedPricePerNight = selected.PricePerNight;
      CalculateTotalPrice();
      await CheckAvailability();
    }
  }

  private void CalculateTotalPrice() {
    if (bookingModel.CheckIn < bookingModel.CheckOut && selectedPricePerNight >
0) {
      nightCount = (bookingModel.CheckOut - bookingModel.CheckIn).Days;
      estimatedTotal = nightCount * selectedPricePerNight;
    } else {
      nightCount = 0;
      estimatedTotal = 0;
    }
  }

  private async Task CheckAvailability() {
    if (bookingModel.PropertyId > 0 &&
        bookingModel.CheckIn < bookingModel.CheckOut) {
      var url = string.Format("/api/bookings/check-availability?" +
                "propertyId={0}&checkIn={1:yyyy-MM-dd}&checkOut={2:yyyy-MM-dd}",
                bookingModel.PropertyId,                    bookingModel.CheckIn,
bookingModel.CheckOut);
      // (interpolált string helyett string.Format, a JS szintaxis miatt)
      var result = await Http.GetFromJsonAsync<AvailabilityResult>(url);
      hasConflict = result?.HasConflict ?? false;
    }
  }

  private async Task HandleSubmit() {
    isSubmitting = true;
    await CheckAvailability();
    if (hasConflict) { isSubmitting = false; return; }

    var response = await Http.PostAsJsonAsync("/api/bookings", bookingModel);
    if (response.IsSuccessStatusCode) {
      NavManager.NavigateTo("/bookings");
    } else {
      // Hiba kezelés...
      isSubmitting = false;
    }
  }
}
```

E.2.3. AdminBookings.razor – Admin foglaláskezelő komponens

Az AdminBookings.razor az adminisztrátor számára biztosítja az összes foglalás kezelését. Ez az egyik legkomplexebb admin-oldali komponens, amely szűrőket, táblázatos megjelenítést, modális ablakokat és API-hívásokat kombinál. A komponens életciklus-metódusa (OnInitializedAsync) az oldal betöltésekor azonnal lekéri az összes foglalást az API-ból, majd a szűrők változásakor dinamikusan újra lekérdezi az adatokat.

Az AdminBookings komponens állapotkezelése több szinten zajlik: a rawBookings lista tartalmazza az összes API-ból kapott foglalást, a filteredBookings lista a szűrők alapján szűrt eredményt, a selectedBooking az éppen nyitott részletes nézet / modál foglalása, a approveNote és rejectReason a modál formjában megadott szövegek. Az állapotok változásakor StateHasChanged() hívásra nincs explicit szükség, mivel a Blazor automatikusan felismeri az @bind attribútumokhoz kötött állapotok változásait.

```
// AdminBookings.razor.cs (code-behind)
public partial class AdminBookings : ComponentBase {
  [Inject] private HttpClient Http { get; set; } = default!;
  [Inject] private IHttpContextAccessor HttpContextAccessor { get; set; } =
default!;

  private List<BookingAdminDto> rawBookings = new();
  private List<BookingAdminDto> filteredBookings = new();
  private BookingAdminDto? selectedBooking;
  private string statusFilter = "All";
  private int propertyFilter = 0;
  private string approveNote = string.Empty;
  private string rejectReason = string.Empty;
  private bool isProcessing = false;
  private string? errorMessage;

  protected override async Task OnInitializedAsync() {
    await LoadBookings();
  }

  private async Task LoadBookings() {
    var response = await
Http.GetFromJsonAsync<List<BookingAdminDto>>("/api/bookings");
    rawBookings = response ?? new();
    ApplyFilters();
  }

  private void ApplyFilters() {
    filteredBookings = rawBookings
      .Where(b => statusFilter == "All" || b.Status == statusFilter)
      .Where(b => propertyFilter == 0 || b.PropertyId == propertyFilter)
      .OrderByDescending(b => b.CreatedAt)
      .ToList();
  }

  private async Task ApproveBooking(int bookingId) {
    isProcessing = true;
    var payload = new { Status = "Approved", AdminNote = approveNote };
    var response = await Http.PatchAsJsonAsync(
```

```
      $"/api/bookings/{bookingId}/status", payload);
    if (response.IsSuccessStatusCode) {
      await LoadBookings();
      selectedBooking = null;
    } else {
      errorMessage = "Hiba a státusz módosításakor.";
    }
    isProcessing = false;
  }

  private async Task RejectBooking(int bookingId) {
    if (rejectReason.Length < 10) {
      errorMessage = "Az elutasítás indoka legalább 10 karakter legyen!";
      return;
    }
    isProcessing = true;
    var payload = new { Status = "Rejected", AdminNote = rejectReason };
    var response = await Http.PatchAsJsonAsync(
      $"/api/bookings/{bookingId}/status", payload);
    if (response.IsSuccessStatusCode) {
      await LoadBookings();
      selectedBooking = null;
    }
    isProcessing = false;
  }
}
```

E.3. Komponens kommunikációs minták

A Blazor komponensek közötti kommunikáció a Nagypapa Házai alkalmazásban három fő minta szerint valósul meg: szülő-gyermek paraméterátadás, event callback-ek és injektált szervizek. Az alábbiakban mindhárom mintát konkrét példával illusztráljuk.

E.3.1. Szülő-gyermek paraméterátadás

A PropertyCard és BookingCard komponensek példái a szülő-gyermek paraméterátadásnak. Ezek az újrahasználható kártyakomponensek [Parameter] attribútummal dekorált publikus tulajdonságokon keresztül kapják meg az adatokat a szülő listázó komponensektől. Ez az egyirányú adatfolyam (szülőtől gyermeknek) az ajánlott Blazor kommunikációs minta az egyszerű adatmegjelenítési esetekre.

```
<!-- PropertyCard.razor -->
@* Gyermek komponens – egyirányú adatfogadás *@

<div class="card h-100">
  <div class="card-body">
    <h5 class="card-title">@Property.Name</h5>
    <p class="text-muted">@Property.Location</p>
    <p>Kapacitás: @Property.Capacity fő</p>
    <p class="fw-bold">@Property.PricePerNight.ToString("N0") Ft / éjszaka</p>
    <p class="card-text">@TruncateDescription(Property.Description)</p>
  </div>
  <div class="card-footer">
```

```
    <a        href="/properties/@Property.Id"        class="btn        btn-outline-
primary">Részletek</a>
  </div>
</div>

@code {
  [Parameter] public PropertyDto Property { get; set; } = default!;

  private string TruncateDescription(string? desc) =>
    desc?.Length > 150 ? desc.Substring(0, 150) + "..." : desc ?? string.Empty;
}
```

E.3.2. Event Callback-ek gyermek-szülő kommunikációhoz

Az EventCallback<T> mechanizmus lehetővé teszi, hogy a gyermek komponens eseményeket küldjön a szülő komponensnek. A Nagypapa Házai alkalmazásban ezt a mintát alkalmazzuk a ConfirmModal komponensnél: a gyermek modális ablak az „OK" gomb megnyomásakor eseményt küld a szülő oldalnak, amely elvégzi a tényleges műveletet (pl. foglalás lemondása).

```
<!-- ConfirmModal.razor -->
@* Megerősítő modális ablak - event callback-kel *@

@if (IsVisible) {
  <div class="modal d-block" tabindex="-1">
    <div class="modal-dialog">
      <div class="modal-content">
        <div class="modal-header">
          <h5 class="modal-title">@Title</h5>
        </div>
        <div class="modal-body">
          <p>@Message</p>
        </div>
        <div class="modal-footer">
          <button class="btn btn-secondary" @onclick="Cancel">Mégse</button>
          <button class="btn btn-danger" @onclick="Confirm">@ConfirmText</button>
        </div>
      </div>
    </div>
  </div>
  <div class="modal-backdrop fade show"></div>
}

@code {
  [Parameter] public bool IsVisible { get; set; }
  [Parameter] public string Title { get; set; } = "Megerősítés";
  [Parameter] public string Message { get; set; } = string.Empty;
  [Parameter] public string ConfirmText { get; set; } = "OK";
  [Parameter] public EventCallback OnConfirm { get; set; }
  [Parameter] public EventCallback OnCancel { get; set; }

  private async Task Confirm() => await OnConfirm.InvokeAsync();
  private async Task Cancel() => await OnCancel.InvokeAsync();
}
```

E.3.3. Injektált szervizek használata komponensekben

A dependency injection (DI) mechanizmuson keresztül injektált szervizek lehetővé teszik az alkalmazás-szintű állapot és funkciók megosztását a komponensek között. A Nagypapa Házai alkalmazásban az IHttpContextAccessor és a HttpClient a leggyakrabban injektált szervizek. Az IHttpContextAccessor lehetővé teszi a session adatok olvasását a Blazor komponensekből, ami az aktuálisan bejelentkezett felhasználó adatainak meghatározásához szükséges.

Fontos megjegyzés: az IHttpContextAccessor Blazor Server kontextusban speciális elővigyázatossággal kell kezelni, mivel a SignalR circuit és a HTTP kérés életciklusa különböző. Az IHttpContextAccessor.HttpContext értéke null lehet, ha a komponens nem egy HTTP kérés kontextusában renderelődik (pl. SignalR push esetén). Ezért minden IHttpContextAccessor-hívást null-ellenőrzéssel kell védeni: HttpContextAccessor.HttpContext?.Session?.GetString(...).

F. Függelék – C# entitásmodellek és DTO-k teljes kódlistája

Ez a függelék a NagypapaHazai.API és a NagypapaHazai.Shared projekt összes entitásosztályát és adatátviteli objektumát (DTO – Data Transfer Object) tartalmazza teljes forráskóddal. Ezek az osztályok az adatbázis-séma és az API kommunikáció alapjai; megértésük elengedhetetlen a projekt teljes megismeréséhez.

F.1. Entitásosztályok (Domain Models)

F.1.1. User entitás

```
// NagypapaHazai.API/Models/User.cs
using System.ComponentModel.DataAnnotations;

namespace NagypapaHazai.API.Models {
  public class User {
    public int Id { get; set; }

    [Required(ErrorMessage = "A teljes név megadása kötelező.")]
    [StringLength(200, MinimumLength = 2,
      ErrorMessage = "A név 2-200 karakter között legyen.")]
    public string FullName { get; set; } = string.Empty;

    [Required(ErrorMessage = "Az email-cím megadása kötelező.")]
    [EmailAddress(ErrorMessage = "Érvénytelen email-cím formátum.")]
    [StringLength(255)]
    public string Email { get; set; } = string.Empty;

    [Required]
    public string PasswordHash { get; set; } = string.Empty;

    [Required]
    [RegularExpression("^(User|Admin)$",
      ErrorMessage = "A szerepkör csak User vagy Admin lehet.")]
    public string Role { get; set; } = "User";

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;

    // Navigációs tulajdonságok
    public ICollection<Booking> Bookings { get; set; } = new List<Booking>();
    public ICollection<EventRegistration> EventRegistrations { get; set; }
      = new List<EventRegistration>();
  }
}
```

F.1.2. Property entitás

```
// NagypapaHazai.API/Models/Property.cs
using System.ComponentModel.DataAnnotations;
```

```
namespace NagypapaHazai.API.Models {
  public class Property {
    public int Id { get; set; }

    [Required(ErrorMessage = "Az ingatlan nevének megadása kötelező.")]
    [StringLength(300, MinimumLength = 3)]
    public string Name { get; set; } = string.Empty;

    [Required(ErrorMessage = "A helyszín megadása kötelező.")]
    [StringLength(500)]
    public string Location { get; set; } = string.Empty;

    public string? Description { get; set; }

    [Range(1, 100, ErrorMessage = "A kapacitás 1-100 fő között legyen.")]
    public int Capacity { get; set; }

    [Range(typeof(decimal), "1000", "10000000",
      ErrorMessage = "Az ár 1 000 és 10 000 000 Ft között legyen.")]
    public decimal PricePerNight { get; set; }

    public bool IsActive { get; set; } = true;

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;

    // Navigációs tulajdonság
    public ICollection<Booking> Bookings { get; set; } = new List<Booking>();
  }
}
```

F.1.3. Booking entitás

```
// NagypapaHazai.API/Models/Booking.cs
using System.ComponentModel.DataAnnotations;

namespace NagypapaHazai.API.Models {
  public class Booking {
    public int Id { get; set; }

    // Foreign keys
    public int UserId { get; set; }
    public int PropertyId { get; set; }

    [Required]
    public DateTime CheckIn { get; set; }

    [Required]
    public DateTime CheckOut { get; set; }

    [Range(1, 100)]
    public int Guests { get; set; }

    [Range(typeof(decimal), "0", "100000000")]
    public decimal TotalPrice { get; set; }

    [StringLength(500)]
    public string? SpecialRequests { get; set; }
```

```
    [StringLength(300)]
    public string? BarterOffer { get; set; }

    public string Status { get; set; } = "Pending";
    // Érvényes értékek: Pending, Approved, Rejected, Cancelled, Closed

    [StringLength(500)]
    public string? AdminNote { get; set; }

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;

    // Navigációs tulajdonságok
    public User User { get; set; } = default!;
    public Property Property { get; set; } = default!;
  }
}
```

F.1.4. Event és EventRegistration entitások

```
// NagypapaHazai.API/Models/Event.cs
namespace NagypapaHazai.API.Models {
  public class Event {
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string? Description { get; set; }
    public string Location { get; set; } = string.Empty;
    public DateTime EventDate { get; set; }
    public DateTime? EndDate { get; set; }
    public int MaxParticipants { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;

    // Navigációs tulajdonság
    public ICollection<EventRegistration> Registrations { get; set; }
      = new List<EventRegistration>();

    // Számított tulajdonság (nem tárolt)
    public int RegisteredCount => Registrations.Count;
    public bool IsFull => RegisteredCount >= MaxParticipants;
    public bool IsPast => EventDate < DateTime.UtcNow;
  }
}

// NagypapaHazai.API/Models/EventRegistration.cs
namespace NagypapaHazai.API.Models {
  public class EventRegistration {
    public int Id { get; set; }
    public int UserId { get; set; }
    public int EventId { get; set; }
    public DateTime RegisteredAt { get; set; } = DateTime.UtcNow;

    // Navigációs tulajdonságok
    public User User { get; set; } = default!;
    public Event Event { get; set; } = default!;
  }
}
```

F.2. DTO osztályok (Data Transfer Objects)

Az adatátviteli objektumok (DTO-k) a NagypapaHazai.Shared osztálykönyvtárban találhatók, és mind az API, mind a Blazor alkalmazás hivatkozik rájuk. A DTO-k három típusra oszthatók: Create DTO-k (új erőforrás létrehozásához – pl. BookingCreateDto), Read DTO-k (adatok visszaadásához – pl. BookingDto, BookingAdminDto) és Update DTO-k (erőforrás módosításához – pl. BookingStatusUpdateDto). A DTO-k biztosítják, hogy az API csak a szükséges adatokat fogadja be és adja vissza, és nem exponálja a belső entitásokat közvetlenül.

```
// NagypapaHazai.Shared/DTOs/BookingDtos.cs
using System.ComponentModel.DataAnnotations;

namespace NagypapaHazai.Shared.DTOs {

  // CREATE - Új foglalás létrehozásához
  public class BookingCreateDto {
    [Required(ErrorMessage = "Az ingatlan kiválasztása kötelező.")]
    [Range(1,   int.MaxValue,   ErrorMessage   =   "Érvényes   ingatlant   kell
kiválasztani.")]
    public int PropertyId { get; set; }

    [Required(ErrorMessage = "Az érkezési dátum megadása kötelező.")]
    public DateTime CheckIn { get; set; } = DateTime.Today;

    [Required(ErrorMessage = "A távozási dátum megadása kötelező.")]
    public DateTime CheckOut { get; set; } = DateTime.Today.AddDays(1);

    [Range(1, 100, ErrorMessage = "A vendégszámnak 1-100 között kell lennie.")]
    public int Guests { get; set; } = 1;

    [StringLength(500, ErrorMessage = "A különleges kérések max. 500 karakter.")]
    public string? SpecialRequests { get; set; }

    [StringLength(300, ErrorMessage = "A csereajánlat max. 300 karakter.")]
    public string? BarterOffer { get; set; }
  }

  // READ - Foglalás adatainak megjelenítéséhez (normál felhasználónak)
  public class BookingDto {
    public int Id { get; set; }
    public int PropertyId { get; set; }
    public string PropertyName { get; set; } = string.Empty;
    public DateTime CheckIn { get; set; }
    public DateTime CheckOut { get; set; }
    public int Guests { get; set; }
    public decimal TotalPrice { get; set; }
    public string? SpecialRequests { get; set; }
    public string? BarterOffer { get; set; }
    public string Status { get; set; } = string.Empty;
    public string? AdminNote { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }
  }

  // READ ADMIN - Foglalás adatai adminisztrátor számára (extra mezőkkel)
  public class BookingAdminDto : BookingDto {
```

```
    public int UserId { get; set; }
    public string UserFullName { get; set; } = string.Empty;
    public string UserEmail { get; set; } = string.Empty;
    public bool HasBarterOffer => !string.IsNullOrEmpty(BarterOffer);
    public bool HasSpecialRequests => !string.IsNullOrEmpty(SpecialRequests);
  }

  // STATUS UPDATE - Foglalás státuszának módosításához (Admin)
  public class BookingStatusUpdateDto {
    [Required]
    [RegularExpression("^(Approved|Rejected|Cancelled|Closed)$")]
    public string Status { get; set; } = string.Empty;

    [StringLength(500)]
    public string? AdminNote { get; set; }
  }
}
```

F.3. DbContext konfiguráció

Az NagypapaHazaiDbContext osztály az Entity Framework Core kapuőre: ő közvetíti a kapcsolatot az alkalmazáskód és az SQL Server adatbázis között. A DbContext-ben definiálják a DbSet tulajdonságokat (amelyek a tábláknak felelnek meg), és az OnModelCreating metódusban kerülnek konfigurálásra a kapcsolatok, indexek, táblanevet-felülírások és az egyéb séma-szintű beállítások.

```
// NagypapaHazai.API/Data/NagypapaHazaiDbContext.cs
using Microsoft.EntityFrameworkCore;
using NagypapaHazai.API.Models;

namespace NagypapaHazai.API.Data {
  public class NagypapaHazaiDbContext : DbContext {
    public        NagypapaHazaiDbContext(DbContextOptions<NagypapaHazaiDbContext>
options)
      : base(options) { }

    // DbSet-ek - minden tábla egy DbSet-nek felel meg
    public DbSet<User> Users { get; set; } = default!;
    public DbSet<Property> Properties { get; set; } = default!;
    public DbSet<Booking> Bookings { get; set; } = default!;
    public DbSet<Event> Events { get; set; } = default!;
    public DbSet<EventRegistration> EventRegistrations { get; set; } = default!;

    protected override void OnModelCreating(ModelBuilder modelBuilder) {
      base.OnModelCreating(modelBuilder);

      // Users tábla konfiguráció
      modelBuilder.Entity<User>(entity => {
        entity.HasIndex(u => u.Email).IsUnique(); // Email egyedi index
        entity.Property(u => u.Role).HasDefaultValue("User");
        entity.Property(u => u.CreatedAt).HasDefaultValueSql("GETUTCDATE()");
        entity.Property(u => u.UpdatedAt).HasDefaultValueSql("GETUTCDATE()");
      });

      // Properties tábla konfiguráció
      modelBuilder.Entity<Property>(entity => {
        entity.Property(p => p.IsActive).HasDefaultValue(true);
        entity.Property(p => p.PricePerNight)
```

```
            .HasColumnType("decimal(10,2)");
        });

        // Bookings tábla konfiguráció
        modelBuilder.Entity<Booking>(entity => {
          entity.HasOne(b => b.User)
            .WithMany(u => u.Bookings)
            .HasForeignKey(b => b.UserId)
            .OnDelete(DeleteBehavior.Restrict); // Ne törüljük kaszkádosan

          entity.HasOne(b => b.Property)
            .WithMany(p => p.Bookings)
            .HasForeignKey(b => b.PropertyId)
            .OnDelete(DeleteBehavior.Restrict);

          entity.Property(b => b.Status).HasDefaultValue("Pending");
          entity.Property(b => b.TotalPrice).HasColumnType("decimal(12,2)");

          // Index a szűrési teljesítmény javításához
          entity.HasIndex(b => b.UserId);
          entity.HasIndex(b => b.PropertyId);
          entity.HasIndex(b => b.Status);
        });

        // EventRegistrations - Összetett egyedi index
        modelBuilder.Entity<EventRegistration>(entity => {
          entity.HasIndex(er => new { er.UserId, er.EventId }).IsUnique();

          entity.HasOne(er => er.User)
            .WithMany(u => u.EventRegistrations)
            .HasForeignKey(er => er.UserId)
            .OnDelete(DeleteBehavior.Cascade);

          entity.HasOne(er => er.Event)
            .WithMany(e => e.Registrations)
            .HasForeignKey(er => er.EventId)
            .OnDelete(DeleteBehavior.Cascade);
        });
      }
    }
}
```

Az OnModelCreating metódusban meghatározott konfigurációk az EF Core Fluent API-t alkalmazzák, amely pontosabb és rugalmasabb, mint a Data Annotations attribútumok egyedüli használata. A Fluent API lehetővé teszi olyan konfigurációk megadását, amelyek nem fedhetők le attribútumokkal: például a kaszkád törlési viselkedés (OnDelete), az összetett indexek definiálása vagy az SQL Server-specifikus alapértelmezett értékek (HasDefaultValueSql). Mindkét megközelítés (Data Annotations és Fluent API) kombináltan alkalmazható, ahogy a Nagypapa Házai projektben is: a kötelezőség és maximális hossz validációk az attribútumokkal, a kapcsolatok és indexek a Fluent API-val kerültek definiálásra.

G. Függelék – Követelmény elemzés és projekt specifikáció

A szoftverfejlesztési projektek sikeres kivitelezésének elengedhetetlen előfeltétele a követelmény elemzés elvégzése. A követelmény elemzés során azonosítjuk, dokumentáljuk és priorizáljuk azokat a funkcionalitásokat és megszorításokat, amelyeket a rendszernek teljesítenie kell. Ezt a dokumentumot a Nagypapa Házai projekt indulásakor állítottuk össze, és a fejlesztési folyamat során számos alkalommal felülvizsgáltuk és pontosítottuk. A jelen függelék az eredeti specifikáció végleges, felülvizsgált változatát tartalmazza.

A követelmény elemzés standardizált módszertan szerint készült, funkcionális és nemfunkcionális követelményeket egyaránt tartalmazva. A funkcionális követelmények megadják, hogy a rendszernek mit kell tudnia csinálni; a nemfunkcionális követelmények azt határozzák meg, hogy a rendszernek milyen tulajdonságokkal kell rendelkeznie (teljesítmény, biztonság, karbantarthatóság stb.). Mindkét kategória egyaránt fontos: egy rendszer teljesítheti az összes funkcionális követelményt, de ha lassú, nem biztonságos vagy karbantarthatatlan, akkor az elfogadhatatlan.

G.1. Projekt kontextus és üzleti probléma leírása

A Nagypapa Házai projekt olyan felhasználói szükségletből ered, amelyet a meglévő piaci megoldások nem elégítenek ki: a vidéki és üdülő ingatlanok tulajdonosai számára nem áll rendelkezésre egyszerű, közösség-orientált platform, amelyen koordinálhatják az ingatlanok megosztását – különösen nem pénz helyett cserealap alapján. Az Airbnb és a Booking.com típusú platformok kizárólag monetáris alapon működnek, és hatalmas méretük miatt nem alkalmasak kisközösségi, személyes kapcsolatokon alapuló ingatlanhasználat koordinálására.

A projekt névadója, a „Nagypapa Háza" fogalom, egy ismerős kulturális elemet idéz: a vidéki nagyszülőktől örökölt vagy kapott ingatlanokat, amelyek sokszor üresen állnak, de a megfelelő felhasználóknak értékes pihenési lehetőséget nyújtanának. A platform célja összekötni ezeket az ingatlanokat a potenciális felhasználókkal – egy

modern, digitális közösségi platform formájában, amely megőrzi a személyesebb, bizalmon alapuló ingatlanhasználat szellemét.

Az üzleti probléma két oldalra bontható: az ingatlan-tulajdonos oldaláról nézve a kihasználatlan ingatlan fenntartási költséget jelent hozam nélkül; a felhasználó oldaláról nézve az ingyenes vagy olcsó üdülési lehetőségek hiánya, illetve a kölcsönös előnyökön alapuló csere lehetőségének hiánya a piacon. A Nagypapa Házai mindkét problémát kezeli: az ingatlan-tulajdonos számára csökkenti a kihasználatlanságot, a felhasználónak pedig elérhetővé teszi a megfizethető vagy csere-alapú szállást.

G.2. Funkcionális követelmények

G.2.1. Felhasználókezelési követelmények

A felhasználókezelési modul az alábbi funkcionális követelményeknek kell megfeleljen. Minden követelménynek MoSCoW prioritási értéket rendeltünk: M = Must Have (kötelező), S = Should Have (erősen ajánlott), C = Could Have (jó lenne), W = Won't Have (nem tervezett ebben a verzióban).

ID	Követelmény leírása	Prioritás	Állapot
F-U01	A rendszer lehetővé teszi új felhasználói fiók létrehozását email, jelszó és teljes név megadásával	M	Kész
F-U02	A rendszer ellenőrzi, hogy az email-cím nem foglalt a regisztráció során	M	Kész
F-U03	A jelszavak BCrypt hash-eléssel tárolódnak	M	Kész
F-U04	A felhasználók email és jelszó kombinációval tudnak bejelentkezni	M	Kész
F-U05	A rendszer megkülönböztet User és Admin szerepköröket	M	Kész
F-U06	A bejelentkezett felhasználók módosíthatják saját adataikat	S	Kész

ID	Követelmény leírása	Prioritás	Állapot
F-U07	A felhasználók módosíthatják jelszavukat a jelenlegi jelszó megadásával	S	Kész
F-U08	Elfelejtett jelszó esetén e-mail alapú visszaállítás lehetséges	S	Tervezett
F-U09	Google/Facebook OAuth bejelentkezés lehetséges	C	Tervezett
F-U10	Kétfaktoros hitelesítés (2FA) elérhető	C	Jövőbeli

G.2.2. Ingatlankezelési követelmények

ID	Követelmény leírása	Prioritás	Állapot
F-P01	Az ingatlanok listázódnak a nyilvános oldalon (cím, ár, kapacitás)	M	Kész
F-P02	Szabad szöveges szűrés az ingatlanok névjén és helyszínén	M	Kész
F-P03	Kapacitás szerinti szűrés az ingatlankereső oldalon	M	Kész
F-P04	Ingatlan részletes oldal megjeleníti a teljes leírást és foglalási gombot	M	Kész
F-P05	Adminisztrátor tud új ingatlant hozzáadni az admin felületen	M	Kész
F-P06	Adminisztrátor tud meglévő ingatlant szerkeszteni az admin felületen	M	Kész
F-P07	Adminisztrátor aktiválhatja/deaktiválhatja az ingatlanokat	M	Kész

ID	Követelmény leírása	Prioritás	Állapot
F-P08	Adminisztrátor törölhet ingatlant (aktív foglalás nélkül)	S	Kész
F-P09	Ingatlan képfeltöltés és képgaléria	C	Tervezett
F-P10	Ingatlan értékelési és véleményi rendszer	C	Tervezett
F-P11	Térkép alapú ingatlan keresés	W	Jövőbeli

G.2.3. Foglalási rendszer követelmények

ID	Követelmény leírása	Prioritás	Állapot
F-B01	Bejelentkezett felhasználók foglalást tudnak indítani	M	Kész
F-B02	Foglaláshoz meg kell adni az ingatlan, dátum, vendégszám adatokat	M	Kész
F-B03	A rendszer ütközésvizsgálatot végez a foglalás elküldésekor	M	Kész
F-B04	A felhasználó megadhat különleges kéréseket a foglaláshoz	S	Kész
F-B05	A felhasználó csereajánlatot adhat meg a foglaláshoz	M	Kész
F-B06	A foglalások Függőben státusszal kerülnek be az adatbázisba	M	Kész
F-B07	Adminisztrátor elfogadhatja vagy elutasíthatja a foglalásokat	M	Kész
F-B08	Elutasításkor kötelező indoklás megadása	M	Kész

ID	Követelmény leírása	Prioritás	Állapot
F-B09	A felhasználó lemondhatja saját, még el nem kezdett foglalásait	S	Kész
F-B10	Foglalási naptár megjeleníti a foglaltságot vizuálisan	S	Kész
F-B11	E-mail értesítés a foglalás státuszváltásakor	S	Tervezett
F-B12	Online fizetési integráció (pl. Stripe, Barion)	C	Jövőbeli

G.2.4. Eseménykezelési követelmények

ID	Követelmény leírása	Prioritás	Állapot
F-E01	Adminisztrátor létrehozhat és kezelhet eseményeket	M	Kész
F-E02	Bejelentkezett felhasználók regisztrálhatnak eseményekre	M	Kész
F-E03	A regisztrált résztvevők száma valós időben frissül	S	Kész
F-E04	Betelt eseményekre nem lehet regisztrálni	M	Kész
F-E05	Felhasználók lemondhatják esemény-regisztrációjukat	S	Kész
F-E06	Esemény dátum szerinti szűrés az esemény listán	S	Kész
F-E07	E-mail értesítés esemény törléskor a regisztrált résztvevőknek	S	Tervezett

G.3. Nemfunkcionális követelmények

G.3.1. Biztonsági követelmények

ID	Követelmény	Megvalósítás módja	Állapot
NF-S01	Jelszavak soha nem tárolódnak visszafejthetően	BCrypt hash-elés	Kész
NF-S02	Session adatok HTTP-only cookie-ban tárolódnak	ASP.NET Core Session	Kész
NF-S03	Az admin funkciók csak Admin szerepkörrel érhetők el	RBAC attribútumok	Kész
NF-S04	SQL injection elleni védelem	EF Core paraméteres lekérdezések	Kész
NF-S05	XSS elleni védelem	Razor/Blazor automatikus escaping	Kész
NF-S06	CSRF elleni védelem	SignalR + SameSite cookie	Kész
NF-S07	HTTPS kötelező minden kommunikációban	SSL/TLS, HTTPS redirect middleware	Kész
NF-S08	GDPR-megfelelő adatkezelés	Adatminimalizálás, hozzáférés-jogok	Részleges

G.3.2. Teljesítmény- és skálázhatósági követelmények

ID	Követelmény	Mérték / Célérték	Állapot
NF-P01	Oldalak betöltési ideje	Max. 3 mp fejlesztői gépen	Kész

ID	Követelmény	Mérték / Célérték	Állapot
NF-P02	Ütközésvizsgálat válaszideje	Max. 500ms	Kész
NF-P03	Egyidejű felhasználók száma	Min. 10 (fejlesztői környezetben)	Kész
NF-P04	Adatbázis mérete	Max. 10 GB (SQL Server Express korlát)	Kész
NF-P05	Memóriahasználat	Max. 2 GB RAM fejlesztői gépen	Kész

G.3.3. Karbantarthatósági és minőségi követelmények

ID	Követelmény	Megvalósítás módja	Állapot
NF-M01	A kód jól strukturált, rétegelt architektúrájú	N-Tier: API + Blazor + Shared	Kész
NF-M02	Az API és az UI szétválasztott, független fejleszthető	Külön projektek, REST API	Kész
NF-M03	Az adatbázis séma migrációkkal kezelhető	EF Core Code-First migrációk	Kész
NF-M04	Az API dokumentált és tesztelhető	Swagger UI	Kész
NF-M05	A kód verziókezelő rendszerben van	Git / GitHub	Kész
NF-M06	Egységtesztek alapvető funkciókhoz	xUnit (tervezett)	Tervezett

G.4. Rendszer-határok és külső kapcsolatok

A rendszer-határok meghatározása elengedhetetlen a felelősségi körök tisztázásához. A Nagypapa Házai platform az alábbi külső rendszerekkel áll (vagy fog állni) kapcsolatban: SQL Server Express (adatbázis, lokális), e-mail SMTP szerver (jövőbeli, értesítések küldéséhez), Google/Facebook OAuth provider (jövőbeli, alternatív bejelentkezéshez), Azure felhő infrastruktúra (jövőbeli, felhő-alapú hosting), CDN (jövőbeli, statikus fájlok kiszolgálásához).

A rendszer jelenlegi verziója csak az SQL Server Express-szel áll valódi külső kapcsolatban; minden más integráció tervezett, de nem implementált. A platform teljesen önállóan, internet-kapcsolat nélkül is működik fejlesztői környezetben, ami előnyös fejlesztői és tesztelési szempontból. Produkciós környezetben természetesen internet-kapcsolat szükséges a felhasználók és a szerver közötti kommunikációhoz.

H. Függelék – Fogalomtár (Glosszárium)

Ez a függelék a záródolgozatban előforduló szakkifejezések, rövidítések és technológiai fogalmak definícióit tartalmazza ábécé-sorrendben. A fogalomtár célja, hogy a nem programozói hátterű olvasók is megérthessék a dokumentum tartalmát, és hogy az egységes terminológia alkalmazása megkönnyítse a dokumentáció értelmezését.

Fogalom / Rövidítés	Magyar megfelelő	Definíció
API (Application Programming Interface)	Alkalmazásprogramozási interfész	Szabványos kommunikációs felület, amelyen keresztül különböző szoftverek adatot cserélhetnek egymással.
ASP.NET Core	–	Microsoft nyílt forráskódú, cross-platform webalkalmazás-fejlesztő keretrendszer, a .NET platform részeként.
BCrypt	–	Adaptív kriptográfiai hash-függvény, jelszavak biztonságos tárolásához használatos. A cost factor növelésével lassítható, védelmet nyújtva a brute-force támadások ellen.
Blazor Server	–	Microsoft UI keretrendszer, amellyel a webalkalmazás logikája a szerveren fut, és a böngészővel SignalR WebSocket-en kommunikál.
CORS (Cross-Origin Resource Sharing)	Kereszt-origin erőforrás-megosztás	HTTP mechanizmus, amely lehetővé teszi a weblapoknak, hogy más domain-ből erőforrásokat kérjenek le.

Fogalom / Rövidítés	Magyar megfelelő	Definíció
CSRF (Cross-Site Request Forgery)	Oldalközi kérés-hamisítás	Webes támadási típus, amellyel a támadó a bejelentkezett felhasználó nevében végeztet el nem kívánt műveleteket.
DTO (Data Transfer Object)	Adatátviteli objektum	Egyszerű adatkonténer osztály, amelyet két rendszer (pl. API és kliens) közötti adatcsere céljára terveztek.
EF Core (Entity Framework Core)	–	Microsoft ORM (Object-Relational Mapper) keretrendszer, amely C# objektumok és relációs adatbázistáblák közötti leképezést végez.
Eager Loading	Korai betöltés	EF Core lekérdezési technika, amellyel a navigációs tulajdonságok (kapcsolódó entitások) az elsődleges lekérdezéssel együtt kerülnek betöltésre (.Include() metódus).
FK (Foreign Key)	Idegen kulcs	Adatbázis-oszlop, amely egy másik tábla elsődleges kulcsára (PK) hivatkozik, kapcsolatot teremtve a táblák között.
HTTP Status Code	HTTP állapotkód	Háromjegyű szám, amellyel a szerver jelzi a kérés feldolgozásának eredményét (pl. 200 OK, 404 Not Found, 500 Internal Server Error).
HTTPS	Biztonságos HTTP	HTTP protokoll TLS titkosítással, amely biztosítja az adatátvitel titkosságát és integritását.
Idempotens	–	Olyan művelet, amelynek azonos inputra azonos output az

Fogalom / Rövidítés	Magyar megfelelő	Definíció
		eredménye, és a többszöri végrehajtás nem okoz eltérő hatást (pl. ugyanaz az adatbázis-állapot marad fenn).
LINQ (Language Integrated Query)	Nyelv-integrált lekérdezés	C# nyelvi funkció, amely lehetővé teszi adatforrások (listák, adatbázisok) lekérdezését SQL-szerű szintaxissal, de C# kódként írva.
Middleware	Köztes szoftver	ASP.NET Core pipeline-beli feldolgozó komponens, amely a bejövő HTTP kéréseket és kimenő válaszokat kezeli (pl. hitelesítés, naplózás, CORS).
Migration (EF Core)	Migráció	EF Core mechanizmus, amellyel az adatbázis sémája szinkronban tartható a C# entitásmodellekkel.
MVC (Model-View-Controller)	Modell-Nézet-Vezérlő	Szoftver-architektúrális minta, amely szétválasztja az alkalmazás adatmodelljét, megjelenítési rétegét és vezérlési logikáját.
N+1 lekérdezési probléma	–	Teljesítményproblema EF Core-ban: ha egy lista minden eleméhez külön SQL lekérdezés fut a kapcsolódó adatokért, ahelyett hogy egyetlen JOIN-os lekérdezés hozná be az összes szükséges adatot.
N-Tier architektúra	Többrétegű architektúra	Szoftver-architektúrális megközelítés, amelyben az alkalmazás különböző felelősségi körök (prezentáció, üzleti logika,

Fogalom / Rövidítés	Magyar megfelelő	Definíció
		adathozzáférés) szerint külön rétegekre van osztva.
NuGet	–	A .NET ökoszisztéma csomag-kezelője, amellyel külső könyvtárak (dependencies) adhatók projektekhez.
ORM (Object-Relational Mapper)	Objektum-relációs leképező	Szoftver réteg, amely az objektum-orientált programozási nyelvek és a relációs adatbázisok közötti impedancia-eltérést kezeli.
PK (Primary Key)	Elsődleges kulcs	Az adatbázistábla egyedi azonosítója, amely minden sorban egyedi értéket tartalmaz (általában auto-növekvő egész szám).
RBAC (Role-Based Access Control)	Szerepkör-alapú hozzáférés-vezérlés	Biztonsági modell, amelyben a hozzáférési jogosultságokat szerepkörök (pl. User, Admin) határozzák meg.
REST (Representational State Transfer)	–	Webes szoftver-architektúrális stílus, amely HTTP protokollon alapuló, állapot nélküli, erőforrás-orientált kommunikációt definiál.
Seed adatok	Kezdőadatok	Az adatbázis indulásakor betöltött alapadatok, amelyek lehetővé teszik az alkalmazás azonnali tesztelhetőségét.
Session	Munkamenet	Szerver-oldali mechanizmus, amely a felhasználó-specifikus adatokat (pl. bejelentkezési állapot) tárolja a kérések között.

Fogalom / Rövidítés	Magyar megfelelő	Definíció
SignalR	–	Microsoft valós idejű kommunikációs könyvtár, amely WebSocket (és fallback protokollok) segítségével lehetővé teszi a szerver-kliens duplex kommunikációt.
SQL (Structured Query Language)	Strukturált lekérdező nyelv	Relációs adatbázisok kezelésére szolgáló szabványos lekérdező nyelv.
TLS (Transport Layer Security)	Szállítási réteg biztonság	Kriptográfiai protokoll, amely titkosítja az internetes kommunikációt. A HTTPS alapja.
WCAG (Web Content Accessibility Guidelines)	Webtartalom-akadálymentesítési irányelvek	W3C által kidolgozott irányelvek, amelyek meghatározzák a webes tartalmak akadálymentesítési követelményeit.
WebSocket	–	Kétirányú, állandó TCP-kapcsolatot biztosító hálózati protokoll, amelyet a SignalR is alkalmaz.
XSS (Cross-Site Scripting)	Oldalközi szkriptelés	Webes biztonsági sebezhetőség, amellyel a támadó rosszindulatú JavaScript kódot injektál a weboldalra más felhasználók böngészőjének megtámadása céljából.

A glosszárium nem tart igényt teljességre – csupán a záródolgozatban leggyakrabban előforduló és legkritikusabb fogalmakat tartalmazza. A teljesebb informatikai szószedetért ajánljuk az alábbi forrásokat: MDN Web Docs Glossary (https://developer.mozilla.org/en-US/docs/Glossary), Microsoft .NET Terminology (https://learn.microsoft.com/en-us/dotnet/), és a W3Schools HTML/CSS/JS Reference (https://www.w3schools.com/).

I. Függelék – A dokumentáció verziókezelési naplója

A záródolgozat dokumentációja több fejlesztési cikluson ment keresztül, mielőtt elérte végleges formáját. Ez a függelék kronológiai sorrendben dokumentálja a dokumentáció egyes verzióinak tartalmát, a változtatások okait és az egyes verziók hozzáférhetőségét. A verziókezelés elsősorban a fejlesztőcsapat belső igényét szolgálja, de transzparenciát biztosít a vizsgabizottság számára is.

Verzió	Dátum	Változtatások tartalma	Felelős
v0.1 – Vázlat	2025. november	A dokumentáció szerkezeti vázlatának elkészítése, fejezetek és alfejezetek tervezete	Bota Csongor
v0.2 – Fejlesztői dok. alap	2025. december	Fejlesztői dokumentáció (2. fejezet) első vázlata, architektúra leírás, kódblokkok	Bota Csongor
v0.3 – Felhasználói dok. alap	2026. január	Felhasználói dokumentáció (3. fejezet) első vázlata, képernyőkép-jelzők elhelyezése	Kania Áron Nontawat
v0.4 – Összefoglalás és irodalom	2026. január	Összefoglalás (4. fejezet) és irodalomjegyzék (5. fejezet) első vázlata	Batsukh Temuujin
v0.5 – Peer review	2026. február 1.	Csapaton belüli felülvizsgálat, kiegészítések, egységesítések	Mindenki
v0.6 – Konzulens visszajelzés	2026. február 10.	Konzulens tanár visszajelzéseinek beépítése, hiányos fejezetek kiegészítése	Bota Csongor
v0.9 – Függelékek hozzáadása	2026. február 20.	A–H függelékek hozzáadása (API dok., adatbázis séma, konfigurációk, fejlesztési napló)	Mindenki
v1.0 – Végleges	2026. február 28.	Végső korrektúra, képernyőképek véglegesítése, formázás ellenőrzése, benyújtás	Bota Csongor

A dokumentáció minden verzióját a csapat GitHub repository-jában tároljuk, a projekt forráskódjával együtt. A verziókezelési napló önmagában is értékes audit trail: bemutatja, hogy a dokumentáció nem egy éjszaka alatt született, hanem a fejlesztési folyamattal párhuzamosan, folyamatos iteráción ment keresztül. Ez a megközelítés – a „living documentation" (élő dokumentáció) elv – egyre elterjedtebb a modern szoftverfejlesztési módszertanokban, és a Nagypapa Házai projekt is ezt a megközelítést alkalmazza.

I.1. Ismert problémák a dokumentáció jelenlegi verziójában

Az átláthatóság és az intellektuális becsületesség jegyében ebben az alfejezetben dokumentáljuk azokat az ismert hiányosságokat és tervezett javításokat, amelyek a dokumentáció végleges (v1.0) verziójában még nem kerültek megvalósításra. Minden tétel tartalmazza a prioritást és a tervezett javítás idejét.

#	Ismert probléma / Hiányosság	Prioritás	Tervezett javítás
1	A képernyőkép-jelzők helyén nem valódi képernyőfelvételek szerepelnek	Magas	Az alkalmazás élesítése után, a végleges képernyőfelvételek elkészültekor
2	Az API dokumentáció (A. függelék) nem tartalmazza az összes végpont teljes kódját	Közepes	A következő dokumentációs iterációban
3	Az automatizált tesztek dokumentációja hiányos (unit tesztek, integrációs tesztek)	Közepes	A tesztelési infrastruktúra kiépítése után
4	A GDPR-megfelelőség részletes elemzése nincs elvégezve	Alacsony	Produkciós verzióhoz szükséges, fejlesztői verzióhoz nem kritikus
5	A mobilalkalmazás (.NET MAUI) fejlesztési specifikációja hiányzik	Alacsony	A mobilalkalmazás fejlesztésének megkezdésekor

Nyilatkozat

Alulírott Bota Csongor, Kania Áron Nontawat és Batsukh Temuujin nyilatkozunk, hogy a „Nagypapa Házai – Közösségi Ingatlanhasználati Platform" elnevezésű záródolgozatot önállóan készítettük, és minden felhasznált forrást az irodalomjegyzékben feltüntettük. A munkában idézett, parafrazált vagy adaptált minden gondolatot, kódrészletet és ábrát a forrás megjelölésével tüntettük fel.

Kijelentjük, hogy a záródolgozat szoftverkomponense – a NagypapaHazai.API, NagypapaHazaiBlazor és NagypapaHazai.Shared projekt – saját fejlesztésű kód, amelynek elkészítése során nyílt forráskódú könyvtárakat és keretrendszereket (ASP.NET Core, Entity Framework Core, Bootstrap 5, BCrypt.Net-Next) vettünk igénybe, a megfelelő licencfeltételek tiszteletben tartásával.

Tudomásul vesszük, hogy a záródolgozat plágium-ellenőrzésen eshet át, és nyilatkozunk, hogy annak tartalmát más intézménynek nem nyújtottuk be záróvizsga-munkának. Jelen nyilatkozat valódiságáért felelősséget vállalunk.

Budapest, 2026. február 28.

.....	.....	.....
Bota Csongor	Kania Áron Nontawat	Batsukh Temuujin

A záródolgozat tartalmi összefoglalója

A „Nagypapa Házai – Közösségi Ingatlanhasználati Platform" elnevezésű projekt egy webalapú szoftverrendszer, amely lehetővé teszi vidéki és üdülő ingatlanok közösségi megosztását, foglalások kezelését és közösségi események szervezését. A rendszer egyedi jellemzője a csereajánlat-alapú foglalási lehetőség, amellyel a felhasználók szállásért cserében munkát, terméket vagy szolgáltatást ajánlhatnak fel.

A rendszer ASP.NET Core Blazor Server technológiára épül (.NET 8 LTS), N-Tier architektúrával (NagypapaHazai.API, NagypapaHazaiBlazor, NagypapaHazai.Shared projektek), SQL Server Express adatbázissal és Entity Framework Core ORM-mel. A felhasználói felület Bootstrap 5 stíluskönyvtárra támaszkodik, és teljes körűen reszponzív. A rendszer fejlesztői (localhost) környezetben a https://localhost:7206 és az http://localhost:5011 URL-en érhető el; az API Swagger dokumentációja a https://localhost:44369/swagger URL-en nézhető meg.

A projekt öt fő fejezetre tagolódik: az 1. fejezet a projektbemutatást és a technológiai környezetet ismerteti; a 2. fejezet a fejlesztői dokumentációt tartalmazza (architektúra, adatbázis-séma, API, adatbázis-migráció, biztonsági megvalósítások, tesztelési metodika); a 3. fejezet a felhasználói dokumentációt, a telepítési útmutatót és a hibaelhárítási útmutatót tartalmazza; a 4. fejezet az összefoglalást, a tanulságokat és a köszönetnyilvánítást foglalja magában; az 5. fejezet az irodalomjegyzéket tartalmazza. A záródolgozathoz kilenc függelék is tartozik (A–I), amelyek az API végpontok teljes dokumentációját, az adatbázis séma részleteit, a konfigurációs fájlokat, a fejlesztési naplót, a Blazor komponens architektúrát, az entitásmodelleket, a követelmény elemzést, a glosszáriumot és a verziókezelési naplót tartalmazzák.

A fejlesztőcsapat három tagból állt: Bota Csongor (háttérrendszer, API, architektúra), Kania Áron Nontawat (felhasználói felület, Blazor komponensek, admin modul) és Batsukh Temuujin (adatbázis tervezés, tesztelés, dokumentáció). A projekt körülbelül öt hónapon keresztül zajlott, és körülbelül 385 fejlesztői munkaórát igényelt összesen. A dokumentáció megírása önmagában a becslések szerint 80 munkaórát vett igénybe, ami a projekt összes munkájának körülbelül 21%-a.

A záródolgozat benyújtásának dátuma: 2026. február 28. A dokumentáció és a forráskód a fejlesztőcsapat GitHub repository-jában érhető el, amelyhez olvasási hozzáférés a konzulens tanár és a vizsgabizottság számára biztosítható. A platform demózásához szükséges fejlesztői környezet beüzemelési útmutatója a dokumentáció 3.1. fejezetében található.